UNIVERSIDAD NACIONAL DE LA MATANZA (UNLaM)
DEPARTAMENTO DE INGENIERÍA E INVESTIGACIONES TECNOLÓGICAS

MATERIAL OFICIAL DE PREPARACIÓN INTEGRAL — EDICIÓN AUDITADA 2026

DISEÑO DE SISTEMAS

GUÍA MAESTRA UNIFICADA DE ESTUDIO TEÓRICO-PRÁCTICA

Manual completo orientado a la resolución desde cero de Parciales y Finales según los criterios exactos de corrección de la **Prof. Ing. Analía Marín** y el **JTP Alejandro** (Comisiones 4300 y 1900).

📐 10 CAPÍTULOS INTEGRADOS

- **Cap. 1:** Criterios, Reglas y Consejos de la Profesora
- **Cap. 2:** Guía Completa de los 8 Diagramas UML y 4+1 Vistas
- **Cap. 3:** Los 9 Patrones GRASP, Cohesión, Acoplamiento y MVC
- **Cap. 4:** Los 23 Patrones GoF y Combinaciones Ganadoras
- **Cap. 5:** Metodología Cronometrada de 120 Minutos
- **Cap. 6:** 7 Casos Prácticos Oficiales de Clase Resueltos
- **Cap. 7:** Los 3 Parciales 2026 Resueltos Paso a Paso
- **Cap. 8:** Tablas Comparativas y Unidades Teóricas Finales
- **Cap. 9:** Los 15 Errores Frecuentes y Eliminatorios
- **Cap. 10:** Los 12 Mandamientos y Checklists Pre-Entrega

🎨 CÓDIGO DE COLORES DE LA CÁTEDRA

Convención oficial de la Ing. Analía Marín (`EJEMPLO-Practica-Teoria-v10.pdf`, Págs. 5–10):

AMARILLO **Sustantivos Principales:** Clases candidatas del dominio.

VERDE **Verbos y Pasarela Externa:** Funcionalidad / Métodos de negocio (y clase pasarela externa `sisExtOS` en el Diagrama de Clases).

CELESTE **Datos Simples / Soporte:** Atributos de las clases o clases secundarias.

📚 FUENTES VERIFICADAS

14 Clases transcriptas (28.9 hs) • Guías y Diagramas Oficiales de Analía Marín • Parciales Escaneados 1C y 2C 2026 • Craig Larman (*UML y Patrones*) • Ian Sommerville (*Ingeniería de Software*).

ESTRUCTURA NAVEGABLE DEL DOCUMENTO

ÍNDICE GENERAL DE CAPÍTULOS Y SECCIONES

Cada capítulo incluye además su propio **Índice Detallado de Capítulo** al comienzo de la unidad con acceso rápido a todos los subtemas, diagramas y casos prácticos.

PARTE PRELIMINAR: AUDITORÍA DEL MATERIAL, CRONOGRAMA DE CLASES Y CATÁLOGO DE EXÁMENES Pág. 3

1. ALERTA CRÍTICA SOBRE LAS TRANSCRIPCIONES (LEER ANTES DE ESTUDIAR) 3
2. MAPA CRONOLÓGICO CLASE POR CLASE (VINCULACIÓN DE VIDEOS, TRANSCRIPCIONES Y MATERIALES) 3
3. CATÁLOGO COMPLETO DE PARCIALES Y FINALES DISPONIBLES (Parciales/) 6
4. BIBLIOGRAFÍA Y APUNTES TEÓRICOS COMPLEMENTARIOS (Teoria/) 9
5. REPORTE DE ARCHIVOS DUPLICADOS Y MATERIAL INCOMPLETO / BRECHAS SUBSANADAS 9
6. ESTRUCTURA DE ESTA GUÍA INTEGRAL DE ESTUDIO 10

CAPÍTULO 1: CRITERIOS, CONVENCIONES Y CONSEJOS DE LA PROFESORA (ING. ANALÍA MARÍN) Pág. 11

1. FILOSOFÍA GENERAL DE CORRECCIÓN: "¿QUÉ BUSCA ANALÍA MARÍN EN EL PARCIAL?" 11
2. EL ERROR ELIMINATORIO N° 1: TRAZABILIDAD E INTEGRACIÓN ENTRE DIAGRAMAS 12
3. CRITERIOS ESPECÍFICOS PARA EL DIAGRAMA DE CLASES 12
4. CRITERIOS ESPECÍFICOS PARA EL DIAGRAMA DE SECUENCIA (PUNTO 2 DEL PARCIAL) 14
5. CRITERIOS PARA LOS DEMÁS DIAGRAMAS UML (CU, ACTIVIDAD, COLABORACIÓN, ESTADO, COMPONENTES, DESPLIEGUE) 15
6. CÓMO RESPONDER LAS JUSTIFICACIONES DE PATRONES EN EL PARCIAL 16
7. CHECKLIST RÁPIDO DE CONTROL ANTES DE ENTREGAR EL PARCIAL (LOS 12 MANDAMIENTOS DE ANALÍA) 17

CAPÍTULO 2: GUÍA COMPLETA DE UML (TODOS LOS DIAGRAMAS, NOTACIÓN Y PROCEDIMIENTOS) Pág. 18

PARTE 0: EL MARCO INTEGRADOR — LAS 4+1 VISTAS DE KRUCHTEN Y RUP 18
A. DIAGRAMA DE CLASES (EL CORAZÓN DEL PARCIAL) 18
B. DIAGRAMA DE CASOS DE USO Y ESPECIFICACIÓN (CLASE 03 Y MODELO LABORATORIO) 22
C. DIAGRAMA DE ACTIVIDAD (CLASE 03 Y CLASE 06) 23
D. DIAGRAMA DE SECUENCIA DE DISEÑO (CLASE 04, 05, 06, 07 Y 09) 23
E. DIAGRAMA DE COMUNICACIÓN / COLABORACIÓN (CLASE 04 Y MODELO LABORATORIO PÁG. 18) 23
F. DIAGRAMA DE ESTADOS (CLASE 03, 04, 06 Y FINALES DIC 2025 / JUL 2026) 24
G. DIAGRAMA DE COMPONENTES (CLASE 04, 05, 06 Y PARCIALES 2026) 24
H. DIAGRAMA DE DESPLIEGUE (CLASE 04, 05, 06 Y 11) 24

CAPÍTULO 3: PATRONES GRASP — FUNDAMENTOS, LOS 9 PATRONES, EJEMPLOS Y JUSTIFICACIONES Pág. 26

PARTE 1: FUNDAMENTOS TEÓRICOS DE PATRONES, COHESIÓN Y ACOPLAMIENTO 26
PARTE 2: LOS 9 PATRONES GRASP Y CÓMO JUSTIFICARLOS EN EL PARCIAL (PUNTO 1) 27

CAPÍTULO 4: PATRONES GOF — LOS 23 PATRONES, CLASIFICACIONES, DIAGRAMAS Y APLICACIONES Pág. 29

PARTE 3: CATÁLOGO COMPLETO DE LOS 23 PATRONES GOF (CON EJEMPLOS DE CÁTEDRA) 29
PARTE 4: MANUAL TÁCTICO PARA RESOLVER EL PUNTO 3 DEL PARCIAL (ELECCIÓN DE 2 PATRONES GOF) 32

CAPÍTULO 5: METODOLOGÍA PARA RESOLVER EJERCICIOS Y PARCIALES DESDE CERO Pág. 33

PARTE 1: EL MÉTODO CRONOMETRADO DE 120 MINUTOS PARA RESOLVER EL PARCIAL DESDE CERO 33

CAPÍTULO 6: CASOS PRÁCTICOS OFICIALES DE LA MATERIA (VENTAS, PRÉSTAMOS, LABORATORIO, GIMNASIO, ESTADIOS, SEGURIDAD Y RESERVAS) Pág. 34

6.1. CASO PRÁCTICO 1: SISTEMA DE VENTAS Y LÍNEAS DE VENTA (CASO BASE DE PATRONES GRASP) 34
6.2. CASO PRÁCTICO 2: OTORGAMIENTO DE PRÉSTAMOS BANCARIOS — "BANCOS SOLIVARES" (CLASE 05) 34
6.3. CASO PRÁCTICO 3: LABORATORIO DE ANÁLISIS CLÍNICOS (MODELO OFICIAL DE PARCIAL DE ANALÍA MARÍN) 37
6.4. CASO PRÁCTICO 4: INSCRIPCIÓN A CLASES EN UN GIMNASIO — CASO GYM (CLASES 01 A 05) 40
6.5. CASO PRÁCTICO 5: EMPADRONAMIENTO BIOMÉTRICO Y ACCESO A ESTADIOS DE FÚTBOL — AFA (CLASE 06) 40
PARTE 4-BIS: CASO PRÁCTICO DE TALLER PRE-PARCIAL (CLASE 09 — PRIMERA MITAD [00:00:00 a 01:55:00]) 41
PARTE 5: RESOLUCIÓN DE DIAGRAMAS DE ESTADOS DE FINALES RECIENTES (DIC 2025 Y JULIO 2026) 42

CAPÍTULO 7: PARCIALES 2026 RESUELTOS PASO A PASO (PETCARE, REENCUENTRO/ANIMALES EXÓTICOS Y AGENCIA DE VIAJES) Pág. 43

PARTE 2: RESOLUCIÓN COMPLETA DEL PARCIAL 1C 2026 (COMISIÓN 4300 — ING. ANALÍA MARÍN) 43
PARTE 3: RESOLUCIÓN COMPLETA DEL PARCIAL 1C 2026 (COMISIÓN 1900 — VALERIA Y ANALÍA MARÍN) 45
PARTE 4: RESOLUCIÓN COMPLETA DEL PARCIAL 2C 2026 (COMISIÓN 1900 — TEMA 1) 48

CAPÍTULO 8: TABLAS COMPARATIVAS Y SÍNTESIS DE UNIDADES TEÓRICAS FINALES Pág. 52

8.0. MATRICES Y TABLAS COMPARATIVAS RÁPIDAS PARA EL EXAMEN 52
PARTE 6: SÍNTESIS DE LAS UNIDADES TEÓRICAS FINALES Y BANCO DE PREGUNTAS DE EXAMEN 53

CAPÍTULO 9: CATÁLOGO DE ERRORES FRECUENTES Y ELIMINATORIOS EN EL PARCIAL Pág. 56

9.1. LOS 15 ERRORES MÁS FRECUENTES Y ELIMINATORIOS EN EL PARCIAL (Y CÓMO EVITARLOS) 56

CAPÍTULO 10: CONSEJOS FINALES Y CHECKLISTS PARA EL EXAMEN Pág. 57

10.1. LOS 12 MANDAMIENTOS DE ANALÍA MARÍN (CHECKLIST PRE-ENTREGA EN EL AULA) 57
10.2. ESTRATEGIA DE DISTRIBUCIÓN DEL TIEMPO EN EL EXAMEN (120 MINUTOS) 57

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

PARTE PRELIMINAR: AUDITORÍA DEL MATERIAL, CRONOGRAMA DE CLASES Y CATÁLOGO DE EXÁMENES

📑 ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO

1. ALERTA CRÍTICA SOBRE LAS TRANSCRIPCIONES (LEER ANTES DE ESTUDIAR) Pág. 3
2. MAPA CRONOLÓGICO CLASE POR CLASE (VINCULACIÓN DE VIDEOS, TRANSCRIPCIONES Y MATERIALES) Pág. 3
3. CATÁLOGO COMPLETO DE PARCIALES Y FINALES DISPONIBLES (Parciales/) Pág. 6
4. BIBLIOGRAFÍA Y APUNTES TEÓRICOS COMPLEMENTARIOS (Teoria/) Pág. 9
4.1. Libros de Cabecera (Teoria/Libros/) Pág. 9
4.2. Apuntes de Cátedra y Resúmenes en Teoria/ y Teoria/ Drive/ Pág. 9
5. REPORTE DE ARCHIVOS DUPLICADOS Y MATERIAL INCOMPLETO / BRECHAS SUBSANADAS Pág. 9
5.1. Archivos Duplicados Exactos (No perder tiempo abriéndolos dos veces) Pág. 9
5.2. Brechas del Material Original que hemos cubierto en esta Guía Integral Pág. 10
6. ESTRUCTURA DE ESTA GUÍA INTEGRAL DE ESTUDIO Pág. 10

1. ALERTA CRÍTICA SOBRE LAS TRANSCRIPCIONES (LEER ANTES DE ESTUDIAR)

Durante la auditoría cruzada entre el código generador (`Transcripcion/build_full_document.py`), el consolidado `Transcripcion/Clases_Diseno_Transcritas.md` y las **transcripciones literales crudas** en `Transcripcion/transcripciones/*.txt`, se detectó un hallazgo fundamental:

1. **Las 14 transcripciones literales en `Transcripcion/transcripciones/*.txt` (y sus `.json` con marcas de tiempo `[hh:mm:ss]`) son 100% fieles y completas** respecto a los videos originales (28.9 horas de audio transcriptas con Whisper `large-v3`).
2. **PRECAUCIÓN CON `Transcripcion/Clases_Diseno_Transcritas.md`:** La primera mitad de ese archivo contiene "resúmenes ejecutivos por clase" que fueron **hardcodeados sintéticamente** dentro del script `build_full_document.py` y **NO coinciden con el contenido real dictado en varias clases**:
   - En `Clases_Diseno_Transcritas.md` dice que la **Clase 06 (24/09)** trata sobre *"Un club deportivo con apto médico y cuota al día"*, cuando en la transcripción real (`06_20260924_082126.txt`) y en la carpeta `Clases/Clases/06/Estadio/` la clase trata íntegramente sobre el **Caso Estadios de Fútbol (AFA) — Empadronamiento Biométrico (Huella + Iris) y Pre-control Policial**.
   - En `Clases_Diseno_Transcritas.md` dice que la **Clase 08 (08/10)** resuelve un caso de *"Envíos con Strategy y Decorator"*, cuando en realidad (`08_20261008_082230.txt` y `ejerFutbol-2026-10-08.pdf`) resuelven el **Caso Juego de Fútbol con GoF** (Observer para la pelota, Strategy para táctica de equipo, Mediator, Command, Memento, Decorator para habilidades/lesiones, Flyweight para hinchas) y el **Caso Banco Olivares** (Facade + Adapter para VISA/Veraz).
   - En `Clases_Diseno_Transcritas.md` dice que la **Clase 09 (15/10)** trata sobre *"Historias Clínicas con Memento"*, cuando en realidad (`09_20261015_082342.txt`) es la **clase clave de repaso pre-parcial** donde corrigen en vivo dos modelos de parcial completos: **Seguridad Ciudadana (Alertas/Denuncias)** y **Veterinaria "PetCare" (Parcial 1C 2026)**.

🎓 CRITERIO Y RECOMENDACIÓN DE LA PROFESORA

Regla aplicada en esta Guía Integral: Toda esta guía ha sido construida leyendo directamente las **transcripciones literales (`Transcripcion/transcripciones/*.txt`)**, las **diapositivas y documentos originales de la Prof. Analía Marín (`Clases/Clases/`)** y los **parciales reales (`Parciales/`)**, descartando cualquier resumen sintético inexacto.

2. MAPA CRONOLÓGICO CLASE POR CLASE (VINCULACIÓN DE VIDEOS, TRANSCRIPCIONES Y MATERIALES)

La cursada consta de **11 fechas de clase** (desde el 20/08/2026 hasta el 29/10/2026). Existen **15 archivos `.mp4`** en `Clases-videos/` que corresponden a **14 grabaciones únicas** (ya que el video de la Clase 08 está duplicado como `Copy.mp4`).

Clase y Fecha	Modalidad y Duración	Archivo de Video (`Clases-videos/`)	Transcripción Literal (`Transcripcion/transcripciones/`)	Materiales de la Profesora (`Clases/Clases/`)	Temas Dictados y Casos Prácticos Trabajados
Clase 01 (Parte 1) 20/08/2026	Virtual 115.1 min	`Clase 01-20260820_082357-Grabación de la reunión.mp4`	`01_20260820_082357.txt` `01_20260820_082357.json`	`01/01-Diseño de Sistemas v2.pdf` (Págs. 1–48) `01/2026-08-20.txt` `01/TP_HTML_JavaScript_Formularios_v2.docx`	Presentación de la materia y reglas de cursada. Ingeniería de Software (Pressman/Sommerville): Compromiso con la calidad, Procesos, Métodos y Herramientas. Proceso Unificado (**RUP**): fases (Inicio, Elaboración, Construcción, Transición), disciplinas y las **4+1 Vistas de Kruchten** (Casos de Uso, Lógica/Diseño, Procesos, Implementación/Componentes, Despliegue).
Clase 01 (Parte 2) 20/08/2026	Virtual 80.1 min	`Clase 01-20260820_102934-Grabación de la reunión.mp4`	`01_20260820_102934.txt` `01_20260820_102934.json`	`01/01-Diseño de Sistemas v2.pdf` (Págs. 49–73)	Metodologías Ágiles: Manifiesto Ágil, **Scrum** (Roles, Eventos, Artefactos), **Kanban**, **Historias de Usuario (HU)** con criterios INVEST y tareas SMART, Product Backlog y Sprint Backlog. Introducción al **Caso Práctico Gimnasio (GYM)** en equipos.

Clase y Fecha	Modalidad y Duración	Archivo de Video (`Clases-videos/`)	Transcripción Literal (`Transcripcion/transcripciones/`)	Materiales de la Profesora (`Clases/Clases/`)	Temas Dictados y Casos Prácticos Trabajados	
Clase 02 27/08/2026	Virtual 187.7 min	`Teams meeting-20260827_081937-Grabación de la reunión.mp4`	`02_20260827_081937.txt` `02_20260827_081937.json`	`02/02-DiseñoUX-Alumno-2Q-2026-JUEVES.pdf` (19 págs.) `02/02-Diseño Calidad- UNLaM-2026Agosto.pdf` (17 págs.) `02/Backlog-GuionTestUX-2026-2Q-Jueves.xlsx` `02/Temas-Clase02-2026-08-27.txt`	**Diseño de Interfaz de Usuario y UX**: 3 Reglas Doradas de Theo Mandel, Principios de Tognozzi, Diseño Centrado en el Usuario, **User Personas**, **Pruebas de Usabilidad (7 pasos)**, **Heat Maps (Mapas de calor)**, Consistencia. Categorización de HU y **Diagrama de Navegación**. Práctica de Roleplay de Test de Usabilidad entre grupos (Caso GYM). **Calidad en el Diseño**: David Garvin, Robert Glass, Factores de McCall y Atributos de Calidad (Performance, Seguridad, Mantenibilidad, Escalabilidad, Disponibilidad, etc.).	
Clase 03 (Parte 1) 03/09/2026	Virtual 102.7 min	`Clase 03-20260903_081406-Grabación de la reunión 1.mp4`	`03_20260903_081406.txt` `03_20260903_081406.json`	`03/03-CU-GYM-2026-09-03.docx` `03/03-FORMATO-VACIO-2026-2c.docx` `03/Temas-Clase03-2026-09-03-JUEVES.txt` `04/Gym/GUIA-EjercicioGYM-Estudiantes-2026- V01.docx`	De Historias de Usuario a **Casos de Uso (CU)**. Diagrama de CU (`<<include>>` vs `<<extend>>`, actores primarios y secundarios). **Especificación de CU**: Forma 1 (párrafo) y **Forma 2 (Tabla de 2 columnas: Actor \	Sistema) **con Precondiciones, Postcondiciones, Normal y Flujos Alternativos/Exc** Diagrama de Acti **(nodos de acció decisión, fork/jo concurrencia, swimlanes/carril Introducción a** D de Estados**.
Clase 03 (Parte 2) 03/09/2026	Virtual 89.2 min	`Clase 03-20260903_101348-Grabación de la reunión.mp4`	`03_20260903_101348.txt` `03_20260903_101348.json`	*(Mismos archivos de carpeta `03/` y `04/Gym/`)*	Repaso profundo del **Paradigma Orientado a Objetos**: Clase vs Objeto, Encapsulamiento, Visibilidad (`+`, `-`, `#`), Herencia, Polimorfismo, **Clase Abstracta (`<<abstract>>`) vs Interfaz (`<<interface>>`)**, Agregación (rombo blanco) vs Composición (rombo negro). **Técnica de análisis de texto con código de colores en vivo** sobre el Caso Gimnasio y Laboratorio (`EJEMPLO-Practica-Teoria-version-10.pdf`, Págs. 5–6 y 8): Amarillo = Clases candidatas (sustantivos principales), Verde = Funcionalidad/Comportamiento (verbos/métodos), Celeste = Atributos o clases secundarias.	
Clase 04 10/09/2026	**Presencial (Aula 136)** *(Sin video)*	*(No hay grabación por ser clase presencial en UNLaM)*	*(Cubierta por las diapositivas de `04-Presencial/` y repasada al inicio de Clase 05)*	`04-Presencial/UNLaM GRASP-original-marin.pdf` (77 págs.) `04-Presencial/Temas-2026SEP10.txt`	**Diagramas de Interacción**: Secuencia vs Colaboración/Comunicación. **Diagrama de Estados** (eventos, condiciones de guarda, acciones, `entry` / `exit`, destrucción). **Diagrama de Componentes** (interfaces provistas/requeridas, puertos, conectores assembly y delegation, y convención de cajas anidadas con `- - - >`). **Diagrama de Despliegue** (nodos, artefactos, protocolos y separación de dominios/ambientes). **Patrones GRASP** (Experto, Creador, Bajo Acoplamiento, Alta Cohesión, Controlador, Polimorfismo, Fabricación Pura, Indirección, Variaciones Protegidas), **Diseño Modular** y **MVC / MVVM**.	
Clase 05 (Parte 1) 17/09/2026	Virtual 105.2 min	`Clase 05-20260917_081823-Meeting Recording.mp4`	`05_20260917_081823.txt` `05_20260917_081823.json`	`05/EJEMPLO-Practica-Teoria-version-10.pdf` (33 págs.) `05/Temas-2026SEP17.txt` `05/Banco/Banco-diagDespliegue.png` y `Banco-diagDesplieguev2.png` `05/Banco/Banco-diagComponente.png`	Presentación del documento maestro `EJEMPLO-Practica-Teoria-version-10.pdf` (**Caso Laboratorio de Análisis Clínicos**). Resolución en vivo del **Caso "Bancos Solivares"** (Otorgamiento de Préstamos Hipotecarios y Prendarios): **Diagrama de Despliegue** (separación en 4 ambientes: Cliente externo, Servicios Externos VISA/Veraz vía XML/JSON, Sucursal, Sede Central) y **Diagrama de Componentes**.	

Clase y Fecha	Modalidad y Duración	Archivo de Video (`Clases-videos/`)	Transcripción Literal (`Transcripcion/transcripciones/`)	Materiales de la Profesora (`Clases/Clases/`)	Temas Dictados y Casos Prácticos Trabajados
Clase 05 (Parte 2) 17/09/2026	Virtual 93.9 min	`Clase 05-20260917_101702-Grabación de la reunión.mp4`	`05_20260917_101702.txt` `05_20260917_101702.json`	`05/Banco/Banco-diagClase.png` `05/Banco/Banco-diagSecuencia.png` `05/Banco/Banco-diagEstado.png` `05/Banco/Banco-diagComunicacion.png` `05/ejercicioGYM/*.png`	Continuación **Caso "Bancos Solivares"**: construcción del **Diagrama de Clases de Diseño** (`C_EvaluacionP`, `C_prestamo`, `C_Cliente`, jerarquía `Producto` → `Prestamo` / `Cuenta`, `Bien` → `Inmueble` / `Automotor`, composición de `Cuota` y `Requisito`) y **Diagrama de Secuencia** de evaluación crediticia. Corrección en vivo de los diagramas del **Caso Gimnasio (GYM)** hechos por los alumnos (`ejercicioGYM/`).
Clase 06 (Parte 1) 24/09/2026	Virtual 141.5 min	`Clase 06-20260924_082126-Grabación de la reunión.mp4`	`06_20260924_082126.txt` `06_20260924_082126.json`	`06/Temas-2026SEP24.txt` `06/Estadio/2026Sep24-CasoEstadios.mdj` (StarUML) `06/Estadio/CU.png`, `Actividad.png`, `Componente.png`, `ComponenteVersion2.png`, `Despliegue.png`, `Despliegue2.png`, `Clase.png`, `Clase2.png`, `Estado.png`, `Secuencia.png`	Práctica integradora tipo parcial desde cero: **Caso "Estadios de Fútbol / AFA (Empadronamiento Biométrico con Huella e Iris y Pre-control Policial)"**. Construcción paso a paso de: 1) Diagrama de CU, 2) Diagrama de Actividad, 3) Diagrama de Componentes, 4) Diagrama de Despliegue (Tótem con cámaras/lectores, Servidor Local, Servidor Central AFA, Renaper/Policía), 5) Diagrama de Clases (`c_empadronar`, `c_ingreso`, `Padron`, `Aficionado`, `Huella`, `Iris`, `Entrada`), 6) Diagrama de Estados de `Aficionado`.
Clase 06 (Parte 2) 24/09/2026	Virtual 19.7 min	`Clase 06-20260924_114136-Grabación de la reunión.mp4`	`06_20260924_114136.txt` `06_20260924_114136.json`	*(Mismos archivos de carpeta `06/Estadio/`)*	Puesta en común de los grupos sobre el Caso Estadios. Explicación de **Multiobjetos / Nóminas / Padrón** en el Diagrama de Secuencia y su correspondencia obligatoria en el Diagrama de Clases.
Clase 07 (Parte 1) 01/10/2026	Virtual 118.4 min	`Clase Diseño-20261001_082247-Grabación de la reunión.mp4`	`07_20261001_082247.txt` `07_20261001_082247.json`	`07/GOF-2Q-2026.pdf` (Págs. 1–29) `07/Temas-2026-10-01.txt`	Consultas clave de diseño: **Visibilidad** (atributo, parámetro, local, global), cuándo un mensaje de secuencia lleva línea de asociación en clases, `create()` en secuencia, y diferencia entre **Controller y Gestor**. Introducción a MDA (CIM, PIM, PSM) y **Patrones GoF**: historia (Christopher Alexander, Gang of Four) y los **5 Patrones Creacionales** (*Singleton, Factory Method, Abstract Factory, Builder, Prototype*).
Clase 07 (Parte 2) 01/10/2026	Virtual 75.2 min	`Clase Diseño-20261001_103650-Grabación de la reunión.mp4`	`07_20261001_103650.txt` `07_20261001_103650.json`	`07/GOF-2Q-2026.pdf` (Págs. 30–44)	**Los 7 Patrones GoF Estructurales**: *Adapter* (ejemplo de enchufes y servicios externos legacy), *Bridge*, *Composite* (estructura de árbol Todo-Parte, ej. menús, carpetas, combos de productos), *Decorator* (agregado dinámico de responsabilidades sin subclasificar), *Facade* (interfaz unificada para un subsistema complejo), *Flyweight* (optimización de memoria compartiendo estado intrínseco), *Proxy* (intermediario de control de acceso, virtual, remoto, protección).

Clase y Fecha	Modalidad y Duración	Archivo de Video (`Clases-videos/`)	Transcripción Literal (`Transcripcion/transcripciones/`)	Materiales de la Profesora (`Clases/Clases/`)	Temas Dictados y Casos Prácticos Trabajados
Clase 08 08/10/2026	Virtual 188.7 min	`Clase-20261008_082230-Grabación de la reunión.mp4` *(y su duplicado `...Copy.mp4`)*	`08_20261008_082230.txt` `08_20261008_082230.json`	`08/000 Diseño de patrones GOF-2026-10-08 v4.pdf` (71 págs.) `08/ejerFutbol-2026-10-08.pdf` (33 págs.) `08/Temas-2026-10-08.txt`	**Los 11 Patrones GoF de Comportamiento**: *Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor*. Resolución interactiva del **Caso Juego de Fútbol (`ejerFutbol-2026-10-08.pdf`)** aplicando Observer, Strategy, Mediator, Command, Memento, Decorator y Flyweight; y **Caso Banco Olivares** aplicando Facade + Adapter para conectarse a VISA y Veraz.
Clase 09 15/10/2026	Virtual 227.0 min	`Clase 1510-20261015_082342-Grabación de la reunión.mp4`	`09_20261015_082342.txt` `09_20261015_082342.json`	*(Sin subcarpeta en `Clases/`; trabajan sobre los PDFs de `Parciales/`)*	**CLASE FUNDAMENTAL DE REPASO Y CORRECCIÓN PRE-PARCIAL.** Corrección en vivo de resoluciones de alumnos sobre: 1) **Caso Seguridad Ciudadana (Alertas/Denuncias, evaluación de falsas denuncias, incidentes, patrullas, mapa de calor)** y 2) **Parcial 1C 2026 Comisión 4300 — Red de Veterinarias "PetCare" (Gato Homero, Turnos, Sucursales Veterinaria, Profesionales, Vacunas, Recetas, Estudios, Historia Clínica, Recordatorios y Notificaciones)**. La profesora detalla exactamente qué busca al corregir, qué errores restan puntos y cómo aplicar GoF sobre el Diagrama de Clases.
(Clase 10) 22/10/2026	Presencial	*(Sin video: Fecha de examen)*	*(Sin transcripción: Fecha de examen)*	*(Evaluación presencial)*	**Primer Parcial Práctico Presencial.**
Clase 11 29/10/2026	Virtual 131.1 min	`Clase 11-20261029_082145-Grabación de la reunión.mp4`	`11_20261029_082145.txt` `11_20261029_082145.json`	`11/soa 2026-v10.pdf` (12 págs.) `11/Testing UNLaM.ppt` `11/str-alumno.pdf` (13 págs.) `11/preguntasGuiaAutoEstudio-2.docx` `11/clase2026-10-29.txt`	**Teoría de Unidades Finales**: 1) **SOA (Arquitectura Orientada a Servicios)**, ESB (Enterprise Service Bus), BPM, BAM, **Microservicios** vs Monolito vs SOA, **APIs REST** y **Swagger/OpenAPI**. 2) **Testing de Software**: Verificación vs Validación, Caja Blanca vs Caja Negra, Pruebas Unitarias, Integración, Regresión, Humo, Estrés, Aceptación. 3) **Sistemas de Tiempo Real (STR)**: Hard vs Soft Real Time, Sensores (analógicos, digitales, lógicos), Actuadores, Planta y Controlador. Ejercicio grupal de **Domótica / IoT** con Diagrama de Despliegue.

3. CATÁLOGO COMPLETO DE PARCIALES Y FINALES DISPONIBLES (`Parciales/`)

En la carpeta `Parciales/` (y sus subcarpetas `drive-download-...` y `Parcial Unido-...`) hay **61 archivos**. Muchos son copias o re-empaquetados en PDF de los mismos exámenes. A continuación se presenta la **tabla depurada de los 12 exámenes únicos** que existen en el repositorio, indicando cuáles tienen resolución y dónde están:

#	Período y Tipo	Comisión / Docente	Caso Práctico / Tema Evaluado	Archivos Correspondientes en `Parciales/`	¿Tiene Resolución?
1	**Parcial 1C 2026** *(Formato actual exacto)*	**Com. 4300 Ing. Analía Marín**	**Caso Red de Veterinarias "PetCare"** (Búsqueda de sucursales/horarios de `Veterinaria`, `Usuario`, `Mascota`, `Turno`, `Profesional`, `HistoriaClinica`, `Consulta`, `Diagnostico`, `Tratamiento`, `Receta`, `Estudio`, `VacunaAplicada`, `Recordatorio`, notificaciones por WhatsApp/Mail + Diagrama de Componentes inicial + Diagrama de Estados de `Consulta Veterinaria` + Pantalla del gato siamés **"Homero"**). *(Nota manuscrita: `max: 4 controladores`).* **Pide:** 1) DC completo integrando el Diagrama de Estados + al menos 2 GRASP justificados; 2) DS de `"Guardar Consulta"` para el escenario de `Homero` (06/06/2026, 13 años, 5.2 kg, dueño Juan Pérez, vacuna Rabia, receta Pastillas desparasitarias, diagnóstico SANO, recordatorio 1 año); 3) Aplicar 2 patrones GoF de distinta categoría sobre el DC.	- `Parciales/1. [Parcial 1][1C2026][4300][Analia][][].pdf` - `Parciales/1. [Parcial 1][1C2026][4300][Analia][][] .pdf` *(duplicado)* - `Parciales/WhatsApp Image 2026-10-02 at 21.24.20.jpeg` y `21.24.21.jpeg` *(fotos del mismo parcial)*	**Sí (Oral en Clase 09 + escrita en esta Guía).** Fue resuelto y corregido en vivo durante más de 2 horas por Analía y Alejandro en `Transcripcion/transcripciones/09_20261015_082342.txt` (`[01:55:00]` a `[03:45:00]`). Además, incluimos la resolución completa diagramada en `04_METODOLOGIA_Y_PARCIALES_RESUELTOS_PASO_A_PASO.md` .
2	**Parcial 1C 2026** *(Formato actual exacto)*	**Com. 1900 Valeria y Analía Marín**	**Caso "Solución de Reencuentro y Adopción de Mascotas" (y Alerta de Animales Exóticos)** (Ingreso de mascotas por *Canal Presencial* en Refugio o *Canal Autogestionado* en Hogar Temporal con `UbicacionHallazgo` ; *Módulo de Reclamo* "Soy el dueño" que agenda `Encuentro` ; *Alerta de Reclamos duplicados* que genera `Conflicto` y pasa a `"En Disputa Legal"` ; *Alerta sobre encuentro de animales exóticos* `AnimalExotico` notificando a **Dirección de Flora y Fauna** y **Policía Ecológica** + Diagrama de Componentes Base). **Pide:** 1) DC completo (**usando solo 4 controladores**); 2) **Modificar el Diagrama de Componentes** según la limitación de 4 controladores; 3) **DS del Escenario: Denuncia de Animal Exótico** (ciudadano avista un **Leopardo** en Rivadavia, Morón, Bs.As., cálculo de peligrosidad `ROJO/NARANJA/AMARILLO/VERDE` y notificación automática a Flora y Fauna y Policía Ecológica); 4) 2 patrones GoF de distinta categoría *(prohíbe Singleton y Facade)*.	- `Parciales/2. -Parcial 1--1C2026--1900--Valeria y Analia-----.pdf` - `Parciales/drive-download-20261111T183304Z-1-001/2. [Parcial 1][1C2026][1900][Valeria y Analia][][].pdf` *(duplicado)*	**No tenía resolución en la carpeta.** Se incluye **resolución completa paso a paso** en el documento `04_METODOLOGIA_Y_PARCIALES_RESUELTOS_PASO_A_PASO.md` .
3	**Parcial 2C 2026** *(Formato actual exacto)*	**Com. 1900 Analía / Cátedra**	**Caso "Agencia de Viajes" (TEMA 1)** (Contratación de `PaqueteTuristico` con destino único, duración, vigencia y servicios de `Transporte` , `Alojamiento` y `Excursion` es opcionales; verificación de disponibilidad, cálculo de precio total según pasajeros, fecha, servicios incluidos y excursiones extras; pago por tarjeta/transferencia y `Reserva` con estados `pendiente de pago` , `confirmada` , `cancelada` , `finalizada`). **Pide:** 1) **Analizar críticamente y mejorar el Diagrama de Componentes defectuoso** impreso en la hoja; 2) **DS de `"realizar Pago"` mejorando el escenario** a partir del paso 4; 3) **Integrar el Diagrama de Estados de `Reserva`** provisto al modelo; 4) **DC completo usando solo 4 controladores**, aplicando un patrón GoF para el cálculo del precio según políticas y un 2° patrón GoF para otro problema de diseño.	- `Parciales/drive-download-20261111T183304Z-1-001/3. [Parcial 1][2C2026][1900][][][].pdf` - `Parciales/1.jpeg` y `2.jpeg` - `Parciales/WhatsApp Image 2026-10-19 at 22.28.19.jpeg` y `22.28.18.jpeg` *(fotos del mismo parcial)*	**No tenía resolución en la carpeta.** Se incluye **resolución completa paso a paso** en el documento `04_METODOLOGIA_Y_PARCIALES_RESUELTOS_PASO_A_PASO.md` .
4	**Modelo Oficial de Parcial / Guía Práctica-Teórica 1C 2025 - 2026**	**Ing. Analía Marín**	**Caso "Laboratorio de Análisis Clínicos"** (Pacientes, Órdenes médicas, Turnos, Estudios por especialidad/categoría, Extracción de muestras con código de barras, Bioquímicos, Resultados y Valores de Referencia, Pagos con Obra Social/Prepaga/Particular). Incluye además **9 páginas de Preguntas y Respuestas teóricas/prácticas** escritas por Analía Marín en texto azul.	- `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` **(Versión más completa: 33 págs.)** - `Parciales/Parcial1C25-Resuelto.pdf` (26 págs.) - `Parciales/drive-download-20261111T183937Z-1-001/Modelo de parcial.pdf` (26 págs.) - `Parciales/Parcial Unido.../6. [Parcial][1C2025][][][Resuelto][].pdf` (26 págs.)	**¡SÍ! Resolución oficial completa hecha por la Prof. Analía Marín** (incluye Especificación de CU, Diagrama de CU, Actividad, Componentes, Despliegue, Clases de Diseño, Secuencia, Colaboración, Estados y Q&A comentado).

#	Período y Tipo	Comisión / Docente	Caso Práctico / Tema Evaluado	Archivos Correspondientes en `Parciales/`	¿Tiene Resolución?
5	**Final Julio 2026 (Llamado A y Mesa B)**	**Cátedra Diseño**	**Llamado A (16/07/2026):** Teoría (Acoplamiento de datos/control/global, Atributos de calidad, GoF Creacional/Estructural/Comportamiento) + Práctica: **Sistema de Gestión de Reclamos Municipales** (DC con 1 patrón de cada tipo: Creacional, Estructural y Comportamiento + DS). **Mesa B (30/07/2026):** Teoría (2 vistas de Kruchten, Factores de calidad de McCall, GoF Adapter, Bridge, State, Observer, Abstract Factory) + Práctica: **Diagrama de Estado de un Artículo de Supermercado** (pedido a proveedor, ingreso a stock, pedido de cliente, entrega, stock mínimo, devolución).	- `Parciales/Final_Diseno_Julio_2026.pdf` (3 págs.)	**Sí (Resuelto en el mismo PDF).** El documento incluye las respuestas teóricas desarrolladas, el Diagrama de Clases con Factory Method + Composite + Observer, el Diagrama de Secuencia y el Diagrama de Estados del Artículo.
6	**Final 17/12/2025 (2C 2025)**	**Cátedra Diseño**	**Parte Teórica (6 pts):** 1) Vistas de arquitectura; 2) Atributos de calidad para e-commerce con picos de demanda; 3) Patrones GoF para: a) familias de objetos (Abstract Factory), b) estado de objeto (State/Memento), c) algoritmos intercambiables (Strategy), d) desacoplar abstracción de implementación (Bridge), e) controlar acceso (Proxy). **Parte Práctica (4 pts): Diagrama de Estado de Reserva de Paquete Turístico** (creación, vencimiento 15 min, pago, confirmación, cancelación con/sin penalidad, check-in, finalización).	- `Parciales/Consigna.jpg` - `Parciales/Final_-_2C2025.jpg` *(mismo examen)*	**No tenía resolución en la carpeta.** Respondido y resuelto en `04_METODOLOGIA_Y_PARCIALES_RESUELTOS_PASO_A_PASO.md`.
7	**Parcial 2C 2025 (Teórico-Práctico)**	**Cátedra Diseño**	**Caso "Sistema de Gestión de Préstamos Bancarios"** (Evaluación crediticia automática, cálculo de cuota por sistema francés/alemán, firma digital y desembolso, gestión de mora con recordatorios, intereses punitorios y bloqueo). Pide DC + DS + 5 preguntas teóricas (Requerimientos de dominio, Kruchten, Diseño de interfaz, Cohesión/Acoplamiento, Patrón GoF).	- `Parciales/Parcial 2C2025.jpg`	**Sí (Caso equivalente a "Bancos Solivares" resuelto en Clase 05)** + preguntas teóricas respondidas en esta Guía.
8	**Parcial 1C 2025 (Hojas manuscritas 1, 2 y 3)**	**Cátedra Diseño**	**Caso Turnos / Pacientes / Profesionales / Historias Clínicas** + **Teoría manuscrita** (5 elementos de un patrón de diseño, Patrones GRASP, Vistas de Kruchten, Diagrama de Secuencia vs Colaboración, Cohesión y Acoplamiento).	- `Parciales/Parcial Unido.../1.pdf`, `2.pdf`, `3.pdf` - `Parciales/drive-download-20261111T183937Z-1-001/1C2025 (1).png` a `(6).png` - `Parciales/0c34a650...png` y demás `.png` sueltos en `Parciales/` *(duplicados exactos)*	**Sí (Manuscritos resueltos).** Son fotos de resoluciones manuscritas de alumnos. *(Nota: leerlas aplicando el filtro de las correcciones de Analía Marín en `01_CRITERIOS_Y_CONSEJOS_DE_LA_PROFESORA.md`).*
9	**Parcial 1C 2024**	**Cátedra Diseño**	**Parte Teórica:** 1) 5 elementos de un patrón (Nombre, Problema, Solución, Consecuencias, Aplicabilidad); 2) 4 Patrones GRASP; 3) Modelo 4+1 de Kruchten; 4) Secuencia vs Colaboración; 5) Cohesión y Acoplamiento. **Parte Práctica: Caso "Consultorios Médicos / Gestión de Turnos"** (Reserva de turnos, confirmación 48hs antes, reprogramación, cancelación, recepción en consultorio, registro en HC y recetas electrónicas). Pide DC + DS + Diagrama de Estados del Turno.	- `Parciales/Parcial Primer Cuatrimestre 2024.pdf` - `Parciales/drive-download-20261111T183937Z-1-001/1C2024.pdf` *(duplicado)* - `Parciales/Parcial Unido.../4.[Parcial][1C2024][][][Resuelto][]/` (`Consigna.pdf`, `Ejercicio Turnos.mdj`, `Resolución Teoría.pdf`, `Diagrama de Clases.jpg`, `Diagrama de Secuencia.jpg`, `Diagrama de Estados.jpg`, `Notificaciones - Strategy + Observer + Factory Method.jpg`)	**¡SÍ! Totalmente resuelto** (incluye proyecto StarUML `.mdj`, imágenes exportadas y PDF de teoría resuelta).
10	**Parcial 2023**	**Cátedra Diseño**	**Parte Teórica:** 1) 4+1 Vistas de Kruchten; 2) Patrones GRASP; 3) Arquitectura de Software y estilos arquitectónicos; 4) Atributos de calidad (mantenibilidad, confiabilidad, etc.); 5) Niveles de prueba (unitaria, integración, validación, sistema). **Parte Práctica: Caso "Compañía Aérea — Pasajes y Embarque"** (Venta de boletos, check-in, despacho de equipaje con cobro por exceso de peso, embarque por puerta, cambio de estado del vuelo). Pide DC + DS + Diagrama de Estado.	- `Parciales/Parcial Diseño de Software 2023.pdf` - `Parciales/drive-download-20261111T183937Z-1-001/2023.pdf` *(duplicado)* - `Parciales/Parcial Unido.../5.[Parcial][2023][][][Resuelto][]/` (`Consigna.pdf`, `Posible resolución - Teoría.pdf`, `Posible resolución - Práctica.pdf`, `Diagrama de Clases.png`, `Diagrama de Secuencia.png`, `Diagrama de Estados.png`)	**¡SÍ! Totalmente resuelto** (Teoría y Práctica en PDF + diagramas PNG).
11	**Recuperatorio 2023**	**Cátedra Diseño**	**Parte Práctica: Caso "Torneo de Tenis Grand Slam"** (Partidos single masculino/femenino a 5 o 3 sets, tie-break, premios por ronda, estadísticas de aces/dobles faltas) + **Diagrama de Estado de Pedido de Productos**. **Parte Teórica:** Proceso de Diseño, Acoplamiento/Cohesión, Patrones Creacionales vs Estructurales, Vistas de Arquitectura, Pruebas de Integración.	- `Parciales/drive-download-20261111T183858Z-1-001/Consigna (parte 1).jpg`, `Consigna (parte 2).jpg`, `Resolución (hoja 1).jpg`, `Resolución (hoja 2).jpg` - `Parciales/Parcial Unido.../8.[Parcial][2023][][][Resuelto][Recuperatorio]/`	**Sí (Resolución manuscrita en 2 hojas JPG/PDF).**

#	Período y Tipo	Comisión / Docente	Caso Práctico / Tema Evaluado	Archivos Correspondientes en `Parciales/`	¿Tiene Resolución?
12	**Cuestionario General de Parciales (40 preguntas)**	**Cátedra Diseño**	Compilado de **40 preguntas teóricas clásicas de examen** respondidas en detalle: Arquitectura, Vistas 4+1, Cohesión (7 niveles) y Acoplamiento (6 niveles), GRASP, GoF, UML (Clases, Secuencia, Colaboración, Estado, Actividad, Casos de Uso, Componentes, Despliegue), Testing, RUP, Scrum, SOA y STR.	- `Parciales/Parcial Unido.../7. Posibles preguntas de parcial.pdf` (13 págs.)	**¡SÍ! Totalmente respondido** a lo largo de sus 13 páginas.

4. BIBLIOGRAFÍA Y APUNTES TEÓRICOS COMPLEMENTARIOS (`Teoria/`)

En el directorio `Teoria/` se encuentran **20 archivos PDF** divididos entre los libros de cabecera de la cátedra y apuntes específicos de UML y patrones:

4.1. Libros de Cabecera (`Teoria/Libros/`)

1. `UML y Patrones - Craig Larman.pdf` **(505 páginas — 1ª/2ª Edición en español):**
   - **Rol en la materia:** Es la **principal referencia bibliográfica de la Prof. Analía Marín** para **Patrones GRASP** (Capítulos 16 y 22: Experto, Creador, Bajo Acoplamiento, Alta Cohesión, Controlador, Polimorfismo, Fabricación Pura, Indirección, No Hablar con Extraños / Variaciones Protegidas), **Visibilidad entre objetos** (Capítulo 18: visibilidad de atributo, de parámetro, local y global), **Diagramas de Interacción** (Cap. 15) y **Diagramas de Clases de Diseño** (Cap. 19).
2. `Ingeniería de Software - Ian Sommerville.pdf` **(792 páginas — 9ª Edición en español):**
   - **Rol en la materia:** Referencia teórica para **Procesos de Software y RUP** (Cap. 2), **Desarrollo Ágil y Scrum** (Cap. 3), **Ingeniería de Requerimientos y Casos de Uso** (Cap. 4), **Modelado del Sistema con UML** (Cap. 5), **Diseño Arquitectónico y Vistas 4+1** (Cap. 6), **Diseño e Implementación Orientada a Objetos y Patrones GoF** (Cap. 7), **Pruebas de Software / Testing** (Cap. 8), **Arquitectura Orientada a Servicios / SOA** (Cap. 18) y **Sistemas de Software Empotrados / Tiempo Real (STR)** (Cap. 20).
3. *(Mencionado en diapositivas de Clase 01 y 02)* **Roger S. Pressman — *Ingeniería del Software: Un enfoque práctico***: Utilizado en las diapositivas `01-Diseño de Sistemas v2.pdf` y `02-Diseño Calidad` para las definiciones de Ingeniería de Software, Factores de Calidad de McCall, Dimensiones de Garvin y las 3 Reglas Doradas de Theo Mandel (UX).

4.2. Apuntes de Cátedra y Resúmenes en `Teoria/` y `Teoria/ Drive/`

Archivo	Páginas	Contenido y Utilidad para el Parcial
`Teoria/Patrones de Diseño (GOF y GRASP).pdf`	15 págs.	Resumen muy claro y directo de todos los patrones **GRASP** y los **23 patrones GoF** con su intención, aplicabilidad y estructura.
`Teoria/Diagramas UML.pdf`	29 págs.	Guía sintética de notación UML para Clases, Secuencia, Colaboración, Estados, Actividad, Casos de Uso, Componentes y Despliegue.
`Teoria/Resumen Diseno de Sistemas.pdf`	30 págs.	Resumen teórico completo de todas las unidades de la materia (Ideal para repaso rápido de definiciones teóricas).
`Teoria/Repaso Teoría.pdf`	14 págs.	Preguntas y respuestas de repaso teórico.
`Teoria/Posibles preguntas de parcial.pdf`	13 págs.	Idéntico a `Parciales/.../7. Posibles preguntas de parcial.pdf` (40 preguntas de examen respondidas).
`Teoria/Casos de Uso.pdf`	12 págs.	Guía detallada de modelado de Casos de Uso (`<<include>>` , `<<extend>>` , generalización y especificación).
`Teoria/2020_modelado_de_clases.pdf`	36 págs.	Apunte universitario sobre relaciones en Diagrama de Clases, multiplicidades, agregación, composición y herencia.
`Teoria/Relaciones_entre_Clases.pdf`	3 págs.	Ficha rápida de diferencias entre Asociación, Agregación, Composición, Dependencia y Herencia.
`Teoria/diagrama de secuencia.pdf`	9 págs.	Notación de mensajes sincrónicos, asincrónicos, retorno, creación/destrucción de objetos y fragmentos combinados (`alt` , `opt` , `loop`).
`Teoria/Teoria de Diagrama de Estados y Componentes.pdf`	10 págs.	Explicación de eventos, guardas, acciones y conectores de componentes.
`Teoria/328430431-Patrones-GRASP-y-GOF.pdf`	8 págs.	Cuadro comparativo rápido de GRASP y GoF.
`Teoria/Patrones GoF - Clasificación.png`	1 img	Mapa visual de clasificación de los 23 patrones GoF (Creacionales, Estructurales, Comportamiento).
`Teoria/ Drive/Apunte - Patrones de Diseño (GRASP).pdf`	19 págs.	Apunte detallado de los 9 patrones GRASP con ejemplos de código y diagramas.
`Teoria/ Drive/Apunte - Patrones de Diseño (GoF - Creacionales).pdf`	19 págs.	Detalle profundo de Abstract Factory, Builder, Factory Method, Prototype y Singleton.
`Teoria/ Drive/Apunte - Patrones de Diseño (GoF - Estructurales).pdf`	26 págs.	Detalle profundo de Adapter, Bridge, Composite, Decorator, Facade, Flyweight y Proxy.
`Teoria/ Drive/Apunte - Patrones de Diseño (GoF - Comportamiento).pdf`	37 págs.	Detalle profundo de los 11 patrones de comportamiento con diagramas UML y ejemplos.
`Teoria/ Drive/Apunte - Principios de Diseño.pdf`	25 págs.	Principios SOLID, Alta Cohesión, Bajo Acoplamiento, Encapsulamiento e Inyección de Dependencias.
`Teoria/ Drive/Apunte - Diagramas (UML).pdf`	20 págs.	Repaso general de diagramas estructurales y de comportamiento en UML 2.x.

5. REPORTE DE ARCHIVOS DUPLICADOS Y MATERIAL INCOMPLETO / BRECHAS SUBSANADAS

5.1. Archivos Duplicados Exactos (No perder tiempo abriéndolos dos veces)

- **Video duplicado:** `Clases-videos/Clase-20261008_082230-Grabación de la reunión Copy.mp4` es una copia byte por byte (`919,665,788` bytes) de `Clase-20261008_082230-Grabación de la reunión.mp4` .
- **Presentación GoF duplicada:** `Clases/Clases/07/GOF-2Q-2026.pdf` y `Clases/Clases/08/000 Diseño de patrones GOF-2026-10-08 v4.pdf` tienen exactamente las mismas 71 diapositivas.

- **Modelo Laboratorio duplicado 4 veces:** `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (33 págs.) contiene íntegramente a `Parciales/Parcial1C25-Resuelto.pdf` (26 págs.), `Parciales/drive-download-20261111T183937Z-1-001/Modelo de parcial.pdf` (26 págs.) y `Parciales/Parcial Unido.../6. [Parcial][1C2025][][][Resuelto][].pdf` (26 págs.), agregando además 7 páginas finales nuevas (págs. 27–33) con respuestas de la profesora sobre el Caso Gimnasio, diferencia entre Controller y Gestor, y Visibilidad. **Usar siempre** `EJEMPLO-Practica-Teoria-version-10.pdf`**.**
- **Cuestionario de 40 preguntas duplicado:** `Teoria/Posibles preguntas de parcial.pdf` es idéntico a `Parciales/Parcial Unido.../7. Posibles preguntas de parcial.pdf`.
- **Imágenes y PDFs de Parciales duplicados:** Todos los archivos `.png` sueltos en la raíz de `Parciales/` (`0c34a650...`, `3b188c5d...`, etc.) son copias idénticas de `Parciales/drive-download-20261111T183937Z-1-001/1C2025 (1..6).png`, los cuales a su vez están compilados en `Parciales/Parcial Unido.../1.pdf`, `2.pdf` y `3.pdf`. Del mismo modo, las fotos de WhatsApp de `Parciales/` ya están pasadas a limpio en los PDFs numerados `1. [Parcial 1][1C2026][4300]...`, `2. [Parcial 1][1C2026][1900]...` y `3. [Parcial 1][2C2026][1900]...`.

5.2. Brechas del Material Original que hemos cubierto en esta Guía Integral

1. **Clase 04 Presencial (10/09/2026) sin video:** Al haberse dictado presencialmente en el Aula 136, no tuvo grabación de Teams. Sin embargo, contamos con el PDF completo de 77 diapositivas de la profesora (`Clases/Clases/04-Presencial/UNLaM GRASP-original-marin.pdf`) y las explicaciones que la profesora retoma en las Clases 05, 06, 07 y 09. Todo ese contenido está integrado en `02_GUIA_COMPLETA_UML.md` y `03_PATRONES_GRASP_Y_GOF_APLICADOS.md`.
2. **Los 3 Parciales 2026 (`PetCare`, `Refugio de Mascotas`, `Agencia de Viajes`) no tenían un PDF de resolución diagramada en la carpeta:** Si bien `PetCare` fue discutido extensamente en la Clase 09, no había un documento con los diagramas pasados en limpio de los tres exámenes 2026. En `04_METODOLOGIA_Y_PARCIALES_RESUELTOS_PASO_A_PASO.md` hemos desarrollado la **resolución completa paso a paso** de los parciales 2026 respetando estrictamente todas las reglas de la Ing. Analía Marín.

---

6. ESTRUCTURA DE ESTA GUÍA INTEGRAL DE ESTUDIO

Para que tu preparación durante esta semana sea máxima, ordenada y directamente aplicable al examen, hemos estructurado el estudio en **5 documentos maestros interconectados**:

1. `00_INDICE_GENERAL_MATERIA.md` *(Este documento)*: Auditoría completa del repositorio, mapa cronológico de clases y transcripciones, catálogo depurado de parciales y alerta sobre resúmenes sintéticos.
2. `01_CRITERIOS_Y_CONSEJOS_DE_LA_PROFESORA.md` : El documento más importante para aprobar con Analía Marín. Extrae **todas sus reglas de corrección**, qué considera un error eliminatorio, cómo exige el Diagrama de Clases, el Diagrama de Secuencia, los Controllers, los Multiobjetos/Nóminas, la Visibilidad y los patrones GRASP y GoF, con citas exactas a sus clases y archivos.
3. `02_GUIA_COMPLETA_UML.md` : Manual completo y progresivo de todos los diagramas UML de la materia (**Diagrama de Clases**, **Casos de Uso y Especificación**, **Actividad**, **Secuencia**, **Comunicación/Colaboración**, **Estados**, **Componentes** y **Despliegue**) y las **4+1 Vistas de Kruchten**, explicando la diferencia entre lo que dicen Larman/Sommerville y las convenciones específicas que exige la cátedra.
4. `03_PATRONES_GRASP_Y_GOF_APLICADOS.md` : Guía definitiva de **Patrones GRASP (los 9 patrones)** y **Patrones GoF (los 23 patrones)** orientada 100% a resolver los Puntos 1 y 3 del parcial, con plantillas de justificación, combinaciones ganadoras de GoF que no usan Singleton ni Facade, y cómo redibujar las clases para inyectar el patrón en el examen.
5. `04_METODOLOGIA_Y_PARCIALES_RESUELTOS_PASO_A_PASO.md` : Estrategia cronometrada para rendir el parcial + **Resolución completa desde cero de los Parciales 2026 de Analía Marín** (*Veterinaria PetCare*, *Agencia de Viajes*, *Refugio de Mascotas*) + Resumen de Unidades Teóricas Finales (Calidad, UX, SOA/Microservicios, Testing y STR) y Banco de Preguntas Teóricas de Examen.

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 1: CRITERIOS, CONVENCIONES Y CONSEJOS DE LA PROFESORA (ING. ANALÍA MARÍN)

📑 **ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO**

1. FILOSOFÍA GENERAL DE CORRECCIÓN: "¿QUÉ BUSCA ANALÍA MARÍN EN EL PARCIAL?" Pág. 11
1.1. Nivel de Baja Abstracción: "Estamos en Diseño, NO en Análisis" Pág. 11
1.2. Prohibición Absoluta de "ABMs / CRUDs Triviales" y Getters/Setters Masivos Pág. 11
1.3. Reglas Formales de Entrega en Hoja (Formato de Examen Presencial) Pág. 11
1.4. Técnica de Análisis de Enunciado con Código de Colores de la Profesora Pág. 12
2. EL ERROR ELIMINATORIO N° 1: TRAZABILIDAD E INTEGRACIÓN ENTRE DIAGRAMAS Pág. 12
2.1. Regla de Oro de Correspondencia de Mensajes y Métodos Pág. 12
3. CRITERIOS ESPECÍFICOS PARA EL DIAGRAMA DE CLASES Pág. 12
3.1. Organización Visual del Diagrama de Clases ("Arriba hacia Abajo, Izquierda a Derecha") Pág. 12
3.2. Cuántos Controladores (Controller GRASP) poner y cómo nombrarlos Pág. 12
3.3. Diferencia entre Controller y Gestor (¡Cuidado con el "Gestor Pasapelotas"!) Pág. 13
3.4. Diferencia Estricta entre "Actor del Sistema" y "Clase del Dominio" (El Error del Recepcionista / Veterinario) Pág. 13
3.5. El Problema de la "Ficha Única" vs. "El Fichero (Colección / Multiobjeto)" y Relación de la Nomina Pág. 13
3.6. Visibilidad (Larman Cap. 18) y la Tabla Oficial de la Profesora sobre qué se dibuja en el Diagrama de Clases Pág. 13
3.7. Pasaje de Parámetros desde la UI: "IDs Primitivos afuera, Referencias a Objetos adentro" Pág. 14
3.8. Uso Correcto de Agregación (Rombo Blanco) vs. Composición (Rombo Negro) vs. Asociación Simple Pág. 14
3.9. Clases Abstractas (<<abstract>>) vs. Interfaces (<<interface>>) Pág. 14
4. CRITERIOS ESPECÍFICOS PARA EL DIAGRAMA DE SECUENCIA (PUNTO 2 DEL PARCIAL) Pág. 14
4.1. Regla N° 1: Escribir Arriba el "Escenario Concreto" con Datos Puntuales Pág. 14
4.2. Regla N° 2: ¡NO Dibujar al Actor Humano en el Diagrama de Secuencia de Diseño! Pág. 15
4.3. Regla N° 3: Creación de Objetos en Tiempo de Ejecución (create()) Pág. 15
4.4. Regla N° 4: Respetar el Alcance Exacto Pedido en el Punto de Secuencia Pág. 15
5. CRITERIOS PARA LOS DEMÁS DIAGRAMAS UML (CU, ACTIVIDAD, COLABORACIÓN, ESTADO, COMPONENTES, DESPLIEGUE) Pág. 15
5.1. Casos de Uso y su Especificación (EJEMPLO-Practica-Teoria-version-10.pdf Págs. 4–7 y Clase 03) Pág. 15
5.2. Diagrama de Actividad (EJEMPLO-Practica-Teoria-version-10.pdf Pág. 7 y Clase 03/06) Pág. 15
5.3. Diagrama de Colaboración / Comunicación (EJEMPLO-Practica-Teoria-version-10.pdf Pág. 12 y Clase 04) Pág. 15
5.4. Diagrama de Estados (EJEMPLO-Practica-Teoria-version-10.pdf Pág. 13, Parciales 1 y 3 de 2026, Finales Dic 2025 y Jul 2026) Pág. 16
5.5. Diagrama de Componentes (EJEMPLO-Practica-Teoria-version-10.pdf Pág. 14, Banco-diagComponente.png, ComponenteVersion2.png y Parciales 1, 2 y 3 de 2026) Pág. 16
5.6. Diagrama de Despliegue (EJEMPLO-Practica-Teoria-version-10.pdf Págs. 15–16, Banco-diagDespliegue v2.png y Despliegue2.png) Pág. 16
6. CÓMO RESPONDER LAS JUSTIFICACIONES DE PATRONES EN EL PARCIAL Pág. 16
6.1. Cómo Justificar los Patrones GRASP (Punto 1 del Parcial) Pág. 16
6.2. Cómo Elegir, Dibujar y Justificar los 2 Patrones GoF Pág. 16
7. CHECKLIST RÁPIDO DE CONTROL ANTES DE ENTREGAR EL PARCIAL (LOS 12 MANDAMIENTOS DE ANALÍA) Pág. 17

1. FILOSOFÍA GENERAL DE CORRECCIÓN: "¿QUÉ BUSCA ANALÍA MARÍN EN EL PARCIAL?"

1.1. Nivel de Baja Abstracción: "Estamos en Diseño, NO en Análisis" `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Criterio de la Profesora:** En Análisis de Sistemas alcanzaba con dibujar cajas con sustantivos (Modelo Conceptual del Dominio). En **Diseño de Sistemas**, el modelo debe tener **baja abstracción** y estar **cerca del código fuente**. Esto exige:
  - Clases del dominio con **atributos concretos** y **métodos de negocio reales**.
  - **Visibilidad** explícita (`-` privado, `+` público, `#` protegido).
  - **Clases de Diseño (Fabricación Pura / Soporte):** Controladores (`C_...`), Nóminas/Catálogos/Padrón (si hace falta manejar colecciones globales), Interfaces (`<<interface>>`), Clases Abstractas (`<<abstract>>`), Clases pasarela hacia sistemas externos (ej. `sisExtOS`) y las clases que implementan **Patrones GoF**.
  - **Parámetros en los métodos clave:** Especialmente en el Diagrama de Secuencia y en los métodos que reciben referencias a objetos o IDs desde la interfaz.
- **Evidencia:**
  - `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 10 y Pág. 20): "*El diagrama de clases muestra las clases del sistema, sus atributos, métodos y las relaciones entre ellas. Es la base del diseño orientado a objetos y sirve de puente entre los requerimientos y el código*".
  - `Transcripcion/transcripciones/09_20261015_082342.txt` (`[00:13:02]` y `[01:15:36]`): "*Acá estamos en diseño, nos tenemos que acercar al código... No nos podemos quedar en un modelo conceptual*".

1.2. Prohibición Absoluta de "ABMs / CRUDs Triviales" y Getters/Setters Masivos `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Criterio de la Profesora:** Está terminantemente prohibido llenar las clases de métodos `alta()` , `baja()` , `modificacion()` , `getNombre()` , `setNombre()` , o poner un `Controller` por cada clase para hacer un ABM. **En el parcial se modela el CORE (corazón) del negocio.**
- **Frases textuales de clase y guías:**
  - `Clases/Clases/04/Gym/GUIA-EjercicioGYM-Estudiantes-2026- V01.docx` (Pág. 2): "*Evitar pensar que cada clase tiene un controller para hacer un ABM (alta-baja-modificación). Eso en los parciales NO VA. Siempre vamos al CORE del negocio.*"
  - `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 27): "*Si cada clase tiene un controlador para el alta, baja o modificación del objeto, ¿para qué están las relaciones de las clases?... Evitar los get y set, eso ya es por defecto por el paradigma.*"
- **Qué hacer en su lugar:** Si la clase necesita inicializarse, usar su constructor (`crear(...)` / `new` / `create()`) y métodos con **nombre de regla de negocio** (ej.: `validarVigencia()` , `registrarVacuna(v)` , `calcularMontoTotal()` , `verificarCupo()` , `confirmarReserva()`). Solo incluir un `get` puntual si en el Diagrama de Secuencia necesitás pedir un dato específico a otro objeto para un cálculo (Experto en Información).

1.3. Reglas Formales de Entrega en Hoja (Formato de Examen Presencial) `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Criterio obligatorio en todos los parciales 2026:**
  - **"ENTREGAR UN DIAGRAMA POR HOJA, ENUMERAR LAS HOJAS A ENTREGAR."** (Aparece en mayúsculas y subrayado al final de `1. [Parcial 1][1C2026][4300][Analia][][].pdf` , `2. -Parcial 1--1C2026--1900--Valeria y Analia-----.pdf` y `3. [Parcial 1][2C2026][1900][][][].pdf`).
  - **Hoja 1:** Diagrama de Clases completo (con las clases del dominio, los Controllers, la integración del Diagrama de Estados si el examen lo provee y los 2 patrones GoF integrados — marcando claramente qué clases pertenecen a cada patrón GoF) + Justificación de GRASP.
  - **Hoja 2:** Escenario redactado arriba + Diagrama de Secuencia completo (y en una hoja separada el Diagrama de Componentes modificado/mejorado si el parcial lo solicita, como en los Parciales 2 y 3 de 2026).
  - **Hoja de Patrones GoF:** Explicación y justificación detallada de los 2 Patrones GoF (Categoría, Nombre, Problema del caso, Por qué es adecuado, Clases reutilizadas y Clases/Interfaces nuevas incorporadas).

1.4. Técnica de Análisis de Enunciado con Código de Colores de la Profesora `[RECOMENDACIÓN PRÁCTICA DE LA PROFESORA]`

- En `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Págs. 5–6 y Pág. 8) y en la Clase 03 (`03_20260903_101348.txt`), la profesora enseña a subrayar el texto del enunciado y la especificación de Casos de Uso con **tres colores**:
  - **Amarillo:** Sustantivos principales que representan **Clases candidatas del dominio** (ej.: `Paciente`, `Turno`, `OrdenMedica`, `Estudio`, `Muestra`, `Factura`).
  - **Verde:** Verbos y frases verbales que representan **Funcionalidades / Comportamiento / Métodos** (ej.: `solicitar turno`, `registrar orden`, `validar cobertura`, `calcular importe`).
  - **Celeste (Azul claro):** **Atributos** de las clases o **clases secundarias/de soporte** (ej.: `DNI`, `fecha`, `hora`, `obra social`, `código de barra`).
- Además, en el **Diagrama de Clases** (`EJEMPLO-Practica-Teoria-version-10.pdf`, Pág. 10), la profesora pinta de **color verde** la clase pasarela hacia sistemas externos (`sisExtOS`) para distinguirla visualmente de las clases propias del dominio (amarillas).

---

2. EL ERROR ELIMINATORIO N° 1: TRAZABILIDAD E INTEGRACIÓN ENTRE DIAGRAMAS `[REGLA EXPLÍCITA DE CÁTEDRA]`

La profesora repite en **todas** las clases prácticas que los diagramas de UML no son dibujos sueltos, sino **distintas vistas del MISMO sistema** (Modelo 4+1 de Kruchten). Por lo tanto, tienen que ser **100% consistentes entre sí**.

2.1. Regla de Oro de Correspondencia de Mensajes y Métodos

> ⚠️ **ALERTA / REGLA CRÍTICA DE CORRECCIÓN (ING. ANALÍA MARÍN)**
>
> **REGLA ELIMINATORIA:** Si en el **Diagrama de Secuencia** (o de Colaboración) el objeto `A` le envía el mensaje `m()` al objeto `B` (`A ──── m() ──►  :B`), entonces:
>
> 1. **El método `+m()` DEBE estar escrito en el compartimento de métodos de la clase `B` en el Diagrama de Clases.** (¡No en `A`! El método pertenece a la clase que **recibe** el mensaje y ejecuta la acción).
> 2. **La clase `A` DEBE tener visibilidad hacia la clase `B` en el Diagrama de Clases** (una línea de asociación si es visibilidad permanente de atributo, o relación/dependencia si entra por parámetro; ver Regla 3.6 para el caso de la visibilidad local).
> 3. Si el parcial provee o pide un **Diagrama de Estados** para un objeto `B` (como `Consulta Veterinaria` en el Parcial 1C 2026 Com. 4300 o `Reserva` en el Parcial 2C 2026 Com. 1900), **todas las transiciones/eventos de ese Diagrama de Estados (`registrarConsulta()`, `registrarDiagnostico()`, `cerrarConsulta()`, `cancelar()`, etc.) DEBEN figurar como métodos de la clase `B` en el Diagrama de Clases**, y la clase `B` debe tener su atributo `-estado` (o el patrón GoF `State` asociado).

- **Evidencia documental directa:**
  - `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 16 y Pág. 20): "*NOTA: Todos los mensajes que están presentes en el diagrama de secuencia, colaboración y estado deben estar en el diagrama de clases. Las vistas deben ser consistentes.*"
  - `Clases/Clases/04/Gym/GUIA-EjercicioGYM-Estudiantes-2026- V01.docx` (Pág. 3): "*Cada mensaje que llega a un objeto es un método en esa clase. Verificar que en el diag de clases se encuentren esos métodos. Si hay mensajes en el diag. de interacción que no están en el diag de CLASES, el ejercicio está MAL.*"
  - `Transcripcion/transcripciones/09_20261015_082342.txt` (`[01:55:12]`): "*Chicos, cuando corregimos ponemos el Diagrama de Secuencia al lado del Diagrama de Clases. Si en secuencia llaman a `validarVacuna()` en `HistoriaClinica`, vamos a buscar `validarVacuna()` en la clase `HistoriaClinica`. Si no está, está mal.*"

---

3. CRITERIOS ESPECÍFICOS PARA EL DIAGRAMA DE CLASES

3.1. Organización Visual del Diagrama de Clases ("Arriba hacia Abajo, Izquierda a Derecha") `[RECOMENDACIÓN PRÁCTICA DE LA PROFESORA]`

La profesora pide que el Diagrama de Clases esté ordenado visualmente por capas para que sea legible al corregir:

1. **Parte Superior (Arriba de todo):** Los **Controladores (`C_...`)**.
2. **Parte Media-Alta (Debajo de los Controllers):** Los **Gestores / Nóminas / Padrón** (únicamente si son necesarios; ver regla 3.3 y 3.5).
3. **Parte Central e Inferior:** Las **Clases del Modelo de Dominio** y las **jerarquías de herencia / patrones GoF**, agrupadas verticalmente en columnas según el módulo o componente al que pertenecen.
4. **¿Qué NO va en el Diagrama de Clases?** No va la capa de Base de Datos (tablas SQL, `DAO`, `Repository` genéricos) ni las pantallas HTML/Vistas, salvo que se aplique un patrón específico que requiera una interfaz de vista.
5. **Evidencia:**
   - `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 10 y Pág. 28): "*Leer el diagrama de arriba hacia abajo, de izquierda a derecha... Arriba van los controladores, debajo las clases del dominio agrupadas por cercanía funcional.*"
   - `Clases/Clases/04/Gym/GUIA-EjercicioGYM-Estudiantes-2026- V01.docx` (Pág. 1): "*Organizar visualmente el diagrama: arriba los controllers, luego las clases principales, y hacia abajo las clases de detalle o soporte.*"

3.2. Cuántos Controladores (`Controller` GRASP) poner y cómo nombrarlos `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Nomenclatura:** La profesora utiliza el prefijo `C_` o `c_` seguido del nombre del módulo/componente o caso de uso agrupado:
  - Ejemplo Laboratorio (`EJEMPLO-Practica-Teoria-version-10.pdf`, Pág. 10): `C_turno`, `C_Recepcion`, `C_Analitica`, `C_Pago`, `C_Validacion`.
  - Ejemplo Banco (`Clases/05/Banco/Banco-diagClase.png`): `C_EvaluacionP`, `C_prestamo`, `C_Cliente`.
  - Ejemplo Estadios (`Clases/06/Estadio/Clase2.png`): `c_empadronar`, `c_ingreso`.
  - Ejemplo Gimnasio (`EJEMPLO-Practica-Teoria-version-10.pdf`, Pág. 28): `C_Inscripcion`.
- **Cantidad exacta en el Parcial:**
  - En la práctica real no se pone 1 Controller para todo el sistema (porque queda **saturado** y viola la Alta Cohesión) ni tampoco 1 Controller por cada Caso de Uso individual si hay 15 casos de uso (por falta de tiempo en el examen).
  - **Criterio de cátedra:** Definir **1 Controller por cada Módulo / Componente funcional principal** del sistema.
  - **Límite explícito en los parciales 2026 (Máximo 4 Controladores):**
  - En `1. [Parcial 1][1C2026][4300][Analia][][].pdf` (Veterinaria PetCare): figura en nota manuscrita superior `max: 4 controladores` (y en la discusión oral de Clase 09 `[02:01:15]` se trabajaron 3 o 4 controladores según cómo se agrupen `C_UsuarioMascota`, `C_Turno`, `C_AtencionVeterinaria` / `C_HistoriaClinica`).
  - En `2. -Parcial 1--1C2026--1900--Valeria y Analia-----.pdf` (Reencuentro/Adopción y Animales Exóticos) y `3. [Parcial 1][2C2026][1900][][][].pdf` (Agencia de Viajes): la consigna impresa exige textualmente **"usando solo 4 controladores"**.
- **Qué tiene adentro un Controller (y qué atributos SÍ puede tener):**
  - **NO tiene atributos de datos propios del dominio** (no guarda el `nombreCliente` ni el `saldo`).
  - **SÍ puede tener atributos de referencia de coordinación y estado temporal de sesión o del caso de uso en curso:** Tal como explica explícitamente la profesora en `EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 31), un Controller puede mantener referencias a servicios/nóminas que coordina y referencias al estado temporal de la sesión activa (por

ejemplo: `-pacienteActual: Paciente`, `-prestamoActual: Prestamo`, `-estadoValidacion: Boolean`), aunque en los diagramas de clases de pizarrón muchas veces se deje el compartimento de atributos vacío para simplificar.
  - **Sus métodos son los manejadores de los eventos de entrada del sistema** (ej. `+guardarConsulta(...)`, `+registrarDenunciaExotico(...)`, `+realizarPago(...)`).
  - **Su responsabilidad es coordinar ("director de orquesta"):** Recibe datos primitivos/IDs de la UI, busca las referencias a los objetos correspondientes y **delega** el trabajo real a las clases del dominio (Experto).
- **Evidencia:**
  - `Clases/Clases/04-Presencial/UNLaM GRASP-original-marin.pdf` (Pág. 62): "*Aclaración: En el examen, debido a limitaciones de tiempo, no vamos a poder implementar un controlador por cada Caso de Uso (CU). En su lugar, el diseño debe contemplar un controlador por módulo o subsistema.*"
  - `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 10, Pág. 28 y Pág. 31): "*El controlador captura el evento y delega a quien corresponda. Tiene lógica mínima... Puede tener referencias a las clases con las que coordina y estado temporal de la sesión.*"

3.3. Diferencia entre `Controller` y `Gestor` (¡Cuidado con el "Gestor Pasapelotas"!) `[REGLA EXPLÍCITA DE CÁTEDRA]`

En la Clase 07 y en las páginas 29–31 de `EJEMPLO-Practica-Teoria-version-10.pdf`, un alumno preguntó cuándo usar un `Gestor` debajo del `Controller`. La respuesta de la profesora es taxativa:

- **El Error del "Pasapelotas":** Si ponés un `C_Inscripcion` con el método `inscribirSocio()` y abajo ponés un `GestorInscripcion` con el mismo método `inscribirSocio()` que hace exactamente lo mismo, **está MAL** (es redundante y agrega acoplamiento inútil).
- **Cuándo SÍ tiene sentido un Gestor o Nómina/Padrón:**
  - Cuando el `Controller` se vuelve **saturado** porque además de recibir eventos de UI hay que orquestar reglas de negocio complejas entre múltiples subsistemas.
  - Cuando necesitás una **clase contenedora de colección global (Fabricación Pura / Multiobjeto)** como `Padron` (en Caso Estadios) o `NominaPacientes` / `NominaMascotas` para buscar objetos en memoria (`buscarAficionado(dni)`, `buscarMascota(id)`) sin ensuciar la clase individual `Aficionado` o `Mascota`.
- **Evidencia:** `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Págs. 29–31) y `Transcripcion/transcripciones/07_20261001_082247.txt` (`[00:18:40]`).

3.4. Diferencia Estricta entre "Actor del Sistema" y "Clase del Dominio" (El Error del `Recepcionista` / `Veterinario`) `[REGLA EXPLÍCITA DE CÁTEDRA]`

Este es uno de los errores más corregidos en la Clase 09 (`09_20261015_082342.txt`):

- **El Error Frecuente:** El enunciado dice "*El veterinario registra los datos de la mascota, aplica vacunas y emite la receta*" o "*La recepcionista da de alta el turno*". Muchos alumnos crean la clase `Veterinario` o `Recepcionista` y le ponen los métodos `+registrarMascota()`, `+aplicarVacuna()`, `+emitirReceta()`, o hacen que el `Controller` llame a `veterinario.atender()`.
- **Por qué está MAL según la Profesora:**
  - La persona humana que tipea en la pantalla es el **Actor** que dispara el evento hacia el `Controller`. **No es un objeto de software ejecutando lógica de negocio.**
  - Si le ponés todos los métodos del sistema a la clase `Veterinario`, convertís a `Veterinario` en una **clase Dios / Gestor encubierto** (violando **Experto en Información** y **Alta Cohesión**). Quien conoce las consultas y vacunas de una mascota es la `HistoriaClinica` (o `Mascota` / `Consulta`), ¡no el `Veterinario`!
  - **¿Cuándo SÍ va la clase `Profesional` / `Veterinario` / `Medico` en el Diagrama de Clases?** Únicamente como **entidad de datos del dominio** cuando el negocio requiere guardar **quién** atendió la consulta o turno (ej. en el Parcial 1C 2026 *PetCare*, `Profesional` guarda sus datos y su horario en la `Veterinaria`, y se asocia al `Turno` y a la `Consulta`), o cuando hay que consultar su agenda.
  - **¿Y si el enunciado habla de permisos de usuarios (Recepcionista, Admin, Bioquímico)?** No se crean subclases de usuario con métodos de negocio, sino el modelo extensible `Usuario` → `Perfil` (o `Rol`) → `Permiso` (como muestra la profesora en `EJEMPLO-Practica-Teoria-version-10.pdf`, Pág. 24).
- **Evidencia:**
  - `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 20 y Pág. 24): "*Recepcionista y Bioquímico son actores, son roles que operan el sistema, NO son clases que lanzan métodos. Si se requiere modelar quién hizo qué o sus permisos, se modela Usuario con Perfil y Permiso para favorecer la Extensibilidad.*"
  - `Transcripcion/transcripciones/09_20261015_082342.txt` (`[01:28:15]` y `[01:42:50]`): "*El veterinario es el actor que está sentado frente a la pantalla cargando la consulta del gato Homero. La clase Profesional/Veterinario solo guarda su matrícula y nombre asociada a la Consulta o Receta, ¡el Veterinario no es el que guarda la vacuna en la base ni valida la fecha, eso lo hace Consulta o HistoriaClinica!*"

3.5. El Problema de la "Ficha Única" vs. "El Fichero (Colección / Multiobjeto)" y Relación de la `Nomina` `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Criterio de la Profesora:** Una clase como `Socio`, `Aficionado`, `Mascota` o `Consulta` representa **UN único individuo o una única ficha atómica** (una instancia).
  - **Pregunta de control de la profesora:** Si tenés la clase `Consulta` (que representa *una* visita médica del día de hoy), ¿puede esa instancia `Consulta` tener el método `buscarConsultasAnteriores()`? **¡NO! Porque una instancia de `Consulta` solo se conoce a sí misma.**
  - **Distinción clave entre Colección Propia de un Objeto vs. Búsqueda Global (`Nomina` / `Multiobjeto`):**
  - **Colección propia de una entidad del dominio:** Todas las consultas de una mascota pertenecen a su `HistoriaClinica` (1 a 1 con `Mascota`), y todos los ítems de una venta pertenecen a `Venta`. Por lo tanto, para agregar o consultar el historial de *esa* mascota **NO hace falta crear una clase** `NominaConsultas`, sino que la propia clase contenedora del dominio (`HistoriaClinica`) maneja su colección `consultas: List<Consulta>` con métodos como `+agregarConsulta(c)` o `+obtenerHistorial()`.
  - **Búsqueda global entre todas las instancias del sistema (`Nomina` / `Padron` / Multiobjeto):** Cuando el `Controller` recibe un `dni` o `id` desde la interfaz y necesita encontrar a qué objeto `Aficionado`, `Usuario` o `Paciente` corresponde entre todos los registrados en el sistema, una instancia individual no puede buscar a las demás. Allí sí se utiliza una clase contenedora global (`Padron`, `NominaUsuarios`, `NominaMascotas` o un **Multiobjeto** `:Mascota` en Secuencia) con el método `+buscarPorId(id)`.
  - **¿Con qué relación se une la `Nomina` a la clase individual en el Diagrama de Clases? (`[02:38:03]` de Clase 09):**
  - Se une con **Asociación simple (`——`, de `1` a `0..*`), NO con Composición fuerte (`◆`)**, porque la `Nomina` es solo un índice/colección en memoria (Fabricación Pura) y si se destruye el listado o filtro no deben destruirse las entidades del negocio.
- **Evidencia:**
  - `Transcripcion/transcripciones/06_20260924_114136.txt` (`[00:05:10]` a `[00:16:30]`).
  - `Transcripcion/transcripciones/09_20261015_082342.txt` (`[00:35:20]`, `[01:36:40]` y `[02:38:03]`): "*Una clase representa una ficha. Para buscar en el fichero necesitás a quien contiene la colección: la Historia Clínica para las consultas de esa mascota, o el Padrón/Nómina para buscar en general... Y entre Nómina y la clase va asociación normal, no composición.*"

3.6. Visibilidad (Larman Cap. 18) y la Tabla Oficial de la Profesora sobre qué se dibuja en el Diagrama de Clases `[REGLA EXPLÍCITA DE CÁTEDRA]`

En la Clase 07 (`07_20261001_082247.txt`) y en las páginas 19, 32 y 33 de `EJEMPLO-Practica-Teoria-version-10.pdf`, la profesora incluye un cuadro comparativo explícito de los **4 tipos de visibilidad de Craig Larman** y define cuáles se grafican en el Diagrama de Clases:

Tipo de Visibilidad (Larman)	¿Cómo se logra en código?	¿Duración de la relación?	¿Aparece graficada en el Diagrama de Clases según Analía Marín (`EJEMPLO-Practica...` Pág. 19)?
1. Visibilidad de Atributo	`B` es un atributo declarado dentro de la clase `A` (`private B miB;`).	**Permanente** (perdura mientras exista `A`).	**SÍ, SIEMPRE.** Se dibuja como una **línea sólida de Asociación** (o Agregación/Composición) entre `A` y `B` con su multiplicidad.

Tipo de Visibilidad (Larman)	¿Cómo se logra en código?	¿Duración de la relación?	¿Aparece graficada en el Diagrama de Clases según Analía Marín (`EJEMPLO-Practica...` Pág. 19)?
2. Visibilidad de Parámetro	`B` se pasa como parámetro en un método de `A` (`metodo(B unB)`).	**Temporal** (dura lo que dura la ejecución del método).	**SÍ (si se detallan firmas o dependencias).** Se ve en la firma del método `+metodo(b: B)` y puede representarse con una **flecha punteada de Dependencia** (`- - - >`). Se transforma en Visibilidad de Atributo si `A` guarda ese parámetro en un atributo propio (`this.b = unB`).
3. Visibilidad Local	`A` declara una variable local de tipo `B` adentro de un método (`B temp = ...`).	**Temporal** (muere al terminar el método).	**NO APARECE en el Diagrama de Clases** (*Cuadro oficial Pág. 19: "¿Aparece en el Diagrama de Clases?: No"*). Solo se observa dinámicamente en el **Diagrama de Secuencia** cuando el método crea o pide localmente esa instancia.
4. Visibilidad Global	`B` es accesible globalmente por cualquier objeto (ej. variable global o `Singleton.getInstance()`).	Permanente/Global.	Solo si se modela el patrón `Singleton` (que además suele estar restringido en los parciales 2026).

- **Advertencia clave de la Profesora sobre las líneas del `Controller` (`EJEMPLO-Practica...` Págs. 32–33):** "*En el diagrama de clases se grafican las relaciones permanentes (visibilidad de atributo) y las más significativas. NO tiren líneas de asociación sólidas desde el Controller hacia las 20 clases del diagrama porque queda una telaraña ilegible y conceptualmente incorrecta (el Controller no es dueño permanente de todo el dominio)*". El Controller solo se relaciona con las 2 o 3 clases raíz/nóminas a las que delega la coordinación.
- **Evidencia:** `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 19 y Págs. 32–33).

3.7. Pasaje de Parámetros desde la UI: "IDs Primitivos afuera, Referencias a Objetos adentro" `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Criterio de la Profesora:** Cuando el evento entra desde la pantalla HTML al `Controller`, la pantalla envía **datos primitivos o IDs** (ej.: `idMascota`, `dniCliente`, `codigoVacuna`, `peso`, `observaciones`).
  - El `Controller` recibe esos datos primitivos: `+guardarConsulta(idMascota, peso, diag, receta, vacuna, recordatorio)`.
  - El primer paso dentro del sistema es **obtener la referencia al objeto** buscando por ese ID (ej.: pidiéndole a la nómina/contenedor `mascota = buscarMascota(idMascota)`).
  - Una vez obtenida la referencia `mascota`, entre los objetos del dominio se pasan **referencias a objetos** (ej.: `historiaClinica.agregarConsulta(nuevaConsulta)` pasa el objeto `nuevaConsulta`, no un `idConsulta`).
- **Evidencia:** `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 28): "*De la interfaz vienen los IDs; el controlador busca el objeto para obtener su referencia y a partir de ahí trabaja con referencias a objetos.*"

3.8. Uso Correcto de Agregación (Rombo Blanco) vs. Composición (Rombo Negro) vs. Asociación Simple `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Asociación simple (`——`):** Relación estructural normal entre dos clases independientes (ej. `Consulta` con `Profesional`, `Turno` con `Mascota`, `Nomina` con `Mascota`). **Ante la duda, usar asociación simple con multiplicidad correcta.**
- **Agregación (`◇——` rombo blanco del lado del Todo):** Relación "Todo-Parte" débil donde las partes pueden existir independientemente del Todo y compartirse (ej.: `Equipo` y `Jugador`, `PaqueteTuristico` y `Excursion`).
- **Composición (`◆——` rombo negro relleno del lado del Todo):** Relación "Todo-Parte" **fuerte y de pertenencia exclusiva**. Si se destruye el Todo, carece de sentido o se destruyen sus Partes, y una instancia de la Parte pertenece a un único Todo (ej.: `Factura` ◆— `ItemFactura`, `Prestamo` ◆— `Cuota`, `Mascota` ◆— `HistoriaClinica`, `Aficionado` ◆— `Huella`).
- **Evidencia:** `Transcripcion/transcripciones/03_20260903_101348.txt` (`[00:41:15]`) y `Clases/Clases/05/Banco/Banco-diagClase.png`.

3.9. Clases Abstractas (`<<abstract>>`) vs. Interfaces (`<<interface>>`) `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Clase Abstracta (`<<abstract>>` o nombre en cursiva):**
  - Se usa cuando existe una **relación de identidad "ES UN" (herencia)** entre las subclases y **comparten atributos comunes** y/o **métodos ya implementados**, además de declarar métodos abstractos que cada hija sobrescribe (polimorfismo).
  - *Ejemplo de la Profesora (`Banco/Banco-diagClase.png` y `EJEMPLO-Practica-Teoria-version-10.pdf` Pág. 23):* La clase `Producto` (`<<abstract>>`) tiene `nroProducto`, `fechaAlta` y es padre de `Prestamo` y `Cuenta`. La clase `Pago` (`<<abstract>>`) tiene `monto`, `fecha` y el método polimórfico `+procesarPago()`, siendo padre de `Efectivo`, `Tarjeta` y `Transferencia`.
- **Interfaz (`<<interface>>`):**
  - Es un **contrato puro de comportamiento**: **NO tiene atributos de estado** ni código implementado (en el modelo clásico OO que evalúa la cátedra), solo firmas de métodos públicos abstractos (`+calcular()`, `+notificar()`, `+conectar()`).
  - La relación desde la clase que la implementa hacia la interfaz es de **Realización** (flecha triangular cerrada con **línea punteada** `- - -▷`).
  - Es fundamental en los **Patrones GoF** (`Strategy`, `Observer`, `State`, `Adapter`, `Command`, `Decorator`, `Factory Method`).
- **Regla de notación de la profesora en hoja (`[RECOMENDACIÓN PRÁCTICA DE LA PROFESORA]`):** Como escribiendo a mano no se nota la letra *cursiva*, **escribir SIEMPRE el estereotipo** `<<abstract>>` **o** `<<interface>>` **arriba del nombre de la clase** y también al lado de los métodos abstractos si querés máxima claridad.
- **Evidencia:** `Transcripcion/transcripciones/03_20260903_101348.txt` (`[00:25:10]` a `[00:39:00]`) y `Clases/Clases/04/Gym/GUIA-EjercicioGYM-Estudiantes-2026- V01.docx`.

4. CRITERIOS ESPECÍFICOS PARA EL DIAGRAMA DE SECUENCIA (PUNTO 2 DEL PARCIAL)

El Punto 2 de los parciales de Analía Marín pide construir un **Diagrama de Secuencia** para un flujo específico del enunciado. Sus reglas son súper precisas:

4.1. Regla N° 1: Escribir Arriba el "Escenario Concreto" con Datos Puntuales `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Por qué lo exige la profesora:** Si intentás dibujar en un solo Diagrama de Secuencia todos los `if/else` posibles del sistema (qué pasa si el cliente existe, si no existe, si paga con efectivo, con tarjeta, con transferencia, si lleva 1 o N vacunas...), el diagrama se vuelve un monstruo inmanejable y no te alcanzan las 2 horas del parcial.
- **Qué tenés que hacer (Obligatorio):** Antes de trazar la primera línea de vida, **escribí un párrafo de 2 o 4 renglones arriba de la hoja definiendo el ESCENARIO CONCRETO** que vas a modelar, usando los datos del enunciado o de la pantalla adjunta al parcial:
  - *En Parcial 1C 2026 Com. 4300 (PetCare):* El enunciado ya fija el escenario exacto del **06/06/2026**: primera atención de `Homero` (gato siamés, 13 años, `5.2 kg`, dueño `Juan Pérez`), se aplica 1 vacuna (`Rabia`), se emite 1 receta (`Pastillas desparasitarias`, `1 comprimido`), diagnóstico `SANO`, tratamiento vacío y recordatorio `"volver a verlo en 1 año"` (`01/06/2027`) al presionar `"Guardar Consulta"`.
  - *En Parcial 1C 2026 Com. 1900 (Reencuentro/Adopción y Animales Exóticos):* El enunciado fija el escenario de **Denuncia de Animal Exótico**: un ciudadano avista un **Leopardo** en Rivadavia, Morón, Bs.As., envía foto/video, se calcula peligrosidad `ROJO` y se notifica a **Dirección de Flora y Fauna** y **Policía Ecológica**.
  - *En Parcial 2C 2026 Com. 1900 (Agencia de Viajes):* El Punto 2 pide redactar un escenario concreto que **mejore** el caso de uso a partir del paso 4 para `"realizar Pago"` (por ejemplo: el cliente selecciona un paquete a Bariloche para 2 pasajeros, agrega 1 excursión extra, paga con tarjeta de crédito aprobada, se confirma la `Reserva` y se envía notificación por email).
- **Evidencia:**

- `Parciales/1. [Parcial 1][1C2026][4300][Analia][][].pdf`, `2. -Parcial 1--1C2026--1900--Valeria y Analia-----.pdf` y `3. [Parcial 1][2C2026][1900][][][].pdf`.
- `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 11): Muestra el recuadro con el escenario concreto antes del Diagrama de Secuencia ("*Un paciente solicita un turno para Hemograma y Hepatograma, para el día 15/05/2025 a las 8hs*").

4.2. Regla N° 2: ¡NO Dibujar al Actor Humano en el Diagrama de Secuencia de Diseño! `[REGLA EXPLÍCITA DE CÁTEDRA]`

- **Criterio estricto de la Profesora:** En el Diagrama de Secuencia de Diseño **NO se dibuja el muñequito (stickman) del Actor** (`:Veterinario`, `:Recepcionista`, `:Cliente`) ni tampoco la pantalla (`:UI`).
- **¿Cómo empieza el Diagrama de Secuencia?** Empieza en el extremo superior izquierdo con un **mensaje entrante** (`——→ :C_Modulo`) que llega directamente a la línea de vida del **Controlador** (`:C_...`).
- **Justificación teórica de la Profesora (pregunta de examen):** Porque gracias al patrón GRASP **Controlador** y a la separación de capas (MVC), al modelo de diseño del backend **no le importa quién disparó el evento** (pudo haber sido una secretaria haciendo clic en una web, un cliente desde una app móvil, un tótem autoservicio o un proceso batch por API). El sistema es agnóstico de la interfaz.
- **Evidencia:**
  - `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 11 y Pág. 28): "*En el diagrama de secuencia NO ponemos al actor. Empezamos directamente desde el evento que llega al controlador, porque no importa si lo dispara una persona desde una pantalla o un servicio.*"
  - `Transcripcion/transcripciones/05_20260917_101702.txt` (`[00:45:10]`) y `09_20261015_082342.txt` (`[01:50:00]`).

4.3. Regla N° 3: Creación de Objetos en Tiempo de Ejecución (`create()`) `[CONVENCIÓN PARTICULAR DE CÁTEDRA vs. ESTÁNDAR UML]`

- **Estándar UML 2.x (Larman):** En UML estándar, la flecha del mensaje `create(...)` apunta directamente a la cajita `:NombreClase` ubicada a la altura vertical del instante en que nace el objeto.
- **Convención de la Prof. Analía Marín en clase y guías (`EJEMPLO-Practica-Teoria-version-10.pdf` Pág. 11 y `Banco-diagSecuencia.png`):**
  - La profesora acepta tanto que la cajita del objeto esté alineada arriba con las demás como a la altura del `create()`, pero **exige como regla obligatoria** que la activación de creación lleve explícitamente el mensaje `create(...)` (y/o el retorno de la instancia creada) y que **quien envíe el** `create()` **sea el objeto Creador (GRASP)** (por ejemplo, `:Orden` creando `:itemOrden`, o `:HistoriaClinica` / `:Consulta` creando sus partes).
  - Además, en el Caso Laboratorio (`EJEMPLO-Practica-Teoria-version-10.pdf`, Págs. 11–12), muestra también la creación mediante el patrón GoF **Prototype** enviando `clone()` a `:especificacionEstudio` para obtener una nueva instancia `:estudio`.
- **Evidencia:** `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Págs. 11–12) y `Transcripcion/transcripciones/07_20261001_082247.txt` (`[00:12:15]`).

4.4. Regla N° 4: Respetar el Alcance Exacto Pedido en el Punto de Secuencia `[REGLA EXPLÍCITA DE CÁTEDRA]`

- Leé con lupa qué parte del enunciado te pide secuenciar el examen:
  - En **Parcial PetCare 1C 2026 (Com. 4300)**, el Punto 2 pide secuenciar el evento `"Guardar Consulta"` para los datos cargados del gato `Homero` (creación y asociación de `Consulta`, `Diagnostico`, `Receta`, `VacunaAplicada`, `Recordatorio` y agregado a `HistoriaClinica`).
  - En **Parcial Reencuentro/Adopción 1C 2026 (Com. 1900)**, el Punto 3 pide secuenciar los **7 pasos de la Denuncia de Animal Exótico** (avistamiento del Leopardo en Morón, registro de `AnimalExotico` y `UbicacionHallazgo`, cálculo de peligrosidad, notificación a Flora y Fauna y Policía Ecológica, marcado de `organismoNotificado = SI` y guardado de `Denuncia`).
  - En **Parcial Agencia de Viajes 2C 2026 (Com. 1900)**, el Punto 2 pide secuenciar `"realizar Pago"` mejorando el escenario a partir del paso 4.
- **Moraleja:** En el **Diagrama de Clases** modelás **TODO** el sistema del enunciado, pero en el **Diagrama de Secuencia** solo secuenciás el escenario exacto que pide la consigna.

---

5. CRITERIOS PARA LOS DEMÁS DIAGRAMAS UML (CU, ACTIVIDAD, COLABORACIÓN, ESTADO, COMPONENTES, DESPLIEGUE)

En los parciales 2026, además de Clases y Secuencia, **los tres exámenes incluyen Diagrama de Componentes y/o Diagrama de Estados** (ya sea para integrarlos al Diagrama de Clases o para criticarlos y rediseñarlos), y en finales se evalúa también **Despliegue**, **Colaboración**, **Actividad** y **Casos de Uso**. Aquí están las reglas exactas de la profesora para cada uno:

5.1. Casos de Uso y su Especificación (`EJEMPLO-Practica-Teoria-version-10.pdf` Págs. 4–7 y Clase 03)

- **Nombres de Casos de Uso:** Siempre **Verbo en infinitivo + Objeto** (ej. `Solicitar Turno`, `Registrar Atención`, `Pagar Cuota`).
- `<<include>>` **vs** `<<extend>>`:
  - `CU_Base - - - <<include>> - - > CU_Incluido`: El paso incluido es **obligatorio** y siempre se ejecuta como parte del flujo normal (o sirve para reutilizar un subflujo común a varios CU, ej. `Validar Identidad`). La flecha apunta hacia el caso incluido.
  - `CU_Extensor - - - <<extend>> - - > CU_Base`: El comportamiento extensor es **opcional/condicional** (solo ocurre bajo cierta condición o punto de extensión, ej. `Registrar Paciente` cuando el paciente no existe). **¡OJO con el sentido de la flecha!** En `<<extend>>`, la flecha punteada **apunta hacia el Caso de Uso Base**.
- **Formato de Especificación preferido por la Profesora (Forma 2 — Tabla de 2 Columnas, Págs. 5–6):**
  - Encabezado con: *Nombre del CU, Actores, Propósito, Precondiciones, Postcondiciones*.
  - **Curso Normal (Camino Feliz):** Tabla dividida en dos columnas: **Acción del Actor** (izquierda) y **Respuesta del Sistema** (derecha), numerando los pasos correlativamente.
  - **Cursos Alternativos:** Al pie de la tabla, referenciando el número exacto del paso donde se bifurca.

5.2. Diagrama de Actividad (`EJEMPLO-Practica-Teoria-version-10.pdf` Pág. 7 y Clase 03/06)

- Representa el flujo de trabajo (workflow) paso a paso, permitiendo mostrar **decisiones** (rombos con condiciones `[si]` / `[no]` mutuamente excluyentes) y **actividades paralelas/concurrentes** mediante **Barras de Sincronización**:
  - **Fork (Bifurcación):** Entra 1 flecha a la barra negra gruesa y salen 2 o más flechas que se ejecutan en paralelo.
  - **Join (Unión):** Entran las N ramas paralelas a la barra negra gruesa y sale 1 sola flecha recién cuando **todas** las ramas paralelas terminaron.
- Se divide en **Carriles (Swimlanes)** verticales u horizontales por área o responsable.

5.3. Diagrama de Colaboración / Comunicación (`EJEMPLO-Practica-Teoria-version-10.pdf` Pág. 12 y Clase 04)

- **Criterio de la Profesora:** Es **semánticamente isomorfo (equivalente)** al Diagrama de Secuencia (ambos son Diagramas de Interacción).
- **Diferencia clave que siempre pregunta en Teoría:**
  - El **Diagrama de Secuencia** enfatiza el **orden temporal** de los mensajes (de arriba hacia abajo a lo largo de las líneas de vida).
  - El **Diagrama de Colaboración (Comunicación)** enfatiza la **estructura estática de enlaces (links)** entre los objetos que colaboran, indicando el orden temporal mediante **numeración correlativa o jerárquica** en las etiquetas de los mensajes (`1: msg1()`, `2: subMsg()`, `3: otroMsg()`).

5.4. Diagrama de Estados (`EJEMPLO-Practica-Teoria-version-10.pdf` Pág. 13, Parciales 1 y 3 de 2026, Finales Dic 2025 y Jul 2026)

- **Regla N° 1 (¡El error más grave!):** El Diagrama de Estados **SIEMPRE modela el ciclo de vida de UN ÚNICO OBJETO de una clase específica del dominio** (ej. un objeto `Consulta`, un objeto `Reserva`, un objeto `Orden`, un objeto `Articulo`, un objeto `Aficionado`). **¡NUNCA es un diagrama de flujo del sistema entero!** (Si mezclás estados del `Cliente`, del `Pago` y del `Envío` en el mismo diagrama, está **MAL**).
- **Regla N° 2 (Nombres de Estados vs. Nombres de Transiciones):**
  - **Dentro de las cajas redondeadas (Estados):** Van **adjetivos o participios** que describen la situación en la que se encuentra el objeto: `Registrada`, `Diagnosticada`, `Cerrada` (en *PetCare*); `Pendiente de pago`, `Confirmada`, `Cancelada`, `Finalizada` (en *Agencia de Viajes*); `Ingresada`, `Validada`, `En Proceso`, `Cerrada` (en *Laboratorio*). (**PROHIBIDO** poner verbos de acción como "*Pagar*" o "*Validar tarjeta*" adentro de una caja de estado).
  - **Sobre las flechas (Transiciones):** Va la sintaxis oficial `evento [condición de guarda] / acción` (donde `evento` es el **método** que recibe el objeto, `[guarda]` es la condición booleana entre corchetes, y `/ acción` es la respuesta inmediata).
- **Regla N° 3 (Integración con el Diagrama de Clases y Patrón GoF State):**
  - En los Parciales 1 y 3 de 2026, la hoja del examen **te da dibujado el Diagrama de Estados** (`Consulta Veterinaria` o `Reserva`) y te pide **integrarlo en el Diagrama de Clases**.
  - Eso significa: (1) asegurarte de que todos los eventos de las flechas (`registrarConsulta()`, `registrarDiagnostico()`, `cerrarConsulta()`, `pagoRealizado()`, `cancelar()`, `cerrarViaje()`) estén como métodos en la clase `Consulta` o `Reserva`; y (2) si aplicás el patrón GoF **State**, crear la interfaz `<<interface>> IEstadoConsulta` / `IEstadoReserva` y una subclase concreta por cada cajita de estado del diagrama impreso.

5.5. Diagrama de Componentes (`EJEMPLO-Practica-Teoria-version-10.pdf` Pág. 14, `Banco-diagComponente.png`, `ComponenteVersion2.png` y Parciales 1, 2 y 3 de 2026) `[CONVENCIÓN PARTICULAR DE CÁTEDRA vs. ESTÁNDAR UML]`

- Muestra la arquitectura modular de alto nivel del software (subsistemas/módulos).
- **Las DOS notaciones que utiliza la cátedra:**
  - **Notación Estándar UML 2.x (enseñada en `UNLaM GRASP-original-marin.pdf` Págs. 12–15 y `Banco-diagComponente.png`):** Componentes con el ícono `<<component>>` unidos mediante **Interfaz Provista (chupetín / círculo completo `—○`)** e **Interfaz Requerida (media luna / enchufe `—( `)** ensamblados (`—○)—`), y conectores de delegación desde los puertos.
  - **Notación Simplificada de Cajas Anidadas y Dependencias (usada en `EJEMPLO-Practica-Teoria-version-10.pdf` Pág. 14 y en las hojas impresas de los 3 Parciales 2026):** Módulos representados como contenedores con sub-módulos internos (ej. `gestion/usuario`, `mascota`, `turno`, `AtencionVeterinario` con `historiaClinica`) conectados mediante **flechas punteadas de dependencia (`- - - >`)** hacia los componentes de los cuales dependen (incluyendo sistemas externos como `Sist Externo Pago`, `Sist Ext. Aerolinea`, `Organismos Externos`).
- **Regla de Oro de Trazabilidad entre Componentes y Controladores (evaluada en Parcial 2 y Parcial 3 de 2026):**
  - **Cada Componente funcional principal del sistema se corresponde con 1 `Controller` (`C_...`) en el Diagrama de Clases.**
  - Por eso, cuando el Parcial 2 (*Reencuentro/Adopción*) y el Parcial 3 (*Agencia de Viajes*) imponen **"usar solo 4 controladores"**, te piden **modificar o mejorar el Diagrama de Componentes** para que los componentes queden agrupados exactamente en **4 componentes cohesivos de negocio** (más las interfaces hacia sistemas externos), evitando tanto el "monolito acoplado" como tener 8 componentes sueltos sin controlador.

5.6. Diagrama de Despliegue (`EJEMPLO-Practica-Teoria-version-10.pdf` Págs. 15–16, `Banco-diagDespliegueV2.png` y `Despliegue2.png`) `[CONVENCIÓN PARTICULAR DE CÁTEDRA vs. ESTÁNDAR UML]`

- Modela la **topología física de hardware**, los entornos de ejecución, los artefactos desplegados y los **protocolos de comunicación** en los enlaces.
- **Los DOS estilos que presenta la profesora en `EJEMPLO-Practica-Teoria-version-10.pdf` (Págs. 15 y 16):**
  - **Diagrama de Despliegue UML 2.0 Estándar (Pág. 15 y `Banco-diagDespliegueV2.png`):** Nodos representados como **cubos 3D** (`<<device>> Servidor Web`, `<<device>> Servidor Aplicacion`, `<<device>> Servidor BD`, `<<device>> PC Cliente`) que contienen artefactos (`.war`, `.jar`, `.dll`) y se conectan indicando el protocolo (`HTTPS`, `TCP/IP`, `JDBC`, `REST/JSON`, `SOAP/XML`).
  - **Diagrama de Despliegue por Dominios / Zonas de Red (Pág. 16 y `Despliegue2.png`):** Diagrama arquitectónico dividido mediante líneas punteadas en **Dominios o Ambientes físicos separados** (ej.: `Dominio Laboratorio` con Red Local LAN vs. `Dominio Obra Social` externo vs. `Dominio Paciente` a través de Internet/Firewall, o en Estadios: `Tótem Local` vs. `Servidor Estadio` vs. `Servidor Central AFA` vs. `Renaper`).
- **En el examen:** Podés utilizar los cubos 3D estándar de UML 2.0 agrupándolos dentro de recuadros punteados que marquen las **Zonas/Ambientes** y escribiendo siempre los **protocolos** (`HTTPS`, `REST/JSON`, `TCP/IP`, `MQTT`, `USB`) sobre las líneas de comunicación.

---

6. CÓMO RESPONDER LAS JUSTIFICACIONES DE PATRONES EN EL PARCIAL

6.1. Cómo Justificar los Patrones GRASP (Punto 1 del Parcial) `[REGLA EXPLÍCITA DE CÁTEDRA]`

En los parciales 2026 (`1. [Parcial 1]...`, `2. -Parcial 1...`, `3. [Parcial 1]...`), se pide aplicar y justificar los patrones GRASP en el diseño.

- **Regla de Oro de la Profesora: ¡NUNCA des una definición teórica genérica de libro!** Si escribís *"Apliqué Experto porque asigna la responsabilidad a la clase que tiene la información"*, te pone **0 puntos o Incompleto**.
- **Fórmula obligatoria para justificar cada GRASP en el examen:**
  > **[Nombre del Patrón GRASP] → ¿En qué clase concreta de TU diagrama lo aplicaste?** + **¿En qué método concreto?** + **¿Por qué esa clase (y no otra) tiene los atributos/referencias o el rol para hacerlo?**
  - *Ejemplo 10 puntos (Experto en Información):* *"Se aplica **Experto en Información** en la clase `HistoriaClinica` con el método `+registrarConsulta(c)`, y en la clase `PaqueteTuristico` con el método `+calcularPrecioBase()`, ya que son las clases que poseen los atributos y las colecciones de objetos propios necesarios para cumplir esa responsabilidad."*
  - *Ejemplo 10 puntos (Creador):* *"Se aplica **Creador** en la clase `Consulta` (o `HistoriaClinica`) al instanciar `Diagnostico`, `Receta` y `VacunaAplicada`, ya que `Consulta` agrega y contiene de forma exclusiva a dichos objetos registrándolos como parte de la atención médica."*
  - *Ejemplo 10 puntos (Bajo Acoplamiento y Alta Cohesión / Polimorfismo / Fabricación Pura / Indirección):* *"Se aplica **Alta Cohesión y Bajo Acoplamiento** al delegar la responsabilidad del registro clínico en `HistoriaClinica` y `Consulta` en lugar de sobrecargar al controlador `C_AtencionVeterinaria` o a la clase `Profesional`, manteniendo cada clase enfocada únicamente en las responsabilidades propias de su abstracción."*

6.2. Cómo Elegir, Dibujar y Justificar los 2 Patrones GoF `[REGLA EXPLÍCITA DE CÁTEDRA]`

En los parciales 2026, el punto de Patrones GoF vale una parte decisiva de la nota y pide:

> 🎓 **CRITERIO Y RECOMENDACIÓN DE LA PROFESORA**
> *"Aplicar dos patrones GOF (de diferente categoría) al problema planteado sobre el diagrama de clases:*
> *a) Indicar categoría y nombre del patrón.*
> *b) Explicar qué problema de diseño resuelve dentro del caso.*
> *c) Justificar por qué ese patrón es adecuado.*
> *d) Indicar qué clases del modelo se reutilizan y qué clases/interfaces nuevas se incorporan al diagrama."*
> *(Nota explícita en Parcial 2 de 2026: **"no usar singleton, ni facade"**).*

- **Criterios de corrección de la Profesora (Clase 08 y Clase 09 `[02:15:00]` a `[03:30:00]`):**
  - **Tienen que ser de DISTINTA CATEGORÍA:** Es decir, elegís 1 **Comportamiento** + 1 **Estructural**, o 1 **Comportamiento** + 1 **Creacional**, o 1 **Estructural** + 1 **Creacional**.
  - **Evitar `Singleton` y `Facade` en los parciales:** Incluso si en tu tema no dice explícitamente "no usar singleton ni facade", en la Clase 09 la profesora aclaró que muchos alumnos metían un Singleton en el Controller sin justificación real de negocio, por lo que empezaron a prohibirlos en las consignas.
  - **El patrón GoF DEBE verse dibujado en el Diagrama de Clases con toda su estructura UML completa:**
  - Si aplicás **Strategy** (Comportamiento): Tiene que verse la clase Contexto (ej. `Reserva` / `CalculadoraPrecio` en *Agencia de Viajes*), la interfaz `<<interface>> IPoliticaPrecio` con el método abstracto (`+calcularPrecio(...)`), y al menos **2 o 3 clases concretas** (`PoliticaTemporadaAlta`, `PoliticaGrupoFamiliar`, `PoliticaAnticipada`) que realizan (`- - -▷`) la interfaz.
  - Si aplicás **State** (Comportamiento): Tiene que verse el Contexto (ej. `Consulta` en *PetCare*, `Animal` en *Reencuentro/Adopción*, o `Reserva` en *Agencia de Viajes*) con el atributo `-estadoActual: IEstado...`, la interfaz `<<interface>> IEstado...` con los métodos de transición y las **clases concretas de estado** que coinciden 1 a 1 con el Diagrama de Estados del enunciado.
  - Si aplicás **Observer** (Comportamiento): Ideal en el **Parcial 2 (*Denuncia de Animal Exótico*)** para notificar automáticamente a **Dirección de Flora y Fauna** y **Policía Ecológica**, o en **PetCare** para los recordatorios por WhatsApp/Mail. Tiene que verse el Sujeto observado con su lista `-observadores: List<IObservador>` y métodos `+suscribir(o)`, `+desuscribir(o)`, `+notificar()`, la interfaz `<<interface>> IObservador` y los observadores concretos.
  - Si aplicás **Adapter** (Estructural): Ideal siempre que el enunciado mencione **conectarse a un organismo o sistema externo con interfaz incompatible** (`OrganismoFaunaFloraBsAs`, `PoliciaEcologica`, `SistExtAerolinea`, `SistExtHotel`, `SistExternoPago`, `VISA`, `Veraz`). Tiene que verse la interfaz esperada por tu dominio (`<<interface>> IOrganismoExterno` o `IPasarelaPago`), la clase `Adaptador...` que implementa esa interfaz y envuelve (asociación) a la clase externa traduciendo el llamado.
  - Si aplicás **Decorator** o **Composite** (Estructural): Ideal para paquetes de viaje con excursiones/servicios adicionales o tratamientos compuestos.
  - Si aplicás **Factory Method**, **Builder** o **Prototype** (Creacional): `Prototype` es usado por la propia profesora en el Caso Laboratorio (`especificacionEstudio.clone()`); `Builder` es ideal para armar un `PaqueteTuristico` personalizado; `Factory Method` para crear distintos tipos de denuncias o estudios.

---

7. CHECKLIST RÁPIDO DE CONTROL ANTES DE ENTREGAR EL PARCIAL (LOS 12 MANDAMIENTOS DE ANALÍA)

Antes de entregar las hojas en el examen, verificá estos 12 puntos uno por uno:

- [ ] **1. ¿Entregué un diagrama por hoja y numeré todas las hojas?**
- [ ] **2. ¿Respeté el límite máximo de `Controllers` (`max: 4 controladores`) en la parte superior del Diagrama de Clases y están alineados con los componentes principales del Diagrama de Componentes?**
- [ ] **3. ¿Me aseguré de que ningún Actor humano (`Recepcionista`, `Veterinario`, `Ciudadano`) esté actuando como un Gestor lleno de métodos del sistema?**
- [ ] **4. ¿Eliminé todos los métodos `get/set` triviales y los `alta/baja/modificación` genéricos, dejando métodos con nombres de reglas de negocio?**
- [ ] **5. ¿Separé correctamente la "Ficha individual" (`Consulta`, `Mascota`, `Aficionado`) de la clase que maneja "la Colección" (`HistoriaClinica` para las consultas de esa mascota, o `Nomina` / `Padron` con asociación simple para búsquedas globales)?**
- [ ] **6. ¿Puse visibilidades (`-` en atributos, `+` en métodos) y multiplicidades (`1`, `0..*`, `1..*`) en todas las asociaciones sólidas del Diagrama de Clases (recordando que la visibilidad local no se dibuja en el DC)?**
- [ ] **7. ¿Escribí el estereotipo `<<interface>>` o `<<abstract>>` arriba del nombre de todas las interfaces y clases abstractas?**
- [ ] **8. ¿Integré todos los eventos y estados del Diagrama de Estados impreso en la hoja dentro de la clase correspondiente del Diagrama de Clases?**
- [ ] **9. ¿Escribí arriba del Diagrama de Secuencia el ESCENARIO CONCRETO con los datos puntuales del caso e inicié la secuencia SIN Actor humano, entrando directo al `:C_Controller`?**
- [ ] **10. ¿Verifiqué uno por uno que TODOS los mensajes que llegan a un objeto `:X` en el Diagrama de Secuencia están escritos como métodos `+metodo()` dentro de la clase `X` en el Diagrama de Clases?**
- [ ] **11. ¿Justifiqué los patrones GRASP nombrando las clases y métodos exactos de MI diagrama en lugar de recitar teoría genérica?**
- [ ] **12. ¿Elegí 2 patrones GoF de DISTINTA categoría (sin usar Singleton ni Facade), los dibujé completos en el Diagrama de Clases y respondí los 4 incisos (`a`, `b`, `c`, `d`)?**

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 2: GUÍA COMPLETA DE UML (TODOS LOS DIAGRAMAS, NOTACIÓN Y PROCEDIMIENTOS)

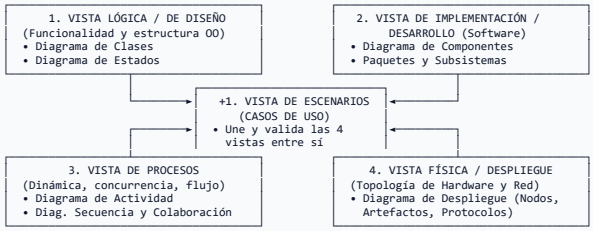
📑 ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO

PARTE 0: EL MARCO INTEGRADOR — LAS 4+1 VISTAS DE KRUCHTEN Y RUP Pág. 18
A.1. ¿Qué representa y para qué sirve? Pág. 18
A.3. Cómo identificar Atributos y Métodos (y dónde ubicarlos) Pág. 19
A.5. Visibilidad: de Atributos/Métodos y entre Objetos (Larman Cap. 18 y Clase 07) Pág. 20
A.7. Agregación (◇——) vs. Composición (◆——) Pág. 21
A.9. Dependencias (- - - >) Pág. 21
B. DIAGRAMA DE CASOS DE USO Y ESPECIFICACIÓN (CLASE 03 Y MODELO LABORATORIO) Pág. 22
B.2. Plantilla Oficial de Especificación de Caso de Uso (Forma 2 — Dos Columnas de Analía Marín) Pág. 22
D. DIAGRAMA DE SECUENCIA DE DISEÑO (CLASE 04, 05, 06, 07 Y 09) Pág. 23
E. DIAGRAMA DE COMUNICACIÓN / COLABORACIÓN (CLASE 04 Y MODELO LABORATORIO PÁG. 18) Pág. 23
F.1. ¿Qué representa y cuándo se usa? Pág. 24
G. DIAGRAMA DE COMPONENTES (CLASE 04, 05, 06 Y PARCIALES 2026) Pág. 24
H. DIAGRAMA DE DESPLIEGUE (CLASE 04, 05, 06 Y 11) Pág. 24
A. DIAGRAMA DE CLASES (EL CORAZÓN DEL PARCIAL) Pág. 18
A.2. Cómo identificar las clases a partir de un enunciado (Método de los 3 Colores de la Profesora + Pasaje a Diseño) Pág. 19
A.4. Cómo determinar Responsabilidades (Conocer vs. Hacer — Larman Cap. 16 y Clase 04) Pág. 19
A.6. Asociaciones, Multiplicidad (Cardinalidad) y Navegabilidad Pág. 20
A.8. Generalización / Herencia (——▷), Clases Abstractas (<<abstract>>) e Interfaces (<<interface>>) Pág. 21
A.10. Clases Asociativas / Intermedias (Cuándo romper un Muchos-a-Muchos * —— *) y Solucionario del Caso Gimnasio (GUIA-EjercicioGYM-Estudiantes-2026- V01.docx) Pág. 22
B.1. Elementos del Diagrama de Casos de Uso Pág. 22
C. DIAGRAMA DE ACTIVIDAD (CLASE 03 Y CLASE 06) Pág. 23
D.1. Elementos y Convenciones de la Cátedra Pág. 23
F. DIAGRAMA DE ESTADOS (CLASE 03, 04, 06 Y FINALES DIC 2025 / JUL 2026) Pág. 24
F.2. Sintaxis Completa de un Estado y de una Transición (UNLaM GRASP-original-marin.pdf Págs. 2–5) Pág. 24
G.1. Las DOS Notaciones que utiliza la Cátedra (Conocer ambas para el Parcial) Pág. 24
H.1. Los DOS Estilos de Diagrama de Despliegue de la Profesora (EJEMPLO-Practica-Teoria-version-10.pdf Págs. 15 y 16, Banco-diagDesplieguev2.png y Despliegue2.png) Pág. 25

PARTE 0: EL MARCO INTEGRADOR — LAS 4+1 VISTAS DE KRUCHTEN Y RUP

Antes de dibujar cualquier diagrama, debés comprender dónde se ubica cada uno dentro de la arquitectura del sistema (**Pregunta fija de parcial y final**: `01-Diseño de Sistemas v2.pdf` Págs. 30–46, `Parcial 1C 2024`, `Parcial 2023`, `Final Julio 2026`).

DIAGRAMA / ESTRUCTURA TÉCNICA UML



Vista de Kruchten	¿A quién le interesa?	¿Qué muestra?	Diagramas UML Asociados (según Cátedra)	Atributos de Calidad que evalúa
1. Vista Lógica (o de Diseño)	Diseñadores, Desarrolladores y Usuarios finales	La descomposición orientada a objetos del sistema: qué abstracciones existen y cómo cumplen los requerimientos funcionales.	**Diagrama de Clases**, **Diagrama de Objetos**, **Diagrama de Estados** *(Nota: en la cátedra a veces agrupan Secuencia/Colaboración entre Vista Lógica y de Procesos).*	Reusabilidad, Mantenibilidad, Extensibilidad.
2. Vista de Procesos	Integradores de sistemas, Arquitectos	La dinámica en tiempo de ejecución: hilos de ejecución, concurrencia, sincronización y flujo de mensajes.	**Diagrama de Actividad** (con barras Fork/Join), **Diagramas de Interacción (Secuencia y Colaboración)**.	Performance (Rendimiento), Disponibilidad, Concurrencia, Tolerancia a fallos.
3. Vista de Implementación (o Desarrollo)	Programadores, Gestores de Configuración	Cómo se organiza el software en módulos, librerías, paquetes y subsistemas dentro del entorno de desarrollo.	**Diagrama de Componentes**, **Diagrama de Paquetes**.	Modularidad, Mantenibilidad, Facilidad de compilación y prueba.
4. Vista Física (o de Despliegue)	Ingenieros de Sistemas, Redes, DevOps	Cómo se mapean los componentes de software sobre los nodos físicos de hardware y cómo se comunican por red.	**Diagrama de Despliegue** (*Deployment*).	Escalabilidad, Seguridad física/red, Disponibilidad, Rendimiento de red.
+1. Vista de Escenarios (Casos de Uso)	Todos (Clientes, Analistas, Testers, Arquitectos)	Es el "pegamento" que articula e integra las otras 4 vistas a través de flujos de negocio de punta a punta.	**Diagrama de Casos de Uso** y **Especificación de Casos de Uso**.	Completitud funcional, Usabilidad, Trazabilidad.

A. DIAGRAMA DE CLASES (EL CORAZÓN DEL PARCIAL)

A.1. ¿Qué representa y para qué sirve?

- **Definición [Larman Cap. 19 / Sommerville Cap. 5]:** El Diagrama de Clases es un diagrama **estructural y estático** perteneciente a la **Vista Lógica / de Diseño**. Representa los tipos de objetos (clases) que conforman el sistema, su estructura interna (atributos y operaciones/métodos) y las relaciones estáticas permanentes o transitorias que existen entre ellos.
- **Diferencia entre Modelo de Dominio (Análisis) y Diagrama de Clases de Diseño (DCD) [Criterio Cátedra - Clase 03 y 05]:**
  - En **Análisis**, el modelo de dominio es un "diccionario visual" de conceptos del mundo real sin métodos de software ni tipos de visibilidad.
  - En **Diseño de Sistemas (lo que te evalúa Analía Marín)**, construimos el **Diagrama de Clases de Diseño (DCD)**. Sirve como **plano directo previo a la codificación** e incluye:
  - Clases de negocio con **atributos tipados** y **métodos con firmas reales**.
  - **Visibilidad** explícita (`-`, `+`, `#`).
  - **Navegabilidad** y **Multiplicidad** precisa.

- **Clases de Fabricación Pura / Diseño:** Controladores (`C_...`), Contenedores/Nóminas (`Padron`, `HistoriaClinica`), Interfaces (`<<interface>>`), Clases Abstractas (`<<abstract>>`) y estructuras de **Patrones GoF**.

A.2. Cómo identificar las clases a partir de un enunciado (Método de los 3 Colores de la Profesora + Pasaje a Diseño)

En `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Págs. 5–6 y Pág. 8), en `Clases/Clases/03/03-CU-GYM-2026-09-03.docx` y en la **Clase 03 Parte 2 (`03_20260903_101348.txt`, `[00:52:00]`)**, la profesora enseña el **Método de Análisis del Texto con Código de Colores** y luego cómo transformarlo al Diseño:

Paso 1: Subrayado del Enunciado y de los Casos de Uso con 3 Colores (`EJEMPLO-Practica-Teoria-version-10.pdf` Págs. 5–6 y 8)

Al leer el texto del caso (y mirar la captura de pantalla o los diagramas base si el parcial los incluye):

1. **Color AMARILLO (Sustantivos con entidad propia) → Clases Candidatas Principales del Dominio:**
   - Buscá sustantivos que tengan **múltiples datos propios**, **ciclo de vida** o **reglas de negocio asociadas**.
   - *Ejemplo en Laboratorio (`EJEMPLO-Practica...` Pág. 8):* `Paciente`, `Turno`, `Orden`, `Estudio`, `Muestra`, `Resultado`, `Factura`, `ObraSocial`.
   - *Ejemplo en Veterinaria PetCare:* `Veterinaria`, `Usuario`, `Mascota`, `HistoriaClinica`, `Consulta`, `Diagnostico`, `Tratamiento`, `VacunaAplicada`, `Receta`, `Estudio`, `Recordatorio`, `Turno`, `Profesional`.
2. **Color VERDE (Verbos y frases verbales de negocio) → Funcionalidad / Comportamiento (Métodos):**
   - En la guía oficial de la profesora (`EJEMPLO-Practica-Teoria-version-10.pdf`, Págs. 5–6 y 8), el color **verde** resalta la **funcionalidad/comportamiento** (los verbos de negocio): "*solicitar turno*", "*registrar orden*", "*validar cobertura*", "*guardar consulta*", "*calcular peligrosidad*", "*notificar organismo*".
   - **Nota sobre el color verde en el Diagrama de Clases (`EJEMPLO-Practica...` Pág. 10):** Además, cuando dibuja el Diagrama de Clases del Laboratorio, la profesora pinta de **color verde la caja de la clase pasarela hacia el sistema externo (`sisExtOS`)** para distinguir visualmente la frontera de integración externa respecto de las clases propias del dominio (amarillas).
3. **Color CELESTE / AZUL CLARO (Sustantivos simples o soporte) → Atributos o Clases Secundarias:**
   - Si un sustantivo es un simple dato primitivo (texto, número, fecha, booleano) que describe a una clase amarilla y no tiene comportamiento propio, **NO es una clase, es un atributo** (`-nombre: String`, `-peso: float`, `-fechaVencimiento: Date`).
   - **¿Cuándo una característica pasa de ser Atributo (Celeste) a ser Clase (Amarillo)? [Pregunta de Clase 05 y 09]:**
   - Si "Especie" o "Categoría" es solo un texto informativo sin reglas propias, puede ser un atributo.
   - Pero si de la `Categoria`, `Plan`, `Especialidad` o `EspecificacionEstudio` dependen **precios, descuentos, valores de referencia, tiempos de ayuno o reglas de validación**, entonces **DEBE ser una Clase** asociada (para no hardcodear valores y permitir **Extensibilidad**).

Paso 2: Filtrado de "Falsas Clases" [Criterio Analía Marín]

Una vez que tenés tus sustantivos candidatos, **eliminá o transformá** estos 4 errores clásicos:

1. **El Sistema mismo:** Si el enunciado dice "*El sistema registra...*", **NO crees una clase** `Sistema` (Atención: en el Parcial 1C 2026 *PetCare*, como es una **Red de Veterinarias** donde el usuario busca sucursales por ubicación y horarios y cada sucursal tiene sus profesionales, la clase `Veterinaria` **SÍ es una clase del dominio** que representa cada sucursal física de la red).
2. **Los Actores Operadores:** Si el texto dice "*La recepcionista carga el turno*" o "*El administrativo imprime...*", `Recepcionista` es el **Actor** que usa la pantalla, **NO es una clase gestora**. Solo modelá una persona como clase del dominio si el negocio guarda datos sobre ella (ej. `Usuario`, `Profesional` que atiende el `Turno` o `Consulta` con su matrícula y horarios).
3. **Pantallas, Formularios y Reportes:** "*Pantalla de Atención*", "*Formulario HTML*", "*Listado por pantalla*" pertenecen a la Vista (UI), **NO van como clases del modelo de dominio**.
4. **Sustantivos sinónimos:** Si en un párrafo dice "*Dueño*" y en otro dice "*Usuario/Cliente*", unificalos en una sola clase.

Paso 3: Incorporación de las Clases de Diseño (Fabricación Pura)

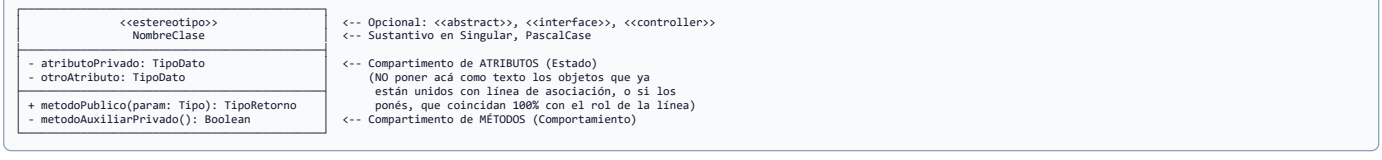
Al modelo de clases de dominio filtrado, agregale las clases propias de **Diseño**:

1. **Los Controladores (`C_...`):** Respetando el tope del parcial (**máximo 4 Controladores** en los parciales 2026) en la parte superior del diagrama (1 por cada gran componente funcional del sistema).
2. **Los Contenedores / Nóminas (solo cuando se requiere búsqueda global):** Clases que contienen colecciones globales de fichas individuales (`Padron` para `Aficionado`s, `Nomina` para buscar por ID), mientras que las colecciones propias de un objeto las administra su propio contenedor de dominio (`HistoriaClinica` para las `Consulta`s de una `Mascota`).
3. **Las Interfaces y Clases de los 2 Patrones GoF.**

A.3. Cómo identificar Atributos y Métodos (y dónde ubicarlos)

Estructura visual de una clase en UML:

DIAGRAMA / ESTRUCTURA TÉCNICA UML



- **Reglas para Atributos [Criterio Cátedra]:**
  - Van siempre con visibilidad **privada (`-`)** (o protegida `#` si es una superclase de una jerarquía de herencia) para respetar el **Encapsulamiento y Ocultamiento de la Información**.
  - **No poner Claves Foráneas de Base de Datos (`idCliente_FK`, `idFactura_FK`):** ¡Estamos en Orientación a Objetos, no en un DER de Base de Datos! En objetos, `Mascota` no tiene `-idDueno: int`, sino que hay una **línea de asociación** entre `Cliente` y `Mascota` (que en código equivale al atributo de referencia `-mascota: List<Mascota>` o `-dueno: Cliente`).
- **Reglas para Métodos [Criterio Cátedra]:**
  - Sus nombres siempre comienzan con un **verbo en infinitivo en `camelCase`**: `+calcularSubtotal()`, `+validarVigencia()`, `+agregarVacuna(v: VacunaAplicada)`.
  - Van con visibilidad **pública (`+`)** si son invocados por otros objetos en el Diagrama de Secuencia, o **privada (`-`)** si es un sub-cálculo interno (mensaje recursivo `self` que no debe ser llamado desde afuera).

A.4. Cómo determinar Responsabilidades (Conocer vs. Hacer — Larman Cap. 16 y Clase 04)

En el PDF de la profesora (`Clases/Clases/04-Presencial/UNLaM GRASP-original-marin.pdf`, Págs. 20–23) y en Larman, una **Responsabilidad** es un **contrato u obligación de un clasificador (clase)** y se divide en dos tipos:

1. **Responsabilidades de CONOCER (Estado / Atributos y Asociaciones):**

- Conocer sus propios datos privados encapsulados (atributos).
- Conocer los objetos relacionados con los que está conectado (asociaciones).
- Conocer las cosas que puede derivar o calcular a partir de lo que posee.

2. **Responsabilidades de HACER (Comportamiento / Métodos):**
   - Hacer algo él mismo (ej. crear un objeto o realizar un cálculo con sus propios datos → **Experto**).
   - Iniciar una acción en otros objetos (enviarles mensajes).
   - Controlar y coordinar actividades en otros objetos (rol del **Controlador**).

> 🎓 **CRITERIO Y RECOMENDACIÓN DE LA PROFESORA**
>
> **Traducción práctica para el parcial:**
>
> *"Una responsabilidad NO es un método, sino que las responsabilidades se implementan mediante uno o más métodos que colaboran entre sí"* (`UNLaM GRASP-original-marin.pdf`, Pág. 20).
>
> Para saber **en qué clase poner un método**, hacete siempre la pregunta del **Experto en Información (GRASP)**: *¿Qué clase tiene los atributos o las referencias necesarias para responder esto?* Esa clase debe tener el método.

A.5. Visibilidad: de Atributos/Métodos y entre Objetos (Larman Cap. 18 y Clase 07)

Hay **dos conceptos de visibilidad** que debés dominar tanto para la práctica como para las preguntas teóricas:

1. Visibilidad de Miembros dentro de la Clase (Modificadores de Acceso UML)

Símbolo	Nombre	Significado en POO y Cuándo Usarlo en el Parcial
`-`	**Privado (`private`)**	Solo visible dentro de la misma clase. **Usalo en TODOS los atributos de las clases concretas** y en métodos internos auxiliares (mensajes `self`).
`+`	**Público (`public`)**	Visible por cualquier otra clase que tenga referencia a esta. **Usalo en los métodos que reciben mensajes desde otras clases en el Diagrama de Secuencia.**
`#`	**Protegido (`protected`)**	Visible por la propia clase y por sus **subclases (hijas)**. **Usalo en los atributos de las Clases Abstractas / Superclases** (ej. en `Pago`, `Producto`, `Bien`) para que sus hijas accedan a ellos directamente.
`~`	**Paquete (`package`)**	Visible solo por clases del mismo paquete/componente. (Rara vez se usa en el parcial escrito).

2. Visibilidad entre Objetos (¿Cómo puede el objeto `A` enviarle un mensaje al objeto `B`?)

Explicado por Analía Marín en **Clase 07 (`07_20261001_082247.txt`)** y en `EJEMPLO-Practica-Teoria-version-10.pdf` **(Pág. 19 y Págs. 32–33)**, basado en **Craig Larman (Cap. 18)**:

Para que un objeto emisor `A` pueda enviar un mensaje a un objeto receptor `B` en el Diagrama de Secuencia, `A` tiene que "ver" (tener una referencia a) `B` de alguna de estas **4 maneras**:

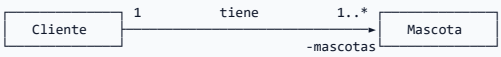
Tipo de Visibilidad (Larman)	¿Cómo se logra en código?	¿Cuándo ocurre en el Parcial?	¿Aparece en el Diagrama de Clases según Analía Marín (`EJEMPLO-Practica...` Pág. 19)?
1. Visibilidad de Atributo (Permanente)	`B` está declarado como un atributo de instancia dentro de la clase `A` (`private B miB;`). Persiste mientras `A` y `B` existan.	`Mascota` tiene su `HistoriaClinica`; `Consulta` tiene su `Diagnostico`, `Receta` y `VacunaAplicada`; `Factura` tiene sus `ItemFactura`.	**SÍ, SIEMPRE.** Se dibuja como **Línea continua (sólida) de Asociación / Agregación / Composición** entre `A` y `B`.
2. Visibilidad de Parámetro (Transitoria)	`B` se pasa como argumento de un método de `A`: `A.calcular(B unObjetoB)`. Dura solo mientras se ejecuta ese método.	El `Controller` busca la `Vacuna` y se la pasa por parámetro a `consulta.aplicarVacuna(v)` (o si `Consulta` guarda `v` en un atributo propio, se convierte en Visibilidad de Atributo).	**SÍ (si se detallan firmas o dependencias).** Se ve en la firma `+metodo(b: B)` o con **flecha punteada de Dependencia (`A - - - > B`)**.
3. Visibilidad Local (Transitoria)	Dentro de un método de `A`, se declara una variable local a la que se le asigna un objeto `B` (`B b = new B()` o `B b = otroObjeto.getB()`).	El `C_Atencion` llama a `hc = mascota.obtenerHistoriaClinica()` y usa `hc` localmente dentro de ese método.	**NO APARECE en el Diagrama de Clases** (*Cuadro oficial Pág. 19: "¿Aparece en el Diagrama de Clases?: No"*). Solo se observa en el **Diagrama de Secuencia**.
4. Visibilidad Global	`B` es accesible globalmente desde cualquier punto (ej. a través de un método estático `Singleton.getInstance()`).	Cuando una clase accede a una configuración global o servicio único.	Solo si se modela el patrón `Singleton`.

A.6. Asociaciones, Multiplicidad (Cardinalidad) y Navegabilidad

1. Asociación (———)

Representa una conexión estructural permanente (**visibilidad de atributo**) entre instancias de dos o más clases.

DIAGRAMA / ESTRUCTURA TÉCNICA UML



- **Nombre de la asociación:** Verbo opcional en el medio de la línea (`tiene`, `solicita`, `emite`).
- **Nombre de rol (`-mascotas`):** Indica cómo llama la clase origen al atributo que referencia a la clase destino.

2. Multiplicidad y Cardinalidad

Indica **cuántas instancias** de la clase de ese extremo pueden vincularse con **una única instancia** de la clase del otro extremo:

- `1` (o `1..1`): Exactamente uno (obligatorio y único). Ej.: Una `HistoriaClinica` pertenece a `1` `Mascota`.
- `0..1`: Cero o uno (opcional). Ej.: Una `Consulta` puede generar `0..1` `Receta` (si en algunas consultas no se receta nada).
- `0..*` (o simplemente `*`): Cero o muchos. Ej.: Un `Cliente` puede haber realizado `0..*` `SolicitudViaje`.
- `1..*`: Uno o muchos (al menos uno obligatorio). Ej.: Una `Factura` debe tener `1..*` `DetalleFactura` (no existe una factura con 0 renglones).

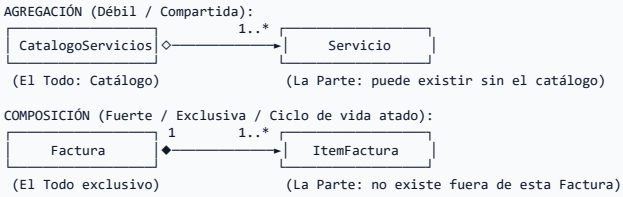
3. Navegabilidad (———→)

- Define la dirección en la que se puede recorrer la asociación en tiempo de ejecución (quién tiene el atributo que apunta a quién):
  - **Unidireccional (`A` ———→ `B`):** `A` conoce a `B` (`A` tiene un atributo de tipo `B`), pero `B` **no** conoce a `A`. **En Diseño se prefiere la navegabilidad unidireccional siempre que alcance**, porque reduce el acoplamiento (si `B` no necesita enviarle mensajes a `A`, no debe conocer a `A`).
  - **Bidireccional (`A` ——— `B` sin flechas en los extremos):** `A` conoce a `B` y `B` conoce a `A`. Solo usala si en el negocio se navega en ambos sentidos.

A.7. Agregación (`◇—`) vs. Composición (`◆—`)

Ambas son casos particulares de Asociación que modelan relaciones **"Todo / Parte"** (*Whole-Part*). **¡REGLA CRUCIAL DE DIBUJO: El rombo SIEMPRE se dibuja pegado a la clase "TODO" (la contenedora), NUNCA pegado a la parte!**

DIAGRAMA / ESTRUCTURA TÉCNICA UML



Característica	Agregación (Rombo Blanco `◇—`)	Composición (Rombo Negro Relleno `◆—`)
Fuerza del vínculo	**Débil ("tiene un").**	**Fuerte / Exclusiva ("está compuesto por").**
Ciclo de vida (Destrucción)	**Independiente:** Si se elimina el Todo (`Equipo`), las Partes (`Jugador`) **siguen existiendo** en el sistema.	**Dependiente:** Si se destruye el Todo (`Factura` , `Prestamo` , `HistoriaClinica`), sus Partes (`ItemFactura` , `Cuota` , `Consulta`) pierden sentido o se eliminan con él.
Exclusividad de la Parte	Una misma instancia de la Parte puede ser compartida por varios Todos (multiplicidad del lado del rombo puede ser `*`).	Una instancia de la Parte pertenece a **UN SOLO Todo a la vez** (la multiplicidad del lado del rombo negro es siempre `1`).
Ejemplos de las Clases de Analía Marín	- `Club` ◇— `Socio` - `Combo` ◇— `Producto` - `Equipo` ◇— `Jugador`	- `Prestamo` ◆— `Cuota` (`Banco/Banco-diagClase.png`) - `Aficionado` ◆— `Huella` y `Iris` (`Estadio/Clase2.png`) - `Mascota` ◆— `HistoriaClinica` ◆— `Consulta` (Clase 09) - `Orden` ◆— `itemOrden` (`EJEMPLO-Practica...` Pág. 10)

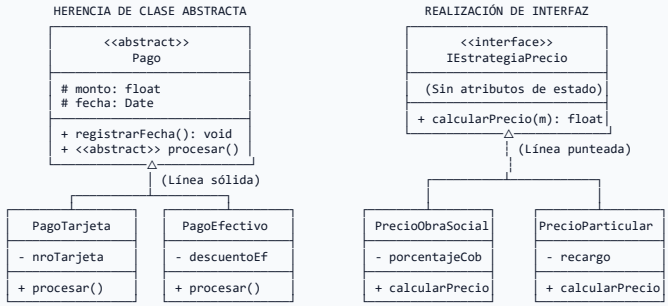
A.8. Generalización / Herencia (`—▷`), Clases Abstractas (`<<abstract>>`) e Interfaces (`<<interface>>`)

1. Generalización / Herencia (`Subclase —▷ Superclase` — línea continua con triángulo blanco cerrado apuntando al Padre)

- Representa una relación taxonómica **"ES UN" (*is-a*)**, nunca un "tiene un".
- La subclase **hereda todos los atributos y métodos** de la superclase, y puede **agregar** atributos/métodos propios o **sobrescribir (redefinir)** métodos del padre (**Polimorfismo**).
- **Prueba de validación en el parcial:** Preguntate siempre: "*¿Un `PrestamoHipotecario` ES UN `Prestamo`?*" (Sí → Herencia). "*¿Un `Cliente` cambia de tipo durante su vida?*" Si un objeto necesita cambiar de clase en tiempo de ejecución o combinar varios tipos a la vez, **NO uses herencia estática** en la clase principal; usá **Composición / Patrón Strategy / State / Decorator** (Principio GoF: "*Favorecer la composición de objetos sobre la herencia de clases*").

2. Clase Abstracta vs. Interfaz (Cuadro Comparativo para Práctica y Teoría — Clase 03 Parte 2)

DIAGRAMA / ESTRUCTURA TÉCNICA UML



Criterio	Clase Abstracta (`<<abstract>>`)	Interfaz (`<<interface>>`)
¿Se puede instanciar (`new`)?	**No.** Solo se instancian sus subclases concretas.	**No.** Solo se instancian las clases concretas que la implementan.
¿Tiene atributos de estado?	**Sí** (generalmente protegidos `#` para factorizar datos comunes a todas las subclases).	**No** tiene atributos de estado (solo constantes estáticas si hiciera falta).
¿Tiene métodos implementados?	**Sí**, puede combinar métodos concretos (con código heredado) y métodos abstractos (`<<abstract>>`).	**No** (en el modelo OO puro de cátedra): solo declara **firmas de métodos públicos abstractos** (el "QUÉ" hacer, sin el "CÓMO").
Herencia múltiple	Una clase solo puede heredar (`—▷`) de **1 sola** superclase (en Java/C#).	Una clase puede implementar (`- - -▷`) **múltiples** interfaces a la vez.
Flecha UML	**Generalización:** Línea **continua** con punta triangular vacía (`—▷`) apuntando a la clase abstracta.	**Realización:** Línea **punteada** con punta triangular vacía (`- - -▷`) apuntando a la interfaz.

A.9. Dependencias (`- - - >`)

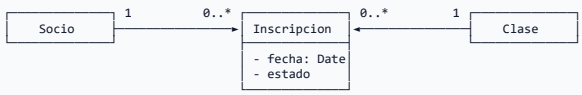
- **Notación:** Línea **punteada** con punta de flecha abierta (`A - - - > B`).
- **Qué significa:** Es la relación más débil entre clases ("**USA UN**" temporalmente). Indica que un cambio en la especificación de `B` puede afectar a `A` , pero `A` **no** guarda una referencia permanente a `B` como atributo.
- **Cuándo aparece:**
  - `A` recibe un objeto de tipo `B` como **parámetro** en uno de sus métodos (**Visibilidad de Parámetro**).
  - El creador en un patrón **Factory Method** (`Creator - - - > ConcreteProduct`) que instancia y devuelve el producto sin quedárselo como atributo.
    *(Recordar que la **Visibilidad Local** no se dibuja en el Diagrama de Clases según el cuadro de la Prof. Analía Marín en `EJEMPLO-Practica-Teoria-version-10.pdf` , Pág. 19).*

A.10. Clases Asociativas / Intermedias (Cuándo romper un Muchos-a-Muchos `* — *`) y Solucionario del Caso Gimnasio (`GUIA-EjercicioGYM-Estudiantes-2026- V01.docx`)

En `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Pág. 28) y en `Clases/Clases/04/Gym/GUIA-EjercicioGYM-Estudiantes-2026- V01.docx`, la profesora explica las **3 Reglas de Craig Larman** para saber cuándo una relación entre dos clases (`Socio` y `Clase` de gimnasio, o `Usuario` y `Veterinaria` a través de `Turno`) **requiere una clase intermedia/asociativa** (`Inscripcion` o `Turno`):

1. **Regla 1:** Cuando existe un atributo que pertenece al vínculo mismo y no a ninguna de las dos clases por separado (por ejemplo: `fechaInscripcion`, `estadoInscripcion`, `calificacion` pertenecen a `Inscripcion`, no al `Socio` ni a la `Clase`).
2. **Regla 2:** Cuando las instancias de la clase asociativa tienen un **ciclo de vida independiente** o histórico (un mismo `Socio` puede inscribirse en varias fechas o darse de baja y volver a inscribirse, quedando el historial de `Inscripcion` es).
3. **Regla 3:** Cuando la relación original es **Muchos a Muchos (`* —— *`)** (`Un Socio se inscribe a muchas Clases, y una Clase tiene muchos Socios`) y necesitamos gestionar el cupo, la asistencia o el estado de cada vínculo individual:

DIAGRAMA / ESTRUCTURA TÉCNICA UML



Solucionario Oficial de los 10 Espacios en Blanco de `Clases/Clases/04/Gym/GUIA-EjercicioGYM-Estudiantes-2026- V01.docx`

En la guía de autoevaluación del Caso Gimnasio, la profesora dejó 10 oraciones con espacios en blanco (`.....`) sobre lectura del Diagrama de Clases y Secuencia. Aquí están las respuestas exactas verificadas con `EJEMPLO-Practica-Teoria-version-10.pdf` (Págs. 27–33):

1. *"En la parte superior del diagrama de clases se ubican los `Controladores (Controllers)`."*
2. *"Evitar pensar que cada clase tiene un controller para hacer un `ABM (Alta, Baja y Modificación)`. Eso en los parciales NO VA. Siempre vamos al `CORE (corazón)` del negocio."*
3. *"Para que un socio pueda inscribirse a una clase del gimnasio, debemos verificar que posea `apto médico vigente`, que tenga la `cuota al día` y que la clase tenga `cupo disponible`."*
4. *"Entre `Socio` y `Clase` existe una relación muchos a muchos, por lo que aparece la clase intermedia `Inscripcion` que registra la fecha y el estado."*
5. *"Cada mensaje que llega a un objeto en el diagrama de secuencia es un `método` en la clase de ese objeto."*
6. *"Si hay mensajes en el diagrama de interacción (secuencia o colaboración) que no están en el diagrama de `Clases`, el ejercicio está `MAL`."*
7. *"En el diagrama de secuencia de diseño NO se dibuja al `Actor`, sino que comienza directamente con el evento que llega al `Controlador`."*
8. *"Desde la interfaz de usuario llegan datos primitivos o `IDs`, con los cuales el controlador busca al objeto para obtener su `referencia`."*
9. *"La visibilidad de `Atributo` es permanente y se representa con una línea de `Asociación` en el diagrama de clases."*
10. *"El patrón GRASP `Experto (en Información)` asigna la responsabilidad a la clase que tiene la información necesaria para cumplirla."*

B. DIAGRAMA DE CASOS DE USO Y ESPECIFICACIÓN (CLASE 03 Y MODELO LABORATORIO)

B.1. Elementos del Diagrama de Casos de Uso

1. **Límite del Sistema (Rectángulo contenedor):** Encierra todos los óvalos (Casos de Uso) que pertenecen al software a construir. Los actores quedan **afuera** del rectángulo.
2. **Actores (`Stickman`):** Rol externo que interactúa con el sistema:
   - **Actor Primario (a la izquierda):** Quien inicia el Caso de Uso para alcanzar un objetivo (ej. `Socio`, `Paciente`, `Recepcionista`).
   - **Actor Secundario / Sistema Externo (a la derecha):** Quien es consultado o notificado durante la ejecución del CU para que éste pueda completarse (ej. `Sistema ARCA/AFIP`, `Sistema Veraz`, `Pasarela de Pago VISA`, `Renaper`).
3. **Relaciones entre Casos de Uso:**
   - **Inclusión (`CU Base - - - <<include>> - - > CU Incluido`):** Comportamiento **obligatorio** que siempre se ejecuta o que se factoriza porque es común a varios casos de uso.
   - **Extensión (`CU Extensor - - - <<extend>> - - > CU Base`):** Comportamiento **opcional / condicional** que se dispara solo si se cumple una condición en un punto de extensión. **¡Recordá siempre que la flecha de `<<extend>>` apunta hacia el Caso de Uso Base!**

B.2. Plantilla Oficial de Especificación de Caso de Uso (Forma 2 — Dos Columnas de Analía Marín)

Tomada de `Clases/Clases/03/03-FORMATO-VACIO-2026-2c.docx` y `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (Págs. 3–7):

Campo	Contenido Requerido
Nombre del Caso de Uso	Verbo en infinitivo + Sustantivo (ej.: *Solicitar Turno de Laboratorio*)
Actor Primario	*Paciente / Recepcionista*
Actores Secundarios	*Servicio de Validación de Obra Social*
Precondiciones	Lo que **debe ser verdadero en el sistema ANTES** de que empiece el CU (ej.: *El usuario debe haber iniciado sesión; las especialidades deben estar cargadas*).
Postcondiciones	Cómo queda el estado del sistema **DESPUÉS** de terminar el CU: - **De éxito:** *El turno queda registrado en estado Confirmado y se descuenta el cupo de la agenda.* - **De fallo / alternativa:** *No se registra el turno y la agenda permanece sin cambios.*

Curso Normal (Camino Feliz):

Acción del Actor	Respuesta del Sistema
1. El Actor solicita iniciar el registro de un nuevo turno e ingresa el DNI del paciente.	**2.** El Sistema busca al paciente en la nómina, valida que exista y muestra sus datos personales y obra social asociada.
3. El Actor carga las prácticas médicas indicadas en la orden y selecciona la fecha y hora deseada.	**4.** El Sistema verifica los requisitos de ayuno de cada práctica, comprueba la disponibilidad en la agenda y valida la cobertura con la Obra Social.
5. El Actor confirma la reserva del turno.	**6.** El Sistema registra el turno, actualiza la disponibilidad de la agenda y emite el comprobante con las indicaciones de preparación. Fin del CU.

Cursos Alternativos / Excepciones:

- **2a. El paciente no se encuentra registrado en el sistema:**
  - `2a.1` El Sistema informa que el DNI no existe y habilita la carga de datos de nuevo paciente.

- `2a.2` El Actor ingresa nombre, apellido, fecha de nacimiento, contacto y obra social.
- `2a.3` El Sistema registra al paciente y continúa en el paso **3**.

- **4a. La Obra Social rechaza la cobertura del estudio:**
  - `4a.1` El Sistema informa el rechazo de cobertura y ofrece cotizar el estudio en modalidad Particular.

C. DIAGRAMA DE ACTIVIDAD (CLASE 03 Y CLASE 06)

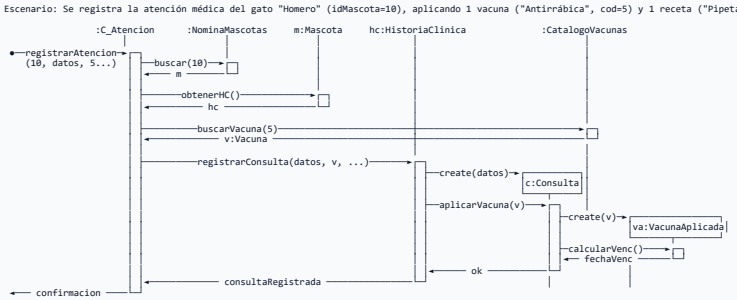
- **Qué representa:** Modela el flujo de trabajo (*workflow*) procedimental paso a paso de un proceso de negocio o de un Caso de Uso, incluyendo sus bifurcaciones condicionales y **operaciones concurrentes (en paralelo)**.
- **Notación exigida por la Profesora (`EJEMPLO-Practica-Teoria-version-10.pdf` Págs. 9–10 y `Estadio/Actividades.jpg`):**
  - **Carriles (*Swimlanes*):** Columnas verticales que dividen quién ejecuta cada actividad (ej. `Cliente` | `Sistema Web` | `Servicio Externo`).
  - **Nodo Inicial (`●`) y Nodo Final de Actividad (`◉`).**
  - **Acciones (Rectángulos con bordes redondeados):** Siempre nombradas con **Verbo + Objeto** (`Validar Huella`, `Calcular Monto`).
  - **Nodo de Decisión (`◇` Rombo):** Entra 1 flecha y salen 2 o más flechas con **condiciones de guarda mutuamente excluyentes entre corchetes** (`[Huella Válida]`, `[Huella Inválida]`).
  - **Barras de Sincronización (Concurrencia):**
  - **Barra Fork (División en paralelo):** Entra 1 flecha y salen 2 o más flechas que se ejecutan al mismo tiempo (ej. en Caso Estadios: validar en paralelo con Renaper y con Policía).
  - **Barra Join (Sincronización / Unión):** Entran las ramas paralelas y sale 1 sola flecha cuando todas finalizaron.

D. DIAGRAMA DE SECUENCIA DE DISEÑO (CLASE 04, 05, 06, 07 Y 09)

Es el **Punto 2 obligatorio de todos los parciales**. Muestra la interacción dinámica entre instancias (objetos) ordenada cronológicamente de arriba hacia abajo.

D.1. Elementos y Convenciones de la Cátedra

DIAGRAMA / ESTRUCTURA TÉCNICA UML



1. **Cajas de Objetos (Cabeceras de Líneas de Vida):**
   - Sintaxis: `nombreInstancia : NombreClase` (ej. `m : Mascota` o simplemente `:Mascota` o `:C_Atencion`), subrayado según el estándar UML porque representa una **instancia**, no la clase estática.
   - Si es una colección o contenedor de objetos del mismo tipo (multiobjeto), en StarUML o en papel se puede representar como la clase contenedora (`:Padron`, `:HistoriaClinica`, `:NominaMascotas`) o como doble cajita superpuesta `:Mascota` *(recordando siempre la regla de Analía Marín: si usás una nómina o contenedor en Secuencia, ¡debe existir en el Diagrama de Clases!)*.
2. **Mensaje Entrante del Sistema (`●——→ :C_Controller`):**
   - **SIN Actor humano.** Parte de un punto negro (o desde el borde izquierdo) hacia la barra de activación del Controlador.
3. **Tipos de Flechas de Mensajes:**
   - **Mensaje Sincrónico (`——→`) línea sólida con punta rellena):** El objeto emisor espera a que el receptor termine su foco de control (barra vertical de activación) para continuar.
   - **Mensaje de Retorno (`<- - - -` línea punteada con flecha abierta):** Devuelve el resultado o la referencia al objeto buscado (`m`, `hc`, `true`, `montoTotal`).
   - **Mensaje de Creación (`——→` con etiqueta `create(...)` apuntando directo a la caja `:ObjetoNuevo`):**La caja del objeto recién instanciado se dibuja más abajo en el eje vertical (en el instante exacto en que nace), no arriba de todo junto con los objetos preexistentes.
   - **Auto-mensaje / Mensaje Recursivo (`self`):** Flecha que sale de la barra de activación de un objeto y vuelve a entrar a la misma barra (un rectángulo superpuesto sobre la activación) cuando el objeto llama a un método privado propio (ej. `calcularVencimiento()`).
4. **Fragmentos Combinados (Marcos de control UML 2):**
   - `alt [condición 1] ... [else]`: Equivalente a un `if / else` (camino mutuamente excluyente).
   - `opt [condición]`: Equivalente a un `if` sin `else` (paso opcional).
   - `loop [1..N / para cada ítem]`: Equivalente a un `for / foreach / while` (iteración sobre una colección).

E. DIAGRAMA DE COMUNICACIÓN / COLABORACIÓN (CLASE 04 Y MODELO LABORATORIO PÁG. 18)

- **Qué representa:** Es el segundo tipo de **Diagrama de Interacción** de UML. Muestra exactamente la **misma información** que el Diagrama de Secuencia, pero en lugar de usar un eje de tiempo vertical, dispone a los objetos como un **grafo de nodos conectados por enlaces (*links*)** y expresa el orden temporal mediante **números con punto decimal (numeración jerárquica de Dewey)**.
- **Cómo se numera (Anidamiento de mensajes):**
  - El mensaje inicial que entra al controlador es `1: registrarAtencion(...)`.
  - Todos los mensajes que el controlador dispara dentro de la ejecución del mensaje `1` se numeran `1.1: buscarMascota(id)`, `1.2: obtenerHC()`, `1.3: registrarConsulta(...)`.
  - A su vez, los mensajes que `:HistoriaClinica` dispara dentro de `1.3` se numeran `1.3.1: create()`, `1.3.2: aplicarVacuna(v)`, y así sucesivamente.

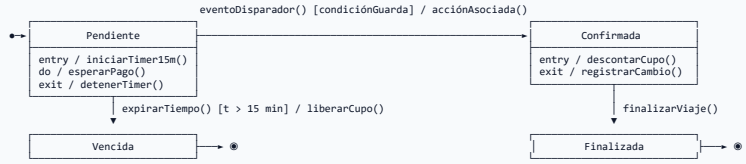
F. DIAGRAMA DE ESTADOS (CLASE 03, 04, 06 Y FINALES DIC 2025 / JUL 2026)

F.1. ¿Qué representa y cuándo se usa?

- Modela el **ciclo de vida completo de UNA instancia de una clase específica** (reactiva a eventos), mostrando todos los **estados** posibles por los que atraviesa desde que se crea hasta que se destruye o archiva, y qué **eventos (métodos)** provocan las **transiciones** de un estado a otro.

F.2. Sintaxis Completa de un Estado y de una Transición (`UNLaM GRASP-original-marin.pdf` Págs. 2–5)

DIAGRAMA / ESTRUCTURA TÉCNICA UML



1. **Estado Inicial (Pseudonodo inicial `●`):** Círculo negro relleno. Todo diagrama de estados tiene **exactamente un** punto de inicio. La flecha que sale de `●` al primer estado suele llevar el evento constructor (`crear()`).
2. **Estado Simple o Compuesto (Rectángulo de bordes redondeados):**
   - **Nombre del Estado:** Siempre en **Adjetivo / Participio** (`Solicitado`, `Pendiente de Pago`, `Confirmado`, `En Preparación`, `Entregado`, `Cancelado`).
   - **Actividades internas opcionales del estado:**
   - `entry / acción`: Se ejecuta instantáneamente al **entrar** al estado.
   - `do / actividad`: Actividad continua que se ejecuta **mientras** permanece en ese estado.
   - `exit / acción`: Se ejecuta instantáneamente justo antes de **salir** del estado.
3. **Transición (Flecha dirigida de un Estado Origen a un Estado Destino):**
   - Etiqueta formal: `evento(parámetros) [condición de guarda] / acción`
   - `evento`: Es el **nombre del método** que recibe el objeto (y que debe figurar en el Diagrama de Clases) o un evento temporal (`after(15 min)`).
   - `[condición de guarda]`: Expresión booleana entre corchetes. La transición solo se dispara si ocurre el evento **Y** la guarda da `true` (ej. `[stock >= cantidadPedida]`).
   - `/ acción`: Proceso atómico e ininterrumpible que se ejecuta durante el disparo de la transición (ej. `/ emitirNotificacion()`).
4. **Estado Final (`◉` Círculo negro rodeado por un anillo):** Indica que el objeto terminó su ciclo de vida (no admite más transiciones).

G. DIAGRAMA DE COMPONENTES (CLASE 04, 05, 06 Y PARCIALES 2026)

- **Ubicación: Vista de Implementación / Desarrollo** de Kruchten.
- **Qué representa:** Muestra cómo se divide el sistema de software en **piezas modulares, reemplazables y reutilizables (Componentes / Subsistemas)** y cómo dependen unas de otras.

G.1. Las DOS Notaciones que utiliza la Cátedra (Conocer ambas para el Parcial)

1. Notación Estándar UML 2.x (Con Interfaces Lollipop/Socket y Puertos — `UNLaM GRASP-original-marin.pdf` Págs. 12–15 y `Banco-diagComponente.png`)

1. **Componente (`<<component>>` o ícono de bloque con dos pestañitas):** Agrupa un conjunto cohesivo de clases (ej. `Gestión de Turnos`, `Atención Clínica`, `Facturación`, `Integración Externa`).
2. **Interfaz Provista (Notación "Lollipop" / Chupetín `——○`):** Servicios u operaciones que el componente **expone y ofrece** al resto del sistema.
3. **Interfaz Requerida (Notación "Socket" / Enchufe `——(`):** Servicios que el componente **necesita consumir** de otro componente para poder funcionar.
4. **Conector de Ensamblaje (*Assembly Connector* `——○)——`):** Une la interfaz requerida de un componente con la interfaz provista de otro.
5. **Puertos (Cuadraditos sobre el borde del componente):** Puntos de interacción explícitos con el entorno (usados para conectores de delegación hacia subcomponentes internos).

2. Notación Simplificada de Cajas Anidadas y Dependencias (Usada en `EJEMPLO-Practica-Teoria-version-10.pdf` Pág. 14 y en los 3 Parciales 2026) `[CONVENCIÓN PARTICULAR DE CÁTEDRA]`

En los enunciados impresos de los Parciales 1, 2 y 3 de 2026 y en la página 14 de `EJEMPLO-Practica-Teoria-version-10.pdf`, la profesora dibuja los componentes como **cajas contenedoras de módulos (con sub-cajas internas)** unidas por **flechas punteadas de dependencia (`- - - >`)**:

DIAGRAMA / ESTRUCTURA TÉCNICA UML



- **Regla de Oro de Trazabilidad Componente ↔ Controlador (Evaluada en Parcial 2 y Parcial 3 de 2026):**
  - En el criterio de la cátedra, **cada Componente funcional principal del sistema se traduce en 1 `Controller` (`C_...`) en el Diagrama de Clases**.
  - Si el parcial pide **"usar solo 4 controladores"** y **"modificar/mejorar el Diagrama de Componentes"**, debés agrupar los módulos de negocio con **Alta Cohesión y Bajo Acoplamiento** en **4 componentes principales** (uno por cada uno de tus 4 controladores), dejando fuera del monolito a los **Sistemas Externos** conectados mediante interfaces/dependencias bien definidas.

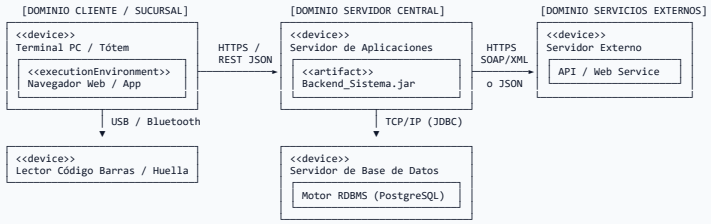
H. DIAGRAMA DE DESPLIEGUE (CLASE 04, 05, 06 Y 11)

- **Ubicación: Vista Física / de Despliegue** de Kruchten.
- **Qué representa:** La arquitectura física de **Hardware (Nodos)** en tiempo de ejecución, qué **artefactos/componentes de software** corren dentro de cada nodo y mediante qué **protocolos de comunicación/red** se conectan los nodos entre sí.

H.1. Los DOS Estilos de Diagrama de Despliegue de la Profesora (`EJEMPLO-Practica-Teoria-version-10.pdf` Págs. 15 y 16, `Banco-diagDesplieguev2.png` y `Despliegue2.png`)

1. **Estilo 1 — UML 2.0 Clásico de Nodos 3D y Artefactos (Pág. 15 y `Banco-diagDesplieguev2.png`):**
   - Muestra los **Nodos 3D (`<<device>>`)**, los entornos de ejecución (`<<executionEnvironment>>`), los **Artefactos (`<<artifact>>`)** desplegados adentro y los protocolos en cada línea.
2. **Estilo 2 — Arquitectura Física Separada por Dominios / Zonas de Red (Pág. 16 y `Despliegue2.png`) `[CONVENCIÓN PARTICULAR DE CÁTEDRA]`:**
   - Divide el plano con líneas punteadas en **Dominios o Ambientes de Red** claramente diferenciados (ej. en Laboratorio Pág. 16: `Dominio Paciente` | `Dominio Laboratorio (Red Local LAN: Servidor Web, Servidor App, Servidor BD, Impresora Código de Barras)` | `Dominio Obra Social`; en Bancos: `Cliente` | `Sucursal` | `Sede Central` | `Servicios Externos VISA/Veraz`; en Estadios: `Tótem Local` | `Servidor Estadio` | `Servidor Central AFA` | `Renaper/Policía`).

DIAGRAMA / ESTRUCTURA TÉCNICA UML



1. **Nodos (Cubos 3D):** Representan recursos computacionales físicos (`<<device>>`: Servidor, PC, Smartphone, Tótem, Microcontrolador PLC/Arduino en STR, Sensor, Actuador) o entornos de ejecución de software (`<<executionEnvironment>>`: Navegador Web, JVM, Contenedor Docker, Motor de BD).
2. **Artefactos (`<<artifact>>`):** Archivos físicos ejecutables desplegados dentro de los nodos (`.jar`, `.war`, `.dll`, `.exe`, scripts, esquemas de base de datos).
3. **Asociaciones de Comunicación (Líneas sólidas entre Nodos): SIEMPRE deben tener escrito el protocolo de red o bus físico** (`HTTPS`, `REST / JSON`, `SOAP / XML`, `TCP/IP`, `JDBC`, `MQTT`, `USB`, `Bluetooth`).
4. **Agrupación por Dominios / Ambientes:** Encerrar o rotular claramente cada zona de red (`Dominio Cliente`, `Dominio Sucursal/Local`, `Dominio Sede Central`, `Dominio Servicios Externos`).

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 3: PATRONES GRASP — FUNDAMENTOS, LOS 9 PATRONES, EJEMPLOS Y JUSTIFICACIONES

📑 ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO

PARTE 1: FUNDAMENTOS TEÓRICOS DE PATRONES, COHESIÓN Y ACOPLAMIENTO	**Pág. 26**	1.1. ¿Qué es un Patrón de Diseño y cuáles son sus Elementos Esenciales? (Pregunta Clásica de Examen)	**Pág. 26**
1.2. Diferencia Fundamental entre Patrones GRASP y Patrones GoF	**Pág. 26**	1.3. Grados de Cohesión y Acoplamiento (Diapositivas de Analía Marín y Pregunta de Final Julio 2026)	**Pág. 26**
1.4. Principios de Diseño Modular y Patrones Arquitectónicos MVC / MVVM (UNLaM GRASP-original-marin.pdf Págs. 50 y 63–71)	**Pág. 27**	**PARTE 2: LOS 9 PATRONES GRASP Y CÓMO JUSTIFICARLOS EN EL PARCIAL (PUNTO 1)**	**Pág. 27**

PARTE 1: FUNDAMENTOS TEÓRICOS DE PATRONES, COHESIÓN Y ACOPLAMIENTO

1.1. ¿Qué es un Patrón de Diseño y cuáles son sus Elementos Esenciales? (Pregunta Clásica de Examen)

- **Origen Histórico (`GOF-2Q-2026.pdf` Págs. 2–7):** Nace en la **Arquitectura civil** en 1977 con **Christopher Alexander** ("*Cada patrón describe un problema que ocurre una y otra vez en nuestro entorno, y describe el núcleo de la solución a ese problema, de tal manera que puedas usar esta solución un millón de veces más sin hacerlo dos veces igual*"). En 1994, la **"Banda de los Cuatro" (GoF — *Gang of Four*: Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides)** publica el libro *Design Patterns*, catalogando los 23 patrones clásicos para diseño orientado a objetos.
- **Los Elementos Esenciales de un Patrón (`GOF-2Q-2026.pdf` Pág. 16 y `Parcial 1C 2024`):**
  - **Nombre del patrón:** Vocabulario común que permite comunicar la idea de diseño en una palabra.
  - **Problema / Objetivo (Intención y Motivación):** Describe el problema específico de diseño y el contexto en que se presenta.
  - **Aplicabilidad (Fuerzas):** En qué situaciones se debe aplicar (y cuándo no).
  - **Solución (Estructura y Participantes):** Describe los elementos (clases, interfaces, relaciones y responsabilidades) que componen el diseño de forma abstracta, sin atarse a un código puntual.
  - **Consecuencias (Pros, Contras y Compromisos / *Trade-offs*):** Los resultados, beneficios (ej. mayor extensibilidad, menor acoplamiento) y costos (ej. mayor número de clases) de aplicar el patrón.

1.2. Diferencia Fundamental entre Patrones GRASP y Patrones GoF

Criterio	Patrones GRASP (Craig Larman)	Patrones GoF (Gamma, Helm, Johnson, Vlissides)
Naturaleza	Son **9 principios fundamentales** para **asignar responsabilidades** a clases y objetos mientras diseñamos diagramas de interacción y de clases.	Son **23 soluciones estructurales/conductuales concretas** (plantillas de clases e interfaces) para resolver problemas recurrentes de diseño OO.
¿Cómo se ven en el diagrama?	Se reflejan en **quién tiene cada método y quién llama a quién** (asignación de métodos y relaciones).	Se reflejan como un **sub-grafo concreto de interfaces y clases** (ej. `Contexto` → `<<interface>> Strategy` ← `ConcreteStrategyA/B`).

1.3. Grados de Cohesión y Acoplamiento (Diapositivas de Analía Marín y Pregunta de Final Julio 2026)

En `UNLaM GRASP-original-marin.pdf` (Págs. 39, 46 y 47) y en el **Final de Julio 2026**, se evalúan explícitamente los niveles de **Acoplamiento** y **Cohesión**:

A. Los 6 Tipos de Acoplamiento (De PEOR / Más Alto a MEJOR / Más Bajo)

> 🎓 **CRITERIO Y RECOMENDACIÓN DE LA PROFESORA**
> **Definición:** El acoplamiento mide el **grado de interdependencia o fuerza con que un módulo/clase está conectado a otros**, tiene conocimiento de ellos o depende de ellos. **Buscamos SIEMPRE BAJO ACOPLAMIENTO.**

1. **Acoplamiento de Contenido (El PEOR — Prohibido):** Un módulo `A` accede o modifica directamente los datos privados o el código interno de otro módulo `B` rompiendo su encapsulamiento (`UNLaM GRASP-original-marin.pdf`, Pág. 39).
2. **Acoplamiento Común (o Global — Malo):** Dos o más módulos acceden y modifican una misma estructura de datos global compartida (variables globales). Si cambia la estructura global, se rompen todos los módulos (`Final Julio 2026`).
3. **Acoplamiento Externo:** Dos módulos comparten un formato de datos, protocolo de comunicación o interfaz de dispositivo impuesta externamente.
4. **Acoplamiento de Control (Moderado — Evitar si es posible):** Un módulo `A` le pasa a un módulo `B` un **"flag" o bandera de control** (ej. un booleano o código `tipoOperacion = 2`) que le dicta a `B` qué flujo lógico interno (`if/switch`) debe ejecutar (`Final Julio 2026`).
5. **Acoplamiento de Estampado / Estructura (*Stamp Coupling*):** Dos módulos se pasan como parámetro un objeto o registro complejo entero cuando el módulo receptor solo necesitaba usar uno o dos campos de ese registro.
6. **Acoplamiento de Datos / Normal (El MEJOR y Deseado):** Dos módulos se comunican exclusivamente pasándose por parámetro los datos elementales (o a través de una interfaz limpia) estrictamente necesarios para realizar la tarea (`Final Julio 2026`).

B. Los 7 Grados de Cohesión (De PEOR / Más Baja a MEJOR / Más Alta)

> 🎓 **CRITERIO Y RECOMENDACIÓN DE LA PROFESORA**
> **Definición:** La cohesión mide **cuán enfocadas, relacionadas y afines entre sí están las responsabilidades de una única clase o módulo**. **Buscamos SIEMPRE ALTA COHESIÓN.** (`UNLaM GRASP-original-marin.pdf`, Págs. 46–47).

1. **Cohesión Coincidente (La PEOR):** Una clase agrupa métodos que no tienen absolutamente ninguna relación entre sí (ej. una clase `Utilidades` que calcula impuestos, imprime facturas y valida contraseñas).
2. **Cohesión Lógica:** Un módulo agrupa varias tareas similares en categoría, pero en tiempo de ejecución se elige solo una mediante un `switch/flag` (ej. un método que lee de disco, de red o de teclado según un parámetro).
3. **Cohesión Temporal:** Agrupa tareas que solo tienen en común que se ejecutan en el mismo instante de tiempo (ej. un método `inicializarSistema()` que abre conexión a BD, limpia pantalla, carga logs y saluda al usuario).
4. **Cohesión Procedimental:** Los elementos se agrupan porque siempre se ejecutan siguiendo un orden de secuencia fijo, aunque operen sobre datos distintos.
5. **Cohesión Comunicacional:** Todos los métodos del módulo operan sobre el mismo conjunto de datos de entrada o producen los mismos datos de salida.
6. **Cohesión Secuencial:** La salida de una operación dentro del módulo sirve directamente como entrada para la siguiente operación (como una línea de montaje).

7. **Cohesión Funcional (La MEJOR — Alta Cohesión en OO):** Cada clase o método realiza **una única responsabilidad bien definida** y todos sus elementos contribuyen íntegramente a ese único propósito de negocio.

1.4. Principios de Diseño Modular y Patrones Arquitectónicos MVC / MVVM (`UNLaM GRASP-original-marin.pdf` Págs. 50 y 63–71)

En la Clase 04 (`UNLaM GRASP-original-marin.pdf`), la profesora complementa Cohesión y Acoplamiento con los **Criterios de Diseño Modular (Bertrand Meyer / Pressman)** y la separación en capas mediante **MVC y MVVM**:

A. Los 5 Criterios de un Diseño Modular (`UNLaM GRASP-original-marin.pdf` Pág. 50)

1. **Descomponibilidad Modular:** El método de diseño permite dividir un problema complejo en subproblemas más pequeños e independientes que pueden diseñarse por separado.
2. **Componibilidad Modular:** Permite ensamblar componentes de software existentes (o nuevos) para producir un nuevo sistema (reutilización de módulos).
3. **Comprensibilidad Modular:** Cada módulo puede ser comprendido por un lector humano como una unidad autónoma sin necesidad de leer todos los demás módulos del sistema.
4. **Continuidad Modular:** Un cambio pequeño en la especificación de los requerimientos provoca un cambio que afecta a un **único módulo** (o a un número mínimo de módulos), en lugar de propagarse en cascada por todo el sistema (logrado mediante **Bajo Acoplamiento** y **Variaciones Protegidas**).
5. **Protección Modular:** Si ocurre una condición anormal o error en tiempo de ejecución dentro de un módulo, sus efectos quedan confinados dentro de ese módulo sin corromper al resto.

B. Separación Interfaz-Dominio: MVC vs. MVVM (`UNLaM GRASP-original-marin.pdf` Págs. 63–71)

Arquitectura	Componentes y Responsabilidades	Relación con el Parcial de Diseño
MVC (*Model - View - Controller*)	- **Modelo (*Model*):** Clases del dominio que encapsulan los datos y las reglas de negocio. - **Vista (*View*):** Pantallas/UI que presentan la información al actor. - **Controlador (*Controller*):** Captura los eventos de usuario desde la Vista y coordina/delega la respuesta hacia el Modelo.	En nuestros Diagramas de Clases y Secuencia modelamos el **Controller** (`C_...`) y el **Modelo (Clases de Dominio)**, dejando afuera la **Vista** para garantizar el **Principio de Separación Modelo-Vista**: si mañana se cambia una pantalla web por una app móvil o un tótem, el Modelo y los Controladores no cambian.
MVVM (*Model - View - ViewModel*)	- **Modelo (*Model*):** Entidades y lógica de dominio. - **Vista (*View*):** Estructura visual declarativa. - **ViewModel:** Abstracción de la Vista que expone propiedades y comandos mediante **Data Binding (enlace bidireccional de datos)** automático sin que el ViewModel tenga referencias directas a la Vista.	Explicado en `UNLaM GRASP-original-marin.pdf` (Págs. 68–71) como evolución moderna en frameworks frontend/móviles para desacoplar aún más el estado de la interfaz respecto de los componentes visuales.

PARTE 2: LOS 9 PATRONES GRASP Y CÓMO JUSTIFICARLOS EN EL PARCIAL (PUNTO 1)

A continuación se detallan los **9 Patrones GRASP** (`UNLaM GRASP-original-marin.pdf` Págs. 17–77) junto con la **plantilla lista para copiar y adaptar en el Punto 1 del parcial**.

> ⚠️ **ALERTA / REGLA CRÍTICA DE CORRECCIÓN (ING. ANALÍA MARÍN)**
> **ATENCIÓN PARA EL PARCIAL:** En varios temas de 2026 la consigna pide: *"Justificar 2 patrones GRASP aplicados (**sin contar Controller**)" o "Justificar 3 patrones GRASP"*. Por eso tenés que saber justificar a la perfección **Experto en Información**, **Creador**, **Alta Cohesión / Bajo Acoplamiento**, **Polimorfismo** y **Fabricación Pura**.

#	Patrón GRASP	Problema que Resuelve	Solución / Regla de Asignación	Cómo Justificarlo en el Parcial (Plantilla Concreta)
1	**Experto en Información** (*Information Expert*)	¿Cuál es el principio general para asignar una responsabilidad (método) a un objeto?	Asignar la responsabilidad a la clase que **posee la información necesaria (atributos o referencias)** para cumplirla ("*Lo hace el que conoce*"). A menudo el cumplimiento se logra colaborando entre varios expertos parciales.	*"Se aplica **Experto en Información** en la clase `[NombreClaseContenedora, ej. HistoriaClinica / Factura / PropuestaViaje]` en el método `+[nombreMetodo, ej. calcularTotal() / verificarVacunasVigentes()]`, ya que es la clase que conoce y posee la colección de `[ObjetosParte, ej. ItemFactura / VacunaAplicada]`, delegando a su vez en cada `[ObjetoParte]` la consulta de su propio atributo privado `-[atributo, ej. subtotal / fechaVencimiento]`."*
2	**Creador** (*Creator*)	¿Quién debe ser responsable de crear (`new` / `create`) una nueva instancia de la clase `A`?	Asignar a la clase `B` la responsabilidad de crear instancias de `A` si se cumple que: 1) `B` **agrega o contiene** objetos `A` (Composición/Agregación). 2) `B` **registra** instancias de `A`. 3) `B` utiliza estrechamente objetos `A`. 4) `B` **tiene los datos de inicialización** que se pasan al constructor de `A`.	*"Se aplica **Creador** en la clase `[ClaseContenedora, ej. HistoriaClinica / SolicitudViaje / Prestamo]` mediante su método `+[metodoCreador, ej. registrarConsulta() / generarCuotas()]`, el cual instancia (`create`) objetos de la clase `[ClaseCreada, ej. Consulta / Cuota]`. Esto se justifica porque `[ClaseContenedora]` contiene de forma exclusiva (composición) y registra en su colección interna a las instancias de `[ClaseCreada]`."*
3	**Controlador** (*Controller*)	¿Qué primer objeto más allá de la capa de Interfaz de Usuario (UI) debe recibir y coordinar un evento de entrada del sistema?	Asignar la responsabilidad a una clase artificial (`C_Modulo`) que represente un subsistema o caso de uso agrupado. El controlador recibe el evento, busca referencias y **delega** el trabajo en el dominio (sin hacer lógica de negocio pesada para no saturarse).	*"Se aplica **Controlador** en las clases `[C_Modulo1, C_Modulo2, C_Modulo3]`, las cuales capturan los eventos provenientes de la interfaz de usuario, desacoplando la capa de presentación (Vista) del modelo de dominio y actuando como coordinadoras que delegan las reglas de negocio en los expertos correspondientes."*
4	**Bajo Acoplamiento** (*Low Coupling*)	¿Cómo dar soporte a una baja dependencia entre clases, reducir el impacto de los cambios y aumentar la reutilización?	Asignar responsabilidades de modo que el acoplamiento (cantidad de clases que una clase conoce directamente) se mantenga bajo. Evitar que el `Controller` cree objetos internos que puede crear su propio contenedor (ej. que `Venta` cree a `Pago` en vez de que lo cree `Caja`).	*"Se aplica **Bajo Acoplamiento** al hacer que `[C_Controller]` solo se comunique con `[ClaseRaiz, ej. HistoriaClinica]` y que sea `[HistoriaClinica]` quien cree y gestione internamente a `[Consulta]` y `[VacunaAplicada]`, evitando acoplar al Controlador con todas las clases internas de detalle del dominio."*
5	**Alta Cohesión** (*High Cohesion*)	¿Cómo mantener los objetos enfocados, comprensibles, manejables y fáciles de mantener?	Asignar responsabilidades de forma que cada clase tenga un conjunto pequeño de métodos estrechamente relacionados con su abstracción. Si una clase hace demasiadas cosas no afines, dividirla y delegar.	*"Se aplica **Alta Cohesión** al separar las responsabilidades en clases específicas: `[Consulta]` solo gestiona los datos clínicos del acto médico, `[Receta]` gestiona las indicaciones farmacológicas y `[Factura]` gestiona el cálculo y cobro, evitando crear una clase monolítica o sobrecargar a `[Veterinario]` con tareas que no le corresponden."*
6	**Polimorfismo** (*Polymorphism*)	¿Cómo manejar alternativas basadas en el tipo de un objeto sin usar condicionales `if / switch` preguntando por el tipo?	Cuando el comportamiento varía según el tipo o subclase, asignar la responsabilidad usando **operaciones polimórficas** (mismo nombre de método en una superclase o interfaz `<<interface>>`, implementado distinto en cada subclase).	*"Se aplica **Polimorfismo** en la jerarquía/interfaz `[Pago / IEstrategiaPrecio / IEstado]`, donde el método `+[procesarPago() / calcularMonto()]` es invocado de forma uniforme sin usar estructuras condicionales (`if/switch`), y cada clase concreta (`[PagoTarjeta, PagoTransferencia]`) ejecuta su propia implementación específica."*
7	**Fabricación Pura** (*Pure Fabrication*)	¿A quién asignamos una responsabilidad cuando dársela a una clase del dominio violaría la Alta Cohesión y el Bajo Acoplamiento?	Crear una **clase artificial de diseño** que NO representa un concepto tangible del dominio del problema, diseñada específicamente para agrupar responsabilidades cohesivas (ej. `NominaMascotas`, `Padron`, `AdaptadorARCA`, `Notificador`).	*"Se aplica **Fabricación Pura** al diseñar la clase `[NominaClientes / Padron / AdaptadorARCA]`, la cual no es un objeto físico del dominio del negocio, sino una clase artificial creada para encapsular la responsabilidad de `[búsqueda en la colección / comunicación con el servicio externo]`, preservando la Alta Cohesión de las clases del dominio."*

#	Patrón GRASP	Problema que Resuelve	Solución / Regla de Asignación	Cómo Justificarlo en el Parcial (Plantilla Concreta)
8	**Indirección** (*Indirection*)	¿Cómo evitar el acoplamiento directo entre dos elementos que no deben conocerse directamente?	Asignar la responsabilidad a un **objeto intermediario** para que medie entre los otros dos componentes (base de los patrones *Adapter*, *Facade*, *Mediator*, *Proxy* y *Controller*).	*"Se aplica **Indirección** mediante la clase intermediaria `[AdaptadorARCA / C_Atencion]`, que actúa como puente entre el modelo de dominio interno y el componente externo, impidiendo que las clases de negocio dependan directamente de la API externa."*
9	**Variaciones Protegidas** (*Protected Variations*)	¿Cómo diseñar objetos y subsistemas para que la variación o inestabilidad en estos elementos no afecte negativamente a otros?	Identificar los puntos de cambio o inestabilidad previstos (ej. nuevas formas de pago, nuevos proveedores externos, nuevas reglas de cálculo) y **envolverlos detrás de una interfaz estable (`<<interface>>`)**.	*"Se aplica **Variaciones Protegidas** al encapsular el punto de inestabilidad de `[las reglas de cálculo de comisión / los servicios externos de aerolíneas]` detrás de la interfaz `<<interface>> [ICalculadorComision / IServicioReserva]`, de modo que agregar nuevas variantes en el futuro no obligue a modificar el código de las clases clientes."*

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 4: PATRONES GOF — LOS 23 PATRONES, CLASIFICACIONES, DIAGRAMAS Y APLICACIONES

📑 ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO

PARTE 3: CATÁLOGO COMPLETO DE LOS 23 PATRONES GOF (CON EJEMPLOS DE CÁTEDRA)	**Pág. 29**	3.1. LOS 5 PATRONES CREACIONALES (GOF-2Q-2026.pdf Págs. 19–29)	**Pág. 29**
3.2. LOS 7 PATRONES ESTRUCTURALES (GOF-2Q-2026.pdf Págs. 30–44)	**Pág. 30**	3.3. LOS 11 PATRONES DE COMPORTAMIENTO (GOF-2Q-2026.pdf Págs. 45–66)	**Pág. 31**
PARTE 4: MANUAL TÁCTICO PARA RESOLVER EL PUNTO 3 DEL PARCIAL (ELECCIÓN DE 2 PATRONES GOF)	**Pág. 32**	Plantilla Exacta para Redactar el Punto 3 en la Hoja del Parcial (Incisos a, b, c, d)	**Pág. 32**

PARTE 3: CATÁLOGO COMPLETO DE LOS 23 PATRONES GOF (CON EJEMPLOS DE CÁTEDRA)

Los 23 patrones GoF se dividen en **3 Categorías** (`GOF-2Q-2026.pdf` Pág. 18):

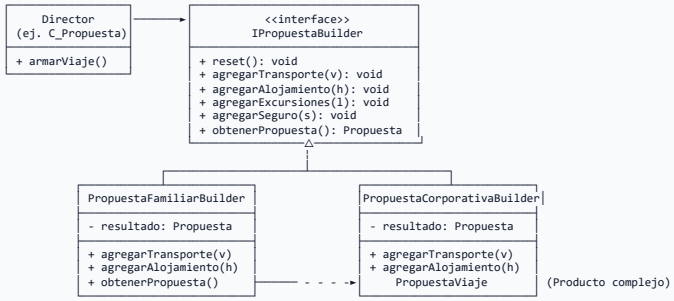
1. **Patrones Creacionales (5):** Abstraen el proceso de instanciación de objetos, independizando al sistema de cómo se crean, componen y representan sus productos.
2. **Patrones Estructurales (7):** Tratan sobre cómo se combinan y componen clases y objetos para formar estructuras más grandes y flexibles.
3. **Patrones de Comportamiento (11):** Tratan sobre algoritmos y la asignación de responsabilidades e interacción entre objetos en tiempo de ejecución.

3.1. LOS 5 PATRONES CREACIONALES (`GOF-2Q-2026.pdf` Págs. 19–29)

1. `Builder` (Constructor Paso a Paso) — ★ *ALTAMENTE RECOMENDADO PARA EL PARCIAL*

- **Problema / Intención:** Separar la construcción de un **objeto complejo** de su representación, de modo que el mismo proceso de construcción paso a paso pueda crear diferentes representaciones (o configuraciones con partes opcionales).
- **Cuándo usarlo en el Parcial:**
  - **Parcial Agencia de Viajes (2C 2026):** Armar una `PropuestaViaje` (o `PaqueteTuristico`) compleja paso a paso: `+construirVuelo()`, `+construirAlojamiento()`, `+construirExcursiones()`, `+construirSeguro()`. Un viaje corporativo y un viaje familiar usan los mismos pasos pero arman configuraciones distintas sin tener un constructor monstruoso de 15 parámetros.
  - **Parcial Veterinaria / Laboratorio / Refugio:** Construir un `InformeMedicoCompleto` o un `ContratoAdopcion` con cláusulas paso a paso.
- **Estructura UML a dibujar en tu Diagrama de Clases:**

DIAGRAMA / ESTRUCTURA TÉCNICA UML



2. `Factory Method` (Método de Fábrica / Constructor Virtual) — ★ *ALTAMENTE RECOMENDADO PARA EL PARCIAL*

- **Problema / Intención:** Definir una interfaz o clase abstracta para crear un objeto, pero **dejar que sean las subclases quienes decidan qué clase concreta instanciar**.
- **Cuándo usarlo en el Parcial:**
  - Cuando el sistema debe crear distintos tipos de objetos de una jerarquía (ej.: en **Reclamos Municipales — Final Julio 2026**: `CreadorReclamo` con subfábricas para `ReclamoLuminaria`, `ReclamoBacheo`, `ReclamoResiduos`; en **Seguridad Ciudadana — Clase 09**: crear distintos tipos de `Alerta` o `Denuncia`; en **Refugio 1C 2026**: crear distintos tipos de `Donacion` —monetaria, insumo médico, alimento— o distintos tipos de `FichaAnimal`).
- **Estructura UML:**
  - `<<abstract>> Creador` (con el método `+ <<abstract>> crearProducto(): IProducto`) ← heredan `CreadorConcretoA` y `CreadorConcretoB`.
  - `<<interface>> IProducto` (o `<<abstract>> Producto`) ← implementan `ProductoConcretoA` y `ProductoConcretoB`.

3. `Abstract Factory` (Fábrica Abstracta de Familias)

- **Problema / Intención:** Proporcionar una interfaz para crear **familias de objetos relacionados o dependientes entre sí** sin especificar sus clases concretas, garantizando que los productos de una familia no se mezclen con los de otra (`Final Dic 2025` y `Final Julio 2026`).
- **Diferencia clave con Factory Method:** *Factory Method* crea **UN** tipo de producto mediante herencia del método creador; *Abstract Factory* tiene **múltiples métodos de creación** en la misma fábrica (`+crearVuelo()`, `+crearHotel()`, `+crearSeguro()`) para producir una **familia entera coherente** (ej. `FabricaViajeLowCost` vs `FabricaViajePremium`, o `FabricaDocumentacionArgentina` vs `FabricaDocumentacionExterior`).

4. `Prototype` (Prototipo / Clonación) — ★ *USADO POR LA PROFESORA EN EL CASO LABORATORIO*

- **Problema / Intención:** Especificar los tipos de objetos a crear por medio de una **instancia prototípica ya existente**, y crear nuevos objetos **copiando/clonando (`+clone(): Prototipo` / `+clonar()`)** ese prototipo.
- **Ejemplo Oficial de la Prof. Analía Marín (`EJEMPLO-Practica-Teoria-version-10.pdf`, Págs. 10–12 — Caso Laboratorio):**
  - En el laboratorio existe un catálogo de **Especificaciones de Estudio** (`especificacionEstudio`: ej. la plantilla estándar del estudio "Hemograma" con sus determinaciones, unidades y valores de referencia preconfigurados).
  - Cuando un paciente ingresa una `Orden` médica y se crea cada `itemOrden`, en lugar de construir desde cero toda la estructura del estudio, el `itemOrden` le envía el mensaje `clone()` a `:especificacionEstudio`, obteniendo una nueva instancia clonada `:estudio` lista para cargarle los resultados particulares de ese paciente.
- **Otros usos en Parciales:** En **Agencia de Viajes (2C 2026)**, cuando el cliente toma un `PaqueteTuristico` base prearmado del catálogo (ej. "Bariloche Clásico 7 días") y el sistema lo **clona (`+clonar()`)** para que el cliente pueda cambiarle el hotel o agregarle/quitarle excursiones sin alterar el paquete plantilla del catálogo; o en **PetCare (1C 2026)** para repetir una receta crónica.

5. `Singleton` **(Instancia Única) —** ***(¡ATENCIÓN: Restricción particular en Parciales 2026!)***

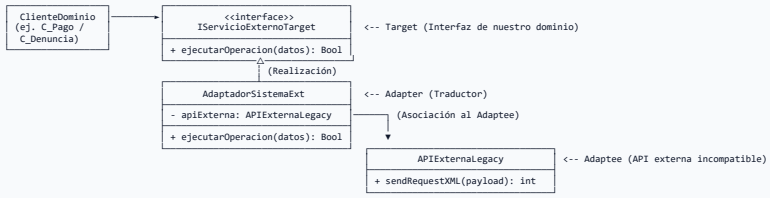
- **Problema / Intención:** Garantizar que una clase tenga **una única instancia** en todo el sistema y proporcionar un punto de acceso global a ella (`- instancia: Singleton` estática, constructor privado `- Singleton()` y método estático `+ getInstance(): Singleton`).
- **Diferencia entre Regla General GoF y Restricción de Parciales 2026:** En la teoría general de GoF es un patrón creacional válido, pero en los **Parciales 2026** (ej. Parcial 2 `1C2026 Com. 1900` y en la Clase 09 `[02:15:00]`) la cátedra **prohíbe explícitamente elegir** `Singleton` **(y** `Facade` **)** porque los alumnos solían aplicarlo mecánicamente sobre cualquier Controller sin resolver un problema real del dominio.

3.2. LOS 7 PATRONES ESTRUCTURALES (`GOF-2Q-2026.pdf` Págs. 30–44)

1. `Adapter` **(Adaptador / Wrapper) —** ***★ COMODÍN N° 1 PARA EL PARCIAL***

- **Problema / Intención:** Convertir la interfaz de una clase existente (generalmente un **sistema externo, API de terceros o librería legacy**) en otra interfaz que el cliente de nuestro dominio espera. Permite que clases con interfaces incompatibles trabajen juntas sin modificar su código interno.
- **Cuándo usarlo en el Parcial (¡Aparece en casi TODOS los parciales!):**
  - **Parcial 2 Reencuentro/Adopción y Animales Exóticos (1C 2026 Com. 1900):** El enunciado incluye explícitamente las clases `OrganismoExternos` y `organismoFaunaFloraBsAs` y pide notificar a la **Dirección de Flora y Fauna** y a la **Policía Ecológica**, que son organismos estatales externos con APIs distintas.
  - **Parcial 3 Agencia de Viajes (2C 2026 Com. 1900):** El Diagrama de Componentes impreso incluye explícitamente `Sist Ext. Aerolinea` , `Sist Ext. Hotel` y `Sist Externo Pago` , cuyas interfaces externas deben adaptarse a las interfaces del dominio (`IServicioTransporteExt` , `IServicioAlojamientoExt` , `IPasarelaPago`).
  - **Parcial 1 Veterinaria PetCare (1C 2026 Com. 4300):** Adaptar los proveedores externos de mensajería (**WhatsApp API** / **Servidor SMTP Mail**) detrás de una interfaz común `ICanalNotificacion` .
  - **Caso Banco Olivares (Clase 05 y Clase 08** `ejerFutbol-2026-10-08.pdf` **Págs. 29–32):** Conectarse con **VISA** y **Veraz**.
  - **Caso Estadios (Clase 06):** Conectarse con **Renaper** y **Policía**.
- **Estructura UML a dibujar en tu Diagrama de Clases:**

DIAGRAMA / ESTRUCTURA TÉCNICA UML



2. `Composite` **(Compuesto / Árbol Todo-Parte) —** ***★ MUY RECOMENDADO PARA EL PARCIAL***

- **Problema / Intención:** Componer objetos en **estructuras de árbol** para representar jerarquías "Todo-Parte", permitiendo que los clientes **traten de manera uniforme a los objetos individuales (Hojas /** ***Leaf*****) y a las composiciones de objetos (Compuestos /** ***Composite*****)**.
- **Cuándo usarlo en el Parcial:**
  - **Parcial 3 Agencia de Viajes (2C 2026):** Un `ServicioTuristico` (`<<abstract>>` o `<<interface>>` con `+obtenerPrecio(): float` y `+verificarDisponibilidad(): bool`) que puede ser tanto un servicio atómico (**Hoja**: `Transporte` , `Alojamiento` , `Excursion`) como un `PaqueteTuristico` **(Composite)** que compone múltiples servicios hijos y calcula su disponibilidad y precio base recorriendo polimórficamente sus hojas.
  - **Caso Laboratorio (Modelo de Parcial):** Una `PracticaMedica` que puede ser un estudio individual (**Hoja**: `Glucemia`) o un `PerfilCompuesto` (**Composite**: `Hemograma Completo` o `Hepatograma` compuesto por múltiples determinaciones hijas).
  - **Parcial 1 Veterinaria PetCare:** Un `Tratamiento` clínico que puede ser una indicación simple (**Hoja**: `Receta` , `VacunaAplicada` , `Estudio`) o un `ProtocoloTratamientoCompuesto` (**Composite**) que agrupa varias prácticas médicas y estudios.

3. `Decorator` **(Decorador / Envoltorio Dinámico) —** ***★ MUY RECOMENDADO PARA EL PARCIAL***

- **Problema / Intención:** Adjuntar **responsabilidades o funcionalidades adicionales a un objeto dinámicamente** en tiempo de ejecución, proporcionando una alternativa flexible a la herencia para extender la funcionalidad (`ejerFutbol-2026-10-08.pdf` Págs. 24–25: jugador con habilidades especiales, lesionado, cansado).
- **Cuándo usarlo en el Parcial:**
  - **Parcial 3 Agencia de Viajes (2C 2026):** Cuando el cliente selecciona un `PaqueteTuristico` base y decide **agregarle opcionalmente excursiones extras o mejoras de hotel** en tiempo de ejecución (`DecoradorExcursionExtra` , `DecoradorUpgradeHotel`), donde cada decorador envuelve al paquete y adiciona su costo al método `+calcularPrecio()` .

4. `Proxy` **(Apoderado / Subrogado de Control de Acceso)**

- **Problema / Intención:** Proporcionar un **sustituto o intermediario** de otro objeto para **controlar el acceso a él** (`Final Dic 2025`).
- **Tipos de Proxy vistos en clase (** `GOF-2Q-2026.pdf` **Págs. 43–44):**
  - **Proxy de Protección:** Verifica permisos/roles antes de delegar la llamada al objeto real (ej. validar si el profesional o usuario tiene permiso para modificar la `HistoriaClinica`).
  - **Proxy Virtual (Carga diferida /** ***Lazy Loading*****):** Retrasa la carga en memoria de objetos muy pesados (ej. imágenes de estudios/radiografías en `PetCare` o evidencia de fotos y videos en la `Denuncia` de `AnimalExotico` del Parcial 2) hasta que realmente se solicitan.
  - **Proxy Remoto / Caché:** Representa localmente a un objeto remoto o guarda en caché respuestas de servicios externos costosos.

5. `Bridge` **(Puente)**

- **Problema / Intención: Desacoplar una abstracción de su implementación**, de manera que ambas puedan variar y extenderse independientemente mediante dos jerarquías separadas unidas por una asociación/agregación (`Final Dic 2025` y `Final Julio 2026`).
- **Ejemplo aplicable en Parciales:** En **PetCare (1C 2026)**, separar la abstracción del mensaje (`Notificacion` → `RecordatorioVacuna` , `ConfirmacionTurno` , `AlertaEstudio`) de la implementación del canal de envío (`ICanalEnvio` → `CanalWhatsApp` , `CanalEmail`).

6. `Flyweight` **(Peso Mosca)**

- **Problema / Intención:** Usar el uso compartido para soportar eficientemente **grandes cantidades de objetos de grano fino** que comparten datos idénticos, separando el **estado intrínseco** (compartido e inmutable, ej. datos de camiseta/club en los miles de hinchas del estadio en `ejerFutbol-2026-10-08.pdf` Págs. 26–28) del **estado extrínseco** (particular de cada instancia).

7. `Facade` **(Fachada) —** ***★ EJEMPLO OFICIAL DE CÁTEDRA: CASO BANCO OLIVARES (*** *`ejerFutbol-2026-10-08.pdf`* ***Págs. 29–32)***

- **Problema / Intención:** Proporcionar una **interfaz unificada y simplificada** de alto nivel para un conjunto de interfaces de un subsistema complejo, haciendo que el subsistema sea más fácil de usar y reduciendo el acoplamiento de los clientes con las clases internas del subsistema.
- **Ejemplo Oficial de la Profesora (Caso Banco Olivares — Clase 08,** `ejerFutbol-2026-10-08.pdf` **Págs. 29–32):**

- Para otorgar un préstamo o tarjeta, el banco debe consultar múltiples subsistemas internos y externos complejos (`SubsistemaVeraz`, `SubsistemaVISA`, `SubsistemaScoring`, `SubsistemaBCRA`).
- En lugar de que el controlador `C_EvaluacionP` conozca y llame uno por uno a todos esos subsistemas y sus adaptadores, se introduce una clase `FachadaEvaluacionCrediticia` (`Facade`) con un único método simple `+evaluarRiesgoCliente(dni, monto): ResultadoEvaluacion`, la cual internamente coordina los llamados a los `Adapter`s de `VISA` y `Veraz` y al motor de scoring.

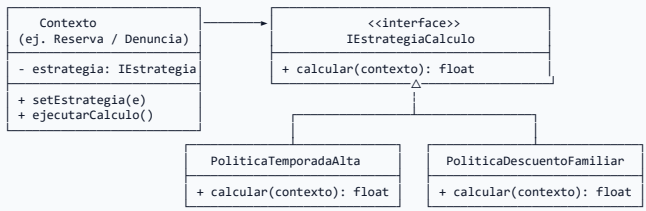
- **Advertencia sobre Restricción Particular en Parciales 2026:** Aunque `Facade` se enseña en la Clase 08 con el Caso Banco Olivares y se evalúa en preguntas teóricas, **en las consignas de los Parciales 2026 (como el Parcial 2 `1C2026 Com. 1900`) se aclara explícitamente** `"no usar singleton, ni facade"` para evitar que el alumno ponga una fachada genérica sin modelar la estructura interna. Por lo tanto, **para sistemas externos en el parcial práctico elegí siempre** `Adapter`.

3.3. LOS 11 PATRONES DE COMPORTAMIENTO (`GOF-2Q-2026.pdf` Págs. 45–66)

1. `Strategy` (Estrategia) — ★ *COMODÍN N° 1 DE COMPORTAMIENTO PARA EL PARCIAL*

- **Problema / Intención:** Definir una **familia de algoritmos**, encapsular cada uno de ellos en una clase separada y hacerlos **intercambiables** en tiempo de ejecución a través de una interfaz común (`Final Dic 2025` y `ejerFutbol-2026-10-08.pdf` Pág. 14).
- **Cuándo usarlo en el Parcial (¡Encaja en TODOS los parciales!):**
  - **Parcial 3 Agencia de Viajes (2C 2026 — Pedido explícito en Punto 4.A):** La consigna pide textualmente: *"A- la complejidad del cálculo del precio de contratación según diferentes políticas"*. Se resuelve con `Strategy`: `Reserva` (o `PaqueteTuristico`) delega en `<<interface>> IPoliticaPrecio` (`+calcularPrecioTotal(reserva): float`), implementada por `PoliticaTemporadaAlta`, `PoliticaGrupoPasajeros`, `PoliticaConExcursionesExtras`.
  - **Parcial 2 Reencuentro/Adopción y Animales Exóticos (1C 2026):** Estrategia para **calcular el nivel de peligrosidad (`ROJO`, `NARANJA`, `AMARILLO`, `VERDE`)** de un `AnimalExotico` según distintos criterios (`IEstrategiaCalculoPeligrosidad` ← `CalculoPorEspecie Salvaje`, `CalculoPorZonaUrbana`, `CalculoPorTamañoEstado`), o estrategia de **evaluación de reclamos de dueños**.
  - **Parcial 1 Veterinaria PetCare (1C 2026):** Estrategia de cálculo de esquema de recordatorios o dosificación según tipo de mascota/edad/peso.
  - **Préstamos Bancarios (2C 2025):** Cálculo de cuotas por `SistemaFrances` vs `SistemaAleman`.
- **Estructura UML a dibujar en tu Diagrama de Clases:**

DIAGRAMA / ESTRUCTURA TÉCNICA UML



2. `State` (Estado) — ★ *COMODÍN N° 2 DE COMPORTAMIENTO PARA EL PARCIAL*

- **Problema / Intención:** Permitir que un objeto **altere su comportamiento cuando su estado interno cambia**; el objeto parecerá cambiar de clase. Reemplaza condicionales gigantes (`if (estado == PENDIENTE) ... else if (estado == CONFIRMADO)`) delegando el comportamiento en un objeto Estado polimórfico (`Final Julio 2026` y Clase 08/09).
- **Cuándo usarlo en el Parcial (¡Especialmente cuando el examen te da un Diagrama de Estados impreso!):**
  - **Parcial 1 Veterinaria PetCare (1C 2026 Com. 4300):** El examen trae impreso el Diagrama de Estados de `Consulta Veterinaria`: `EstadoConsulta` (`<<interface>>` con `+registrarConsulta()`, `+registrarDiagnostico()`, `+cerrarConsulta()`) implementado por `EstadoRegistrada`, `EstadoDiagnosticada` y `EstadoCerrada`.
  - **Parcial 2 Reencuentro/Adopción (1C 2026 Com. 1900):** El enunciado detalla que el `Animal` cambia de comportamiento según su estado legal y de reclamo: `EstadoAnimal` (`<<interface>>` con `+iniciarReclamo()`, `+registrarReclamoDuplicado()`, `+resolverDisputa()`, `+confirmarEntrega()`) implementado por `EstadoPerdido`, `EstadoEnProcesoDeReclamo`, `EstadoEnDisputaLegal` (que bloquea la agenda y suspende el encuentro) y `EstadoReencontrado`.
  - **Parcial 3 Agencia de Viajes (2C 2026 Com. 1900):** El examen trae impreso el Diagrama de Estados de `Reserva` y pide integrarlo al modelo: `IEstadoReserva` (`+pagoRealizado()`, `+cancelar()`, `+cerrarViaje()`, `+cerrarReservaCancelada()`) implementado por `EstadoPendienteDePago`, `EstadoConfirmada`, `EstadoCancelada` y `EstadoFinalizada`.

3. `Observer` (Observador / Publicador-Suscriptor) — ★ *COMODÍN N° 3 DE COMPORTAMIENTO PARA EL PARCIAL*

- **Problema / Intención:** Definir una **dependencia de uno a muchos** entre objetos, de forma que cuando un objeto (**Sujeto / Observado**) cambia su estado, **todos sus dependientes (Observadores) sean notificados y actualizados automáticamente** (`ejerFutbol-2026-10-08.pdf` Págs. 5–12: la `Pelota` es el Sujeto observado y los `Jugador`es son los Observadores).
- **Cuándo usarlo en el Parcial:**
  - **Parcial 2 Reencuentro/Adopción y Animales Exóticos (1C 2026 Com. 1900 — Sugerido explícitamente en el Paso 4 del enunciado):** El paso 4 del escenario dice textualmente: *"4. Notificar automáticamente a: Dirección de Flora y Fauna y Policía Ecológica // recomendable usar un Patron GOF"*. `DenunciaExotico` actúa como **Sujeto Observado** que mantiene una lista de `<<interface>> IObservadorAlertaExotico` (`+actualizarAlerta(denuncia)`) y al registrarse o calcularse la peligrosidad ejecuta `+notificarOrganismos()`, disparando la notificación a `ObservadorFloraYFauna` (`organismoFaunaFloraBsAs`) y `ObservadorPoliciaEcologica`.
  - **Parcial 1 Veterinaria PetCare (1C 2026 Com. 4300):** Notificar automáticamente al `Usuario` (por **WhatsApp** y **Mail**) cuando se genera un `Recordatorio` o se confirma un `Turno` (analizado en Clase 09, `[02:25:00]`).
  - **Parcial 3 Agencia de Viajes (2C 2026 Com. 1900):** Notificar automáticamente por email al `Cliente` y a los proveedores cuando la `Reserva` pasa a `Confirmada` o `Cancelada`.

4. `Template Method` (Método Plantilla)

- **Problema / Intención:** Definir el **esqueleto de un algoritmo** en una operación de una superclase abstracta, delegando a las subclases la redefinición de ciertos pasos específicos del algoritmo sin cambiar su estructura general.
- **Ejemplo:** Proceso de `EvaluacionCrediticia` o `EvaluacionAdopcion` que siempre ejecuta `1. validarIdentidad()`, `2. verificarAntecedentes()`, `3. calcularPuntajeEspecifico()` (abstracto), `4. emitirDictamen()`.

5. `Chain of Responsibility` (Cadena de Responsabilidad)

- **Problema / Intención:** Evitar acoplar el emisor de una petición a su receptor dando a **más de un objeto la posibilidad de responder a la petición**, encadenando los objetos receptores y pasando la petición a través de la cadena (`siguienteHandler`) hasta que uno la procese o valide.
- **Ejemplo:** Cadena de validaciones de un préstamo o de una adopción (1° Validador de DNI → 2° Validador de Ingresos/Vivienda → 3° Validador de Antecedentes), o escalamiento de atención de un reclamo (Nivel 1 Bot → Nivel 2 Operador → Nivel 3 Supervisor).

6. `Command` **(Comando / Orden)**

- **Problema / Intención:** Encapsular una **petición o acción como un objeto**, permitiendo parametrizar clientes con distintas peticiones, encolar tareas, registrar bitácoras (logs) y soportar operaciones de **deshacer (** `undo` **)**. (`ejerFutbol-2026-10-08.pdf` Págs. 19–21: acciones patear, saltar, pasar).

7. `Memento` **(Recuerdo / Instantánea de Estado)**

- **Problema / Intención:** Capturar y externalizar el **estado interno de un objeto (** `Originator` **)** dentro de un objeto `Memento` (custodiado por un `Caretaker`) sin violar el encapsulamiento, para poder **restaurar el objeto a ese estado más adelante** (ej. historial de versiones de una `PropuestaViaje` o estado del partido en `ejerFutbol-2026-10-08.pdf` Págs. 22–23).

8. `Mediator` **(Mediador)**

- **Problema / Intención:** Definir un objeto que **encapsula cómo interactúa un conjunto de objetos (** `Colleagues` **)**, evitando que se referencien explícitamente todos contra todos (convierte una red "muchos a muchos" en una estrella centralizada; `ejerFutbol-2026-10-08.pdf` Págs. 17–18).

9. `Iterator` **(Iterador)**

- **Problema / Intención:** Proporcionar un modo de **acceder secuencialmente a los elementos de una colección agregada** (`hasNext()` , `next()`) sin exponer su representación interna (lista, árbol, grafo).

10. `Visitor` **(Visitante)**

- **Problema / Intención:** Representar una **operación que se realiza sobre los elementos de una estructura de objetos**, permitiendo definir una nueva operación (ej. exportar a PDF, calcular impuestos, auditar) sin cambiar las clases de los elementos sobre los que opera.

11. `Interpreter` **(Intérprete)**

- **Problema / Intención:** Dada una lengua o gramática simple (ej. expresiones regulares, fórmulas de reglas), definir una representación de su gramática y un intérprete que la evalúe.

PARTE 4: MANUAL TÁCTICO PARA RESOLVER EL PUNTO 3 DEL PARCIAL (ELECCIÓN DE 2 PATRONES GOF)

En el Punto 3 del parcial te piden elegir **2 patrones GoF de DIFERENTE CATEGORÍA** (sin usar `Singleton` ni `Facade`). Para no dudar ni un segundo en el examen, usá esta **Matriz de Combinaciones Ganadoras**:

Si el Enunciado tiene...	Patrón 1 Recomendado	Categoría 1	Patrón 2 Recomendado	Categoría 2
Conexión con sistema externo (ARCA/AFIP, Veraz, VISA, Renaper, Aerolíneas/Hoteles) **+ Distintas formas de calcular precios/asignar/evaluar**	`Adapter` (Desacopla y adapta la interfaz del sistema externo a una interfaz propia del dominio)	**Estructural**	`Strategy` (Intercambia dinámicamente el algoritmo de cálculo, descuento, evaluación o asignación)	**Comportamiento**
Armado de un paquete/combo o servicios anidables (Paquete turístico con vuelos+hotel+excursiones, o Estudios médicos simples y compuestos) **+ Ciclo de vida con estados cambiantes** (Reserva, Turno, Solicitud, Adopción)	`Composite` *(o `Decorator`)* (Trata igual a un servicio simple que a un paquete compuesto, o agrega adicionales dinámicos)	**Estructural**	`State` (Encapsula las reglas y transiciones de cada estado del objeto en clases polimórficas)	**Comportamiento**
Construcción paso a paso de un objeto complejo con partes opcionales (Propuesta de viaje, Contrato de adopción, Informe clínico) **+ Envío de alertas/notificaciones ante eventos o vencimientos** (Vencimiento de vacunas, falta de stock, cambio de estado)	`Builder` *(o `Factory Method`)* (Construye paso a paso distintas variantes del objeto complejo)	**Creacional**	`Observer` (Notifica automáticamente a múltiples canales/interesados cuando ocurre el evento en el Sujeto)	**Comportamiento**

Plantilla Exacta para Redactar el Punto 3 en la Hoja del Parcial (Incisos `a` , `b` , `c` , `d`)

Copiá esta estructura exacta para cada uno de tus 2 patrones GoF elegidos:

> 💡 **EJEMPLO APLICADO DE CÁTEDRA / PLANTILLA DE EXAMEN**
>
> #### **PATRÓN GOF N° 1**
>
> - **a) Categoría y Nombre del Patrón:** Patrón **Estructural** — **Adapter (Adaptador)**.
>
> - **b) Problema de diseño que resuelve dentro del caso:** El sistema debe comunicarse con un servicio externo (`[ej. ARCA / Sistemas de Aerolíneas y Hoteles]`) cuya API e interfaz de datos son propietarias, externas e incompatibles con el modelo de dominio de nuestra aplicación, además de estar sujetas a cambios externos que no controlamos.
>
> - **c) Justificación de por qué es adecuado:** Es adecuado porque permite convertir la interfaz incompatible de la clase externa en una interfaz propia esperada por nuestro dominio (`<<interface>> [IServicioExterno]`). De este modo se logra **Bajo Acoplamiento**, **Indirección** y **Variaciones Protegidas**: si el proveedor externo cambia su API o si mañana se agregan nuevos proveedores externos, solo se modifica o agrega un nuevo `AdaptadorConcreto` sin tocar una sola línea de código de nuestras clases de negocio.
>
> - **d) Clases reutilizadas del Punto 1 y Clases/Interfaces nuevas incorporadas:**
>
> - *Clase del Punto 1 que actúa como Cliente (* `Client` *):* `[ej. Factura / C_PropuestaViaje]` .
>
> - *Interfaz nueva incorporada (* `Target` *):* `<<interface>> [IAutorizadorFacturacion / IProveedorTransporte]` con el método `+[autorizarFactura(f) / consultarDisponibilidad(fecha, destino)]` .
>
> - *Clase nueva incorporada (* `Adapter` *):* `[AdaptadorARCA / AdaptadorAerolineaExt]` que implementa la interfaz `Target` y mantiene una referencia por asociación hacia el `Adaptee` .
>
> - *Clase externa adaptada (* `Adaptee` *):* `[API_ARCA_SOAP / SistemaExternoAerolinea]` con su método nativo incompatible.
>
> ---
>
> #### **PATRÓN GOF N° 2**
>
> - **a) Categoría y Nombre del Patrón:** Patrón de **Comportamiento** — **Strategy (Estrategia)** *(o **State** / **Observer**)*.
>
> - **b) Problema de diseño que resuelve dentro del caso:** El sistema necesita `[ej. calcular el costo final de la atención / asignar automáticamente un agente de viajes / evaluar la solicitud]` utilizando distintos algoritmos o reglas de negocio según `[el tipo de cobertura / el tipo de cliente familiar o corporativo]` , evitando llenar la clase principal de condicionales `if / switch` rígidos.
>
> - **c) Justificación de por qué es adecuado:** Es adecuado porque encapsula cada algoritmo o regla de negocio en una clase separada que implementa una interfaz común (`<<interface>> [IEstrategia...]`), permitiendo intercambiar el algoritmo dinámicamente en tiempo de ejecución y facilitando agregar nuevas reglas en el futuro cumpliendo el Principio Abierto/Cerrado (OCP), **Polimorfismo** y **Alta Cohesión**.
>
> - **d) Clases reutilizadas del Punto 1 y Clases/Interfaces nuevas incorporadas:**
>
> - *Clase del Punto 1 que actúa como Contexto (* `Context` *):* `[ej. Factura / SolicitudViaje]` , a la cual se le agrega la asociación `-estrategia: IEstrategia...` .
>
> - *Interfaz nueva incorporada (* `Strategy` *):* `<<interface>> [IEstrategiaCalculo / IEstrategiaAsignacion]` con el método abstracto `+[calcularMonto(...) / seleccionarAgente(...)]` .
>
> - *Clases nuevas incorporadas (* `ConcreteStrategies` *):* `[EstrategiaParticular]` , `[EstrategiaObraSocial]` y `[EstrategiaPlanVeterinario]` , que implementan dicha interfaz.

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 5: METODOLOGÍA PARA RESOLVER EJERCICIOS Y PARCIALES DESDE CERO

📑 ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO

PARTE 1: EL MÉTODO CRONOMETRADO DE 120 MINUTOS PARA RESOLVER EL PARCIAL DESDE CERO Pág. 33

- Paso 1 (Minutos 0 a 15): Lectura con los 3 Colores y Recorte de Alcance Pág. 33
- Paso 2 (Minutos 15 a 45): Esqueleto del Diagrama de Clases en Hoja 1 (Sin cerrar las cajas por abajo) Pág. 33
- Paso 3 (Minutos 45 a 75): Escenario y Diagrama de Secuencia en Hoja 2 (En simultáneo con los métodos del DC) Pág. 33
- Paso 4 (Minutos 75 a 95): Inyección de los 2 Patrones GoF en Hoja 1 y Redacción en Hoja Adjunta Pág. 33
- Paso 5 (Minutos 95 a 110): Redacción de las Justificaciones GRASP Pág. 33
- Paso 6 (Minutos 110 a 120): Auditoría Final con los 12 Mandamientos Pág. 33

PARTE 1: EL MÉTODO CRONOMETRADO DE 120 MINUTOS PARA RESOLVER EL PARCIAL DESDE CERO

Un error fatal en el parcial de Diseño de Sistemas es dibujar primero todo el Diagrama de Clases en tinta, pasar luego al Diagrama de Secuencia, y descubrir a mitad de la secuencia que te faltan métodos o relaciones en las clases. Para que eso **no te pase**, aplicá este protocolo de 6 pasos:

Paso 1 (Minutos 0 a 15): Lectura con los 3 Colores y Recorte de Alcance

1. Leé el enunciado completo y mirá la interfaz gráfica y los diagramas base (Componentes y/o Estados) que vienen impresos en la hoja.
2. Marcá según el código de colores de la profesora (`EJEMPLO-Practica-Teoria-version-10.pdf`, Págs. 5–6 y 8):
   - **Sustantivos con identidad propia (Amarillo)** → Clases del Dominio.
   - **Funcionalidad / Comportamiento y verbos (Verde)** → Métodos.
   - **Datos simples o clases secundarias (Celeste)** → Atributos.
3. Identificá los **grandes módulos funcionales** del enunciado (alineados con el Diagrama de Componentes) para definir tus **Controladores (`C_...`)**, respetando estrictamente el tope del examen (**máximo 4 controladores** en los parciales 2026).
4. Leé el punto del **Diagrama de Secuencia** para saber exactamente **qué escenario te piden secuenciar** (ahí es donde vas a poner mayor nivel de detalle en firmas de métodos y parámetros).
5. Leé el punto de **Patrones GoF** y elegí tus **2 patrones GoF de distinta categoría** (recordando la restricción de **no usar Singleton ni Facade**) antes de pasar en limpio el Diagrama de Clases, así ya los dibujás integrados en la Hoja 1.

Paso 2 (Minutos 15 a 45): Esqueleto del Diagrama de Clases en Hoja 1 (Sin cerrar las cajas por abajo)

1. En la **Hoja 1**, dibujá arriba los **4** `Controllers` (`C_...`).
2. Debajo, ubicá los contenedores/nóminas (`Nomina...` solo para búsquedas globales con asociación simple `——`, e `HistoriaClinica` para las colecciones propias) y las **Clases del Dominio** con sus atributos privados (`-`) y asociaciones sólidas con **multiplicidad** (`1`, `0..*`, `1..*`), **agregaciones (`◇`)**, **composiciones (`◆`)** y **herencias (`–▷`)**. Si el parcial trae un **Diagrama de Estados impreso**, integrá todos sus eventos como métodos de esa clase y sus estados.
3. **TRUCO DE ORO PARA EL EXAMEN EN PAPEL:** Al dibujar cada clase en la Hoja 1, **dejá abierto el borde inferior del compartimento de métodos** (no cierres la línea horizontal de abajo de la caja todavía). ¿Por qué? Porque cuando hagas el Diagrama de Secuencia en el Paso 3, cada mensaje nuevo que descubras lo vas a ir anotando inmediatamente adentro de la caja de esa clase en la Hoja 1. ¡Al terminar el examen cerrás las cajas y tenés **100% de consistencia garantizada**!

Paso 3 (Minutos 45 a 75): Escenario y Diagrama de Secuencia en Hoja 2 (En simultáneo con los métodos del DC)

1. En la parte superior de la **Hoja 2**, escribí el **Escenario concreto** con los valores puntuales del enunciado.
2. Trazá las líneas de vida de izquierda a derecha empezando por el `:C_Controller` (¡sin Actor!).
3. Por **cada flecha de mensaje** `—— metodo(param) ——►` que dibujes llegando a la línea de vida de un objeto `:ClaseX`:
   - Inmediatamente mirá tu **Hoja 1** y verificá que `+metodo(param)` esté escrito dentro de `ClaseX` y que exista la relación de asociación o dependencia entre el emisor y `ClaseX` (recordando que la visibilidad local no se dibuja en el Diagrama de Clases).
4. Usá `create(...)` para todo objeto nuevo que nazca en ese flujo (`:Consulta`, `:Diagnostico`, `:VacunaAplicada`, `:Receta`, `:Recordatorio`, `:DenunciaExotico`, `:Pago`).

Paso 4 (Minutos 75 a 95): Inyección de los 2 Patrones GoF en Hoja 1 y Redacción en Hoja Adjunta

1. Dibujá en la **Hoja 1** las interfaces (`<<interface>>`) y clases concretas de los **2 Patrones GoF** conectadas a las clases de tu dominio (podés resaltarlas con un recuadro punteado o color indicando `Patrón 1`, `Patrón 2`).
2. En la hoja correspondiente, redactá los incisos `a)`, `b)`, `c)` y `d)` de cada patrón GoF usando la plantilla de `03_PATRONES_GRASP_Y_GOF_APLICADOS.md` (y si el parcial pide modificar o mejorar el **Diagrama de Componentes**, dibujalo en su propia hoja numerada).

Paso 5 (Minutos 95 a 110): Redacción de las Justificaciones GRASP

1. Redactá las justificaciones de **Experto en Información**, **Creador**, **Bajo Acoplamiento / Alta Cohesión** (y **Controlador**), citando **nombres exactos de clases y métodos de tu Diagrama de Clases**.

Paso 6 (Minutos 110 a 120): Auditoría Final con los 12 Mandamientos

- Cerrá la parte inferior de las cajas de clases, verificá que todas las hojas estén numeradas y hacé el chequeo cruzado de métodos entre Secuencia, Estados y Clases.

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 6: CASOS PRÁCTICOS OFICIALES DE LA MATERIA (VENTAS, PRÉSTAMOS, LABORATORIO, GIMNASIO, ESTADIOS, SEGURIDAD Y RESERVAS)

📑 ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO

6.1. CASO PRÁCTICO 1: SISTEMA DE VENTAS Y LÍNEAS DE VENTA (CASO BASE DE PATRONES GRASP) Pág. 34
1. Problema y Asignación de Responsabilidades Paso a Paso Pág. 34
6.2. CASO PRÁCTICO 2: OTORGAMIENTO DE PRÉSTAMOS BANCARIOS — "BANCOS SOLIVARES" (CLASE 05) Pág. 34
1. Síntesis de Reglas de Negocio y Decisiones de Diseño de la Profesora Pág. 35
2. Diagramas Oficiales de Cátedra del Caso "Bancos Solivares" (Clases/Clases/05/Banco/) Pág. 35
6.3. CASO PRÁCTICO 3: LABORATORIO DE ANÁLISIS CLÍNICOS (MODELO OFICIAL DE PARCIAL DE ANALÍA MARÍN) Pág. 37
1. Claves de Diseño del Modelo Oficial de Laboratorio Pág. 37
2. Diagramas Oficiales del Caso Laboratorio (EJEMPLO-Practica-Teoria-version-10.pdf) Pág. 38
6.4. CASO PRÁCTICO 4: INSCRIPCIÓN A CLASES EN UN GIMNASIO — CASO GYM (CLASES 01 A 05) Pág. 40
6.5. CASO PRÁCTICO 5: EMPADRONAMIENTO BIOMÉTRICO Y ACCESO A ESTADIOS DE FÚTBOL — AFA (CLASE 06) Pág. 40
PARTE 4-BIS: CASO PRÁCTICO DE TALLER PRE-PARCIAL (CLASE 09 — PRIMERA MITAD [00:00:00 a 01:55:00]) Pág. 41
CASO: "SISTEMA DE SEGURIDAD CIUDADANA — ALERTAS Y DENUNCIAS" (Transcripcion/transcripciones/09_20261015_082342.txt) Pág. 41
PARTE 5: RESOLUCIÓN DE DIAGRAMAS DE ESTADOS DE FINALES RECIENTES (DIC 2025 Y JULIO 2026) Pág. 42
5.1. Final Diciembre 2025 (Parciales/Final_-_2C2025.jpg): Diagrama de Estados de una Reserva de Paquete Turístico Pág. 42
5.2. Final Julio 2026 Mesa B (Parciales/Final_Diseno_Julio_2026.pdf): Diagrama de Estados de un Articulo de Supermercado Pág. 42

6.1. CASO PRÁCTICO 1: SISTEMA DE VENTAS Y LÍNEAS DE VENTA (CASO BASE DE PATRONES GRASP)

💡 EJEMPLO APLICADO DE CÁTEDRA / PLANTILLA DE EXAMEN

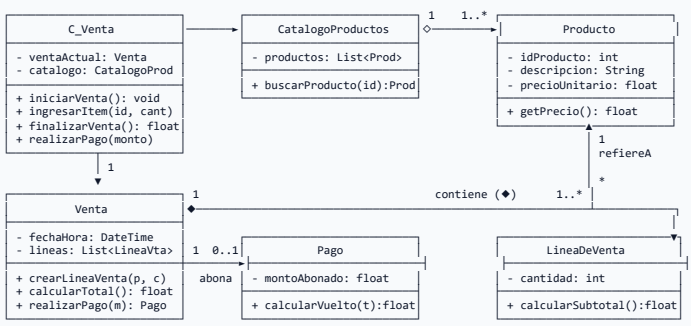
[EJEMPLO DE CLASE / PARCIAL — `UNLaM GRASP-original-marin.pdf` Págs. 24–62]: Este es el caso fundacional con el que Craig Larman y la Prof. Analía Marín enseñan cómo asignar responsabilidades mediante **Experto en Información**, **Creador**, **Controlador**, **Bajo Acoplamiento** y **Alta Cohesión**.

1. Problema y Asignación de Responsabilidades Paso a Paso

En un sistema de Punto de Venta (TPV), un cliente compra varios productos. Cada venta (`Venta`) tiene una fecha/hora y contiene múltiples renglones o ítems (`LineaDeVenta` / `DetalleVenta`), donde cada renglón registra la `cantidad` comprada y referencia al producto del catálogo (`EspecificacionProducto` / `Producto`) que conoce su `descripcion` y `precioUnitario`.

Pregunta de Diseño	Patrón GRASP Aplicado	Decisión Correcta de Diseño y Justificación de Cátedra
1. ¿Quién recibe el evento `ingresarItem(idProd, cant)` desde la pantalla del cajero?	**Controlador (`Controller`)**	La clase `C_Venta` (o `C_Caja`). Recibe los datos primitivos (`idProd`, `cant`), busca la referencia al `Producto` en el `CatalogoProductos` y delega en la instancia actual de `Venta`.
2. ¿Quién debe instanciar (`create`) el objeto `LineaDeVenta`?	**Creador (`Creator`)**	La clase `Venta` mediante su método `+crearLineaVenta(prod: Producto, cant: int)`. **Justificación:** `Venta` contiene y agrega de forma exclusiva (composición `1` ◆── `1..*`) a los objetos `LineaDeVenta` y los registra en su colección interna `-lineas: List<LineaDeVenta>`.
3. ¿Quién debe calcular el total general de la `Venta` (`+calcularTotal()`)?	**Experto en Información (`Information Expert`)**	Es una colaboración entre **3 expertos parciales**: 1) `Venta` es experta en conocer todas sus `LineaDeVenta`, por lo que tiene `+calcularTotal()` que itera sobre su colección. 2) Cada `LineaDeVenta` conoce su `-cantidad` y su referencia a `Producto`, por lo que tiene `+calcularSubtotal()` (`cantidad * prod.getPrecio()`). 3) `Producto` (`EspecificacionProducto`) conoce su propio atributo `-precioUnitario`, por lo que responde a `+getPrecio()`.
4. ¿Quién debe instanciar el objeto `Pago` al abonar la venta: `C_Caja` o `Venta`?	**Bajo Acoplamiento y Alta Cohesión**	Si `C_Caja` crea el `Pago` y luego se lo pasa a `Venta`, acoplamos innecesariamente a `C_Caja` con `Pago`. La solución de **Bajo Acoplamiento** (`UNLaM GRASP-original-marin.pdf` Págs. 40–44) es que `C_Caja` delegue `venta.realizarPago(efectivoEntregado)` y sea la propia `Venta` (Creador) quien instancie a `Pago`.

DIAGRAMA / ESTRUCTURA TÉCNICA UML



6.2. CASO PRÁCTICO 2: OTORGAMIENTO DE PRÉSTAMOS BANCARIOS — "BANCOS SOLIVARES" (CLASE 05)

💡 EJEMPLO APLICADO DE CÁTEDRA / PLANTILLA DE EXAMEN

[EJEMPLO DE CLASE / PARCIAL — `Clases/Clases/05/Banco/` y `Parcial 2C 2025`]: Caso resuelto en vivo por la Prof. Analía Marín en la Clase 05 (`05_20260917_081823.txt` y `05_20260917_101702.txt`), integrando Despliegue, Componentes, Clases, Secuencia, Comunicación y Estados.

1. Síntesis de Reglas de Negocio y Decisiones de Diseño de la Profesora

- El banco ofrece distintos `Producto`**s** financieros (`<<abstract>> Producto` → `Prestamo` y `Cuenta`).
- Un `Prestamo` puede garantizarse con un `Bien` (`<<abstract>> Bien` → `Inmueble` para préstamos hipotecarios o `Automotor` para préstamos prendarios).
- Para otorgar el préstamo, el controlador `C_EvaluacionP` verifica el cumplimiento de cada `Requisito` y consulta servicios externos (`VISA` y `Veraz`) para evaluar el riesgo crediticio del `Cliente`.
- Si la evaluación resulta aprobada, el controlador `C_prestamo` genera el `Prestamo` y éste instancia por **composición fuerte (◆)** su plan de `Cuota`**s** (calculadas según sistema de amortización Francés o Alemán — patrón GoF `Strategy`).

2. Diagramas Oficiales de Cátedra del Caso "Bancos Solivares" (`Clases/Clases/05/Banco/`)

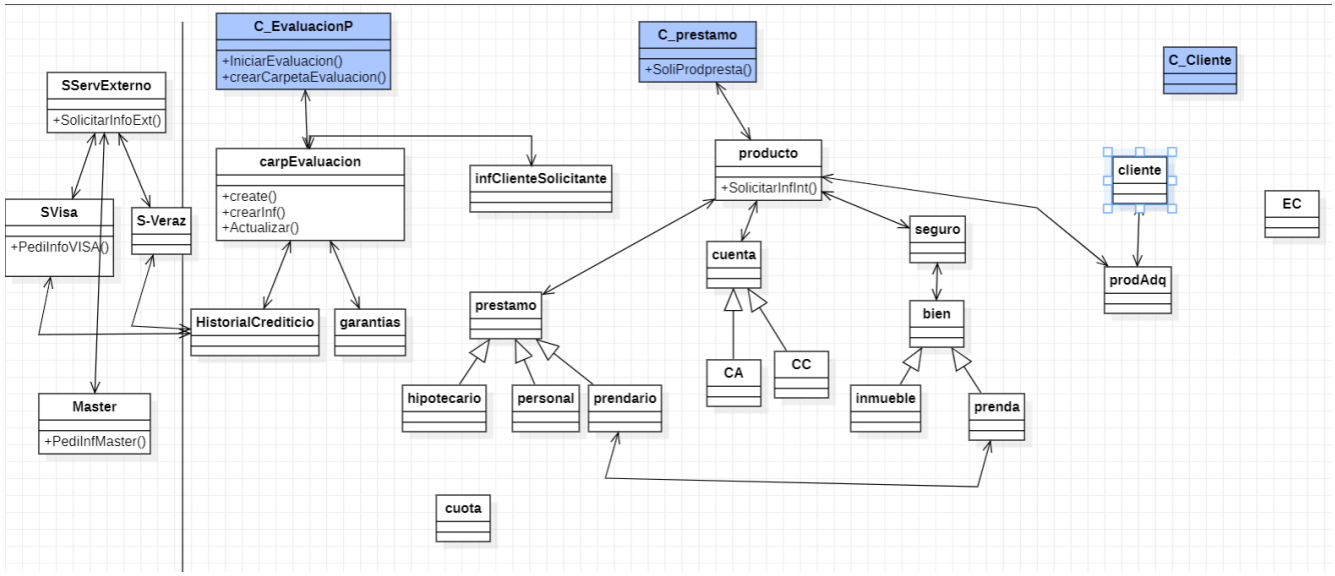


Figura 6.1: Diagrama de Clases original de la Prof. Analía Marín para el Caso Banco (`Banco-diagClase.png`). Obsérvese los controladores `C_EvaluacionP`, `C_prestamo` y `C_Cliente`, las jerarquías de herencia `Producto` y `Bien`, y la composición de `Cuota` y `Requisito`.

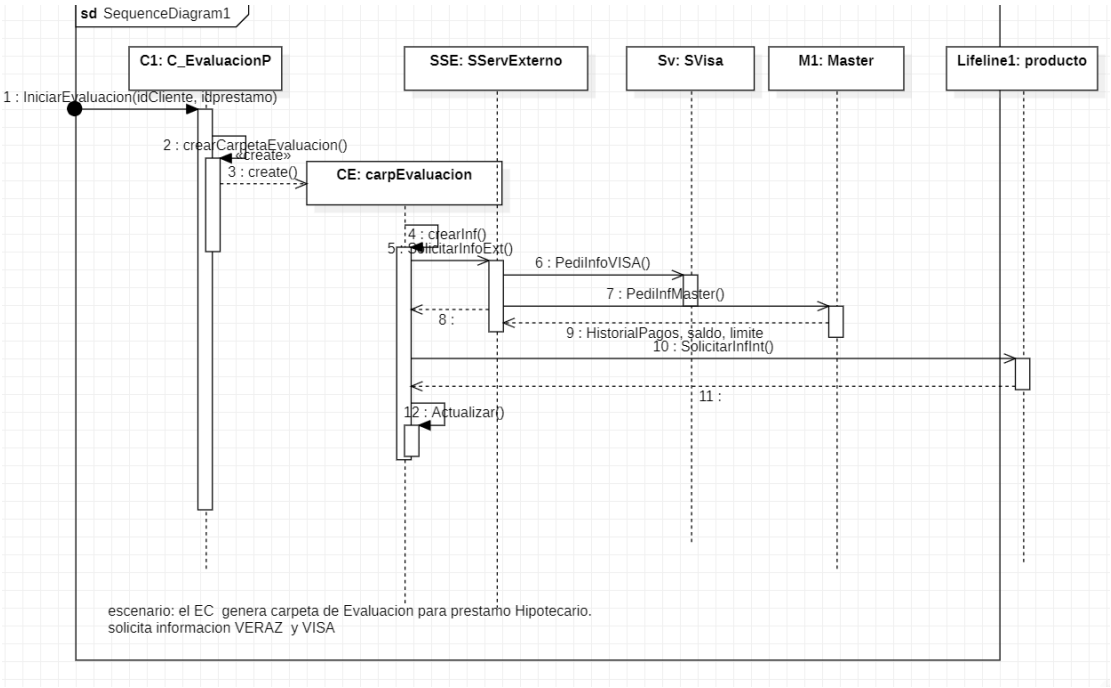


Figura 6.2: Diagrama de Secuencia original de la Prof. Analía Marín (`Banco-diagSecuencia.png`), iniciando directamente en el controlador sin actor humano.

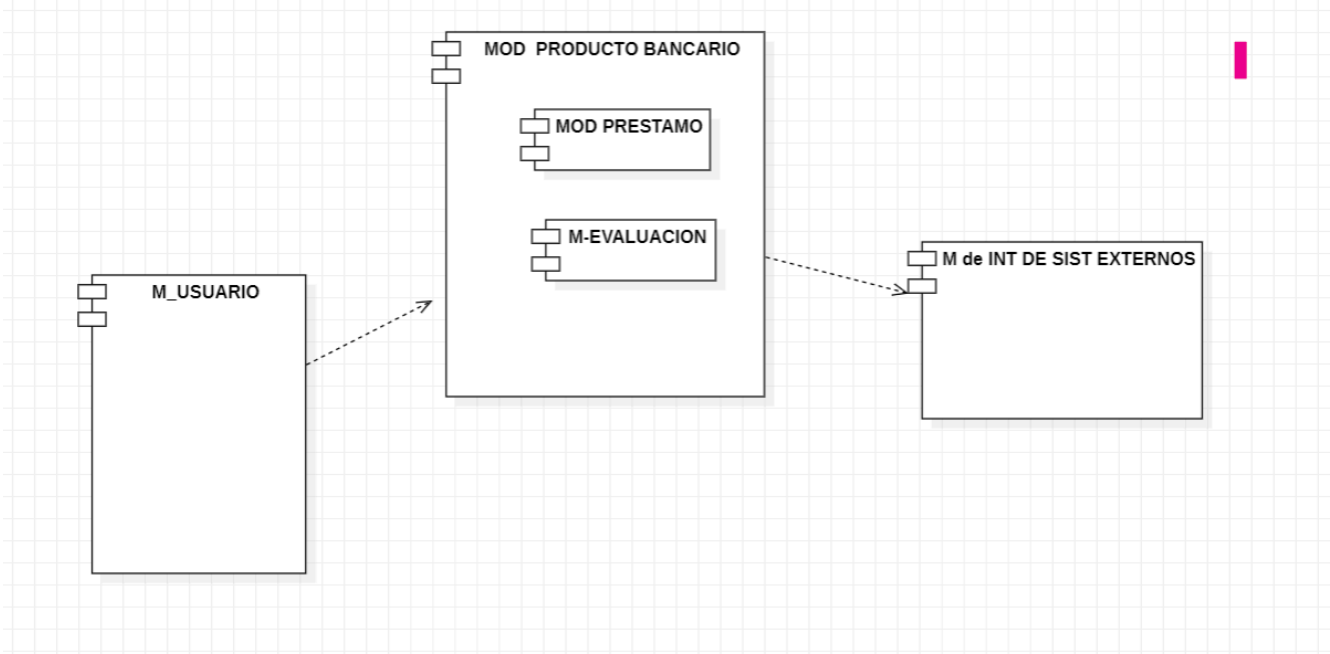


Figura 6.3: Diagrama de Componentes original del Caso Banco (`Banco-diagComponente.png`) con interfaces provistas y requeridas.

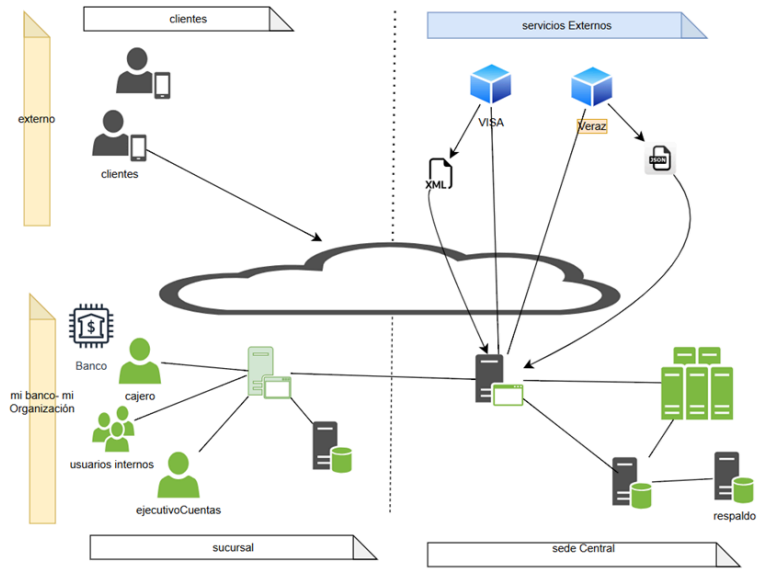


Figura 6.4: Diagrama de Despliegue original del Caso Banco (`Banco-diagDesplieguev2.png`) separando Sucursal, Servidor Central y Servicios Externos (VISA y Veraz).

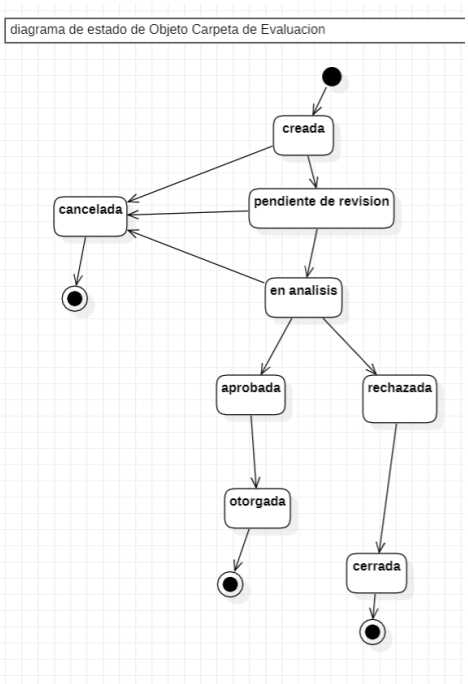


Figura 6.5: Diagrama de Estados del Préstamo (`Banco-diagEstado.png`).

6.3. CASO PRÁCTICO 3: LABORATORIO DE ANÁLISIS CLÍNICOS (MODELO OFICIAL DE PARCIAL DE ANALÍA MARÍN)

> 💡 **EJEMPLO APLICADO DE CÁTEDRA / PLANTILLA DE EXAMEN**
> **[MODELO OFICIAL DE PARCIAL — `Clases/Clases/05/EJEMPLO-Practica-Teoria-version-10.pdf` (33 páginas)]:** Este es el documento maestro elaborado por la propia Ing. Analía Marín como ejemplo integral de todas las vistas UML.

1. Claves de Diseño del Modelo Oficial de Laboratorio

1. **Controladores por Módulo:** `C_turno`, `C_Recepcion`, `C_Analitica`, `C_Validacion`, `C_Pago`.
2. **Separación entre Catálogo Prototípico (`especificacionEstudio`) e Instancia Clínica (`estudio`) — Patrón GoF `Prototype`:**
   - En el catálogo del laboratorio existe `especificacionEstudio` (con el nombre del análisis, muestra requerida, tiempo de ayuno y valores de referencia).
   - Cuando se registra una `Orden` médica con sus `itemOrden`, cada `itemOrden` clona (`clone()`) la `especificacionEstudio` correspondiente para crear la instancia `estudio` donde el bioquímico cargará los `resultado`s de ese paciente.
3. **Clase Pasarela Externa en Verde (`sisExtOS`):** Conecta el dominio del laboratorio con el sistema externo de la Obra Social para validar la cobertura de las prácticas médicas.
4. **Jerarquía Polimórfica de `Pago`:** `Pago` es superclase de `Efectivo`, `Tarjeta` y `Transferencia`.

2. Diagramas Oficiales del Caso Laboratorio (`EJEMPLO-Practica-Teoria-version-10.pdf`)

g) ***Fabricación pura***

- a. ***Principio***:crear una clase artificial (no del dominio) para cumplir la responsabilidad.
  PRINCIPIO MUY UTILIZADO EN EL DISEÑO

h) ***Indirección***

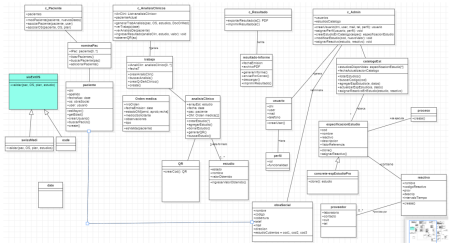
- a. ***Principio***: usar un intermediario para reducir acoplamiento entre componentes.

i) ***Variaciones protegidas***

- ○ ***Principio***: proteger elementos del sistema de los efectos de cambios en otros, creando ***interfaces estables***.

10. Patrones GOF

Diagrama de clases



Ubicamos en la parte superior los controller, modelamos la solución como si fueran capas(controller-modelo de negocio y abajo estaría los objetos que acceden a los datos, que no se modelan porque excede el alcance de la materia). Si usan gestores deberían estar en esa capa de los controler, luego seguimos con las clases mas importantes de negocio. Es MUY importante la ubicación de las clases, porque la lectura en este diagrama es de arriba hacia abajo y de izq a derecha. Las clases con la misma naturaleza las ubico cerca para una mejor lectura, (ayuda mucho definir antes el diag de componente, para llevar a cabo ese ordenamiento).

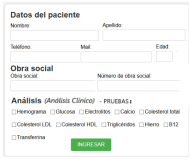
Este diagrama debe tener: nombre de las asociaciones, multiplicidad, navegabilidad (en este diag no lo puse porque respeto la lectura de UML), atributos, comportamiento definido en el caso de estudio, se respeta la escritura de UML

NOTA: Hay una clase en verde porque esa clase se relaciona con un sistema externo, es mi puerta de salida para comunicarme con sistemas externos

Figura 6.6: Diagrama de Clases oficial del Caso Laboratorio (`EJEMPLO-Practica-Teoria-version-10.pdf`, Pág. 10). Nótese la clase verde `sisExtOS` y la relación entre `Orden`, `itemOrden`, `especificacionEstudio` y `estudio`.

Diagrama de Secuencia / comunicación.

Debemos pensar en un escenario para modelar, si no tengo agilidad para pensar en escenarios me puede ayudar pensar cómo sería la pantalla que lanza el evento que captura el controller, les dejo la interfaz donde la recepcionista carga información sobre el paciente



Debo pensar en cómo llevan a cabo dicha funcionalidad. Asumo que la recepcionista lee estudio (pedido del medico) y coloca toda la información en una interfaz (como la que esta modelada) y hago clic en el botón ingresar, ese evento, lo captura el controller acorde al CU o componente, en este caso el controller C-AnalisisClinicos, podría pasarlo a un gestor, pero tendría los mismos métodos que el controller.
Yo saque el gestor, para simplificarlo y el controller tiene la lógica de direccionar el pedido al modelo. Es de buenas practicas de diseño que el controller solo capture el evento y tenga una lógica MINIMA, su responsabilidad es capturar el evento y pasar la responsabilidad al MODELO de negocio.
Defino el escenario y comienzo a modelar el Diag de secuencia.
Saco el actor, el mensaje es de un detinatario anonimo , porque puede ser que el evento lo hizo la recepcionista o recibí un pedido de un servicio, en esta vista no me interesa el actor, eso ya se entendio en el análisis, acá debo entender que el emisor puede venir de diferentes fuentes , por esp coloco un mensaje anonimo ,generarTrabAnalisis(parametros), que lo captura el controller de c_analisiClinicos, el controller debe tener lógica mínima del negocio, solo es un director de orquesta. Si agrego un gestor, el controller solo captura, si no tengo gestor , le adiciono una lógica mínima, yo tome la decisión de sacar el gestor, entonces asumo que el controller va a tener una lógica mínima.
Pienso y defino el siguiente escenario:

- *Envio evento , y lo captura el controller*
- *Si ya tengo un **trabajo** para el día de hoy, lo uso ,, y genero un **análisis clínico** (analisisClinico).*
- *Ese análisis necesita crear varios **estudios**.*
- *En lugar de construirlos "a mano", voy a decir que el sistema **clona las especificaciones de estudio** (especificacionEstudio).*
- *Cada especificación concreta (concrete-espEstudioPro) implementa clone(), devolviendo un **nuevo estudio** con los atributos básicos ya cargados (nombre, tiempo, reactivo).*

*Este escenario me da la idea de usar el **patrón Prototype (GoF)**, lo cual me permite crear nuevos objetos **copiando** (clonando) otros objetos existentes.*
Se usa cuando los objetos a crear comparten gran parte de su estructura y solo cambian en detalles.

Figura 6.7: Diagrama de Secuencia oficial del Caso Laboratorio (`EJEMPLO-Practica-Teoria-version-10.pdf`, Pág. 11) con recuadro de escenario concreto arriba y uso de `create()` y `clone()`.

Un diagrama de estados muestra el cambio de estado de un objeto. Se compone de estados, que representan las situaciones en las que puede encontrarse el sistema; transiciones, las flechas que indican el paso de un estado a otro; y eventos, los desencadenantes que inician esas transiciones.

Los elementos del diag de estado son:

• Estados: Se representan con rectángulos de esquinas redondeadas

• Transiciones: Son las flechas que conectan los estados, indicando el camino de un estado a otro.

• Eventos: Son los estímulos, internos o externos, que activan una transición de un estado a otro. Se suelen etiquetar sobre la flecha de transición.

• Estado inicial: círculo oscuro que indica el punto de partida de la máquina de estados.

• Estado final: círculo con un punto en el interior que señala que el proceso ha concluido

Para que valga la pena modelar el diag de estado hay que pensar en un objeto que realmente sufra cambios de estados en su vida, vamos hacer el Diagrama de Estado del objeto analisisClinico,

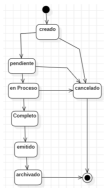
Aclaración, si usan el staruml, se posicionan en la clase del diag de clase, botón derecho y agregar Diagrama de estado, para que esté relacionado con dicho clase



Les recomiendo que piensen qué estados posibles puede transitar ese objeto:

Estados posibles:
• **Creado** (cuando se genera desde la Orden Médica)
• **Pendiente** de Extracción (esperando ser llamado para tomar la muestra)
• **En Proceso** (extracción hecha, y tomando y cargando valores de los estudios)
• **Completado** (todos los estudios ya tienen valores)
• **Emitido** (PDF disponible y enviado)
• **Archivado** (almacenado en histórico del paciente)
• **Cancelado** (cancelado por motivos administrativos o técnicos)

Figura 6.8: Diagrama de Estados de ***`Orden`*** *(* ***`EJEMPLO-Practica-Teoria-version-10.pdf`*** *, Pág. 13).*



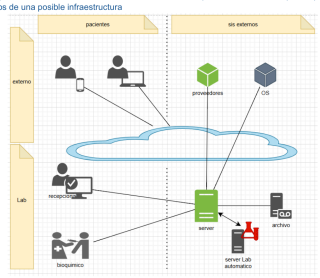
Para estas Transiciones, debemos pensar qué comportamientos se deben ejecutar para hacer este cambio de estado:
Podemos pensar en los siguientes métodos:
• **Creado** -> crearAnalisis()
• **Pendiente** registrarExtraccion()
• **En Proceso** cargarAnValorEstudio()// cuando se carga el primer valor
• **Completado** cargarAnValorEstudio()// cuando se carga el ultimo valor
• **Emitido** finalizarAnalisis()
• **Archivado** archivar// proceso batch
• **Cancelado** cancelado (motivo)// cancelado por diferente motivo: admin, problemas con OS(por ejemplo la OS manda una cancelación), por temas técnicos (por ejemplo no hay reactivo)

Estos comportamientos deben estar reflejado en el Diag de clase
No perdamos de vista que el diagrama de estado se modela para el objeto **analisisClinico** y es el propio objeto que debe ser responsable de transicionar entre estados, porque él es el experto de conocer la información de cambiar el estado (analisisClinico conoce por ejemplo: cantidad de estudios cargados (es contenedor de estudios a realizar), estado actual, condiciones (por ejemplo condiciones si quieren cancelar el analisis))
Existen otros objetos que pueden invocar métodos, como por ejemplo el objeto TRABAJO, que tiene el método CrearAnalisisClin, invoca, pero no cambian atributos internos directamente del objeto de **analisisClinico**.



Figura 6.9: Diagrama de Componentes en notación de cajas anidadas y dependencias (***`EJEMPLO-Practica-Teoria-version-10.pdf`*** *, Pág. 14).*

El segundo diagrama nos basamos en una extensión de elementos que no están en UML, pero representan elementos físicos de una posible infraestructura



Ambos diagramas de despliegue son del mismo sistema, pero con distintos niveles de detalle.

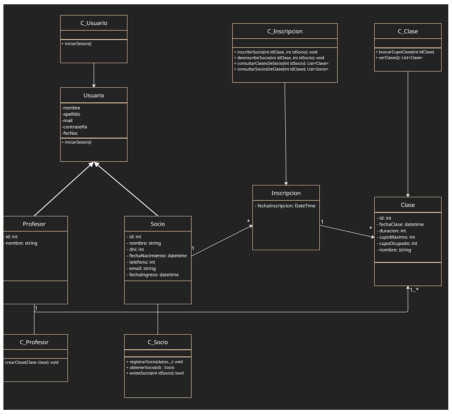
El primero solo se centra en la infraestructura técnica (servidores, conexiones), no distingue a qué dominio pertenece, son solo nodos conectados. Y el segundo diagrama, muestra actores humanos (pacientes, bioquímico, recepcionista), sistemas externos (OS, proveedores), componentes internos (server, serverLab automático, archivo) , modelamos la separación de los dominios y responsabilidades, es decir el segundo ayuda a entender quién usa qué nodo y cuál es la responsabilidad de la organización(laboratorio) y cual es la responsabilidad de serv. Externos. El segundo es más fácil de entender.

Figura 6.10: Diagrama de Despliegue por Dominios de Red (`EJEMPLO-Practica-Teoria-version-10.pdf` , Pág. 16).

6.4. CASO PRÁCTICO 4: INSCRIPCIÓN A CLASES EN UN GIMNASIO — CASO GYM (CLASES 01 A 05)

> 💡 **EJEMPLO APLICADO DE CÁTEDRA / PLANTILLA DE EXAMEN**
> **[EJEMPLO DE CLASE — `EJEMPLO-Practica-Teoria-version-10.pdf` Pág. 28 y `GUIA-EjercicioGYM-Estudiantes-2026- V01.docx`]:** Trabajado durante las primeras 5 clases para enseñar cómo validar precondiciones de negocio y cuándo introducir una clase asociativa `Inscripcion` .

- **Regla de negocio:** Un `Socio` solicita inscribirse a una `Clase` (actividad con `Profesor` , `horario` y `cupoMaximo`). El controlador `C_Inscripcion` busca al `Socio` y a la `Clase` , verifica que el socio tenga `aptoMedico` vigente y `cuota` al día, verifica que la `Clase` tenga cupo disponible (`+hayCupo()`), y de cumplirse todo, instancia la clase intermedia `Inscripcion` .



Cada clase debe responder sobre su propio estado

Entonces:

- Clase sabe si tiene cupo
- C_Clase coordina / busca / delega
- C_Inscripcion orquesta el caso de uso

Figura 6.11: Diagrama de Clases oficial del Caso Gimnasio (`EJEMPLO-Practica-Teoria-version-10.pdf` , Pág. 28).

6.5. CASO PRÁCTICO 5: EMPADRONAMIENTO BIOMÉTRICO Y ACCESO A ESTADIOS DE FÚTBOL — AFA (CLASE 06)

> 💡 **EJEMPLO APLICADO DE CÁTEDRA / PLANTILLA DE EXAMEN**
> **[EJEMPLO DE CLASE — `Clases/Clases/06/Estadio/` y `06_20260924_082126.txt`]:** Práctica integradora completa de las 4+1 Vistas de Kruchten.

- **Reglas clave enseñadas por Analía Marín en este caso:**
  - **Uso de la clase contenedora `Padron` (Multiobjeto / Fabricación Pura):** Una instancia de `Aficionado` no puede buscarse a sí misma ni buscar a otros aficionados por DNI. Por eso `c_empadronar` y `c_ingreso` se comunican con `Padron` (`+buscarAficionado(dni): Aficionado`).
  - **Composición exclusiva de datos biométricos:** `Aficionado` está compuesto (◆) por `1` `Huella` y `1` `Iris`, y se asocia a su `Entrada`.
  - **Validación externa en paralelo:** En el Diagrama de Actividad y Despliegue, el sistema valida identidad con **Renaper** y antecedentes (derecho de admisión) con **Policía**.

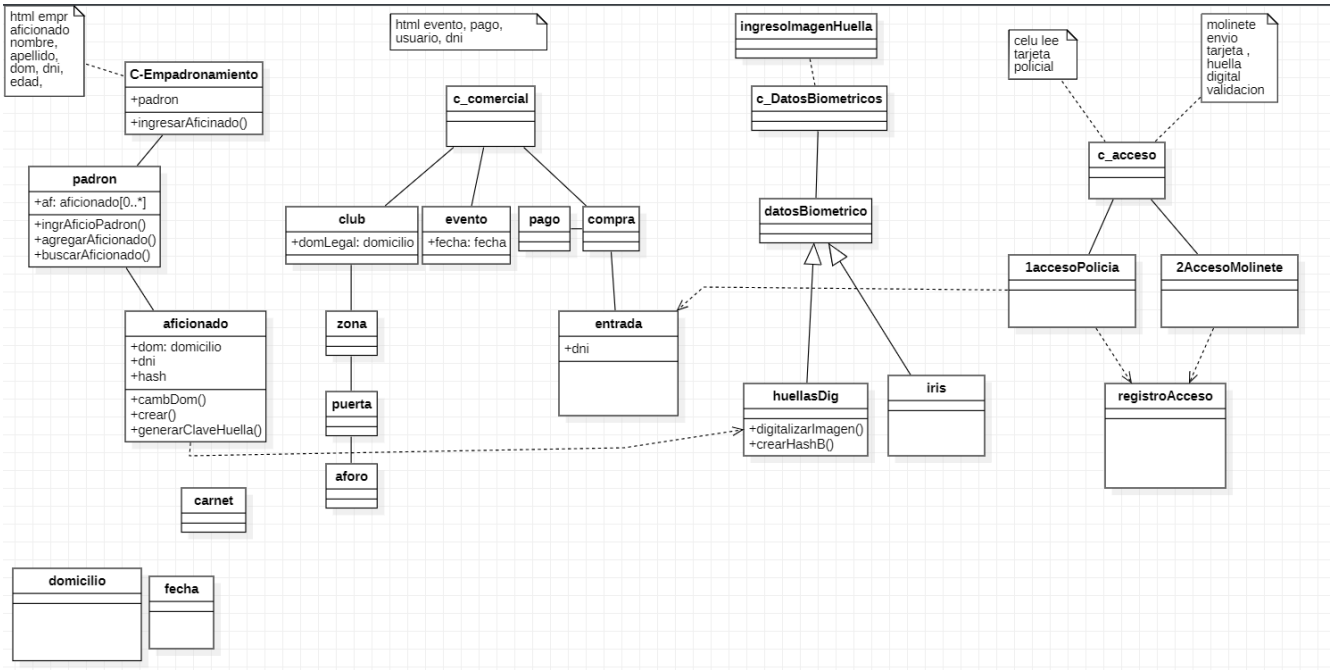


Figura 6.12: Diagrama de Clases oficial del Caso Estadios (`Estadio/Clase2.png`) mostrando `c_empadronar`, `c_ingreso`, `Padron`, `Aficionado`, `Huella`, `Iris` y `Entrada`.

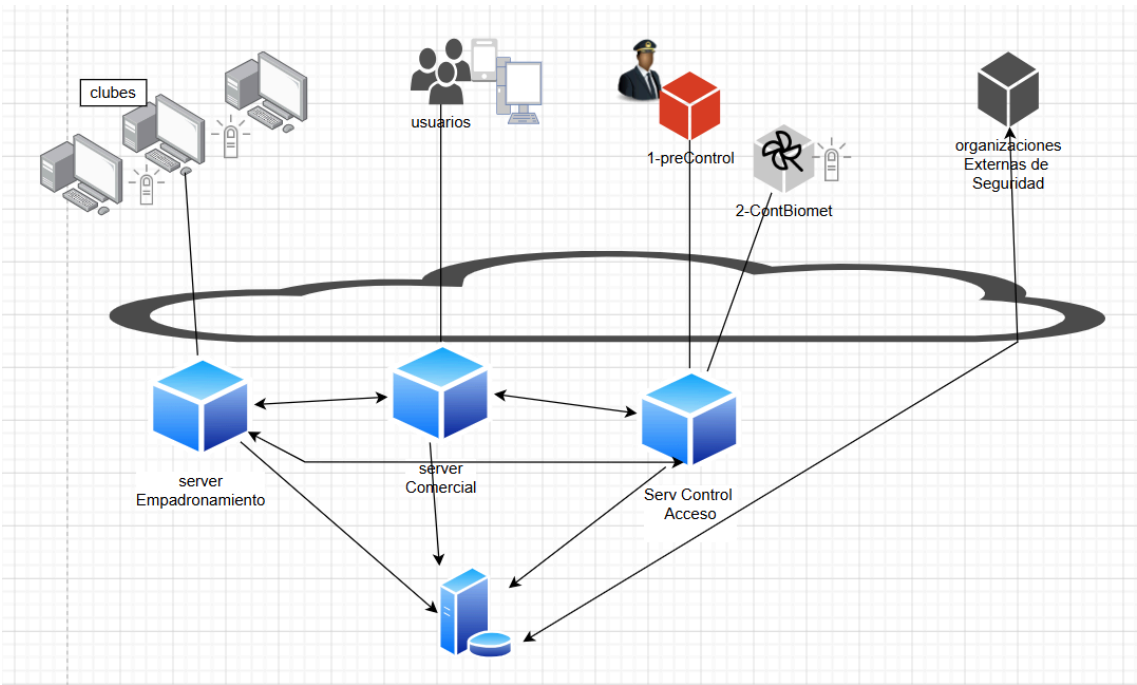


Figura 6.13: Diagrama de Despliegue oficial del Caso Estadios (`Estadio/Despliegue2.png`) con Tótem Biométrico, Servidor Local, Servidor Central AFA y Renaper/Policía.

PARTE 4-BIS: CASO PRÁCTICO DE TALLER PRE-PARCIAL (CLASE 09 — PRIMERA MITAD `[00:00:00 a 01:55:00]`)

CASO: "SISTEMA DE SEGURIDAD CIUDADANA — ALERTAS Y DENUNCIAS" (`Transcripcion/transcripciones/09_20261015_082342.txt`)

> 🎓 **CRITERIO Y RECOMENDACIÓN DE LA PROFESORA**
> **Por qué estudiar este caso:** Durante la primera hora y cincuenta minutos de la Clase 09, la Ing. Analía Marín y Alejandro corrigieron en vivo este modelo de parcial con los grupos. De aquí salieron varias de las advertencias clave para el examen.

4-Bis.1. Requisitos del Caso Discutidos en Clase 09

1. Los **Ciudadanos (Vecinos)** registrados pueden emitir **Alertas rápidas (Botón Antipánico)** o cargar **Denuncias** detalladas (con fotos, audio/video, geolocalización y tipo de incidente: robo, incendio, violencia, accidente).

2. El sistema **evalúa la veracidad / riesgo de la denuncia** para detectar **falsas denuncias**: si un ciudadano acumula múltiples falsas denuncias, se lo **penaliza / bloquea**.
3. Si la alerta o denuncia es válida, se genera un **Incidente**, se calcula su **prioridad/gravedad**, se despacha el recurso correspondiente (**Patrulla Policial, Bomberos, Ambulancia**) según cercanía y disponibilidad, y se actualiza el **Mapa de Calor (*Heat Map*)** delictivo por zona.

4-Bis.2. Lecciones de Corrección de la Profesora sobre este Caso (`09_20261015_082342.txt`)

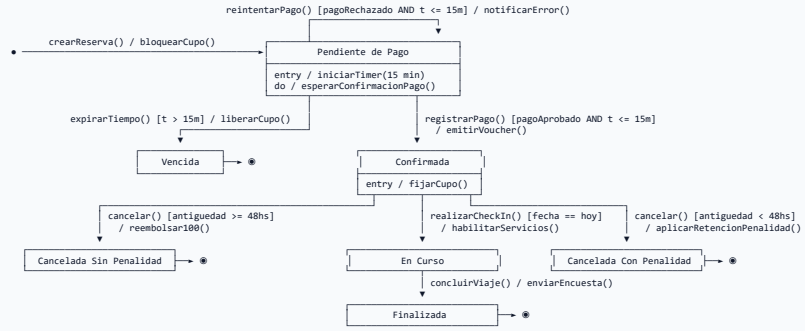
1. **Diferencia entre `Alerta` y `Denuncia`:** Pueden modelarse con una superclase abstracta `<<abstract>> ReporteCiudadano` (con `fechaHora`, `ubicacionGPS`, `estado`) de la cual heredan `AlertaAntipanico` (inmediata, prioridad crítica) y `Denuncia` (con `evidenciaMultimedia`, `descripcion`, `evaluacionVeracidad`).
2. **Evaluación de Falsas Denuncias y Penalización del Ciudadano:**
   - La clase `Ciudadano` conoce su atributo `-cantidadFalsasDenuncias: int` y su `-estadoCuenta`.
   - Cuando un operador o validador marca una denuncia como falsa (`denuncia.marcarComoFalsa()`), la denuncia le avisa al `Ciudadano` (`ciudadano.registrarFalsaDenuncia()`), y es el propio `Ciudadano` (**Experto**) quien incrementa su contador y evalúa si supera el umbral para bloquearse.
3. **Patrones GoF Ideales validados por la Profesora en este Caso:**
   - `Observer` **(Comportamiento):** Cuando se confirma un `Incidente`, notifica automáticamente a las centrales externas (`CentralPolicia`, `CuartelBomberos`, `SAME_Emergencias`) y actualiza el `MapaDeCalor`.
   - `Strategy` **(Comportamiento):** Para el algoritmo de **cálculo de nivel de prioridad/criticidad del incidente** o para la **estrategia de asignación de patrulla más cercana**.
   - `State` **(Comportamiento):** Ciclo de vida de la `Denuncia` / `Incidente` (`Reportada` → `EnVerificacion` → `ConfirmadaEnAtencion` / `DesestimadaPorFalsa` → `Resuelta`).
   - `Adapter` **(Estructural):** Para conectarse con los sistemas externos del **911 / Cámaras Municipales / Bomberos**.

PARTE 5: RESOLUCIÓN DIAGRAMAS DE ESTADOS DE FINALES RECIENTES (DIC 2025 Y JULIO 2026)

5.1. Final Diciembre 2025 (`Parciales/Final_-_2C2025.jpg`): Diagrama de Estados de una `Reserva` de Paquete Turístico

- **Reglas del enunciado:**
  - La reserva se crea en estado **Pendiente de Pago** y mantiene el bloqueo de cupos durante **15 minutos**.
  - Si pasan los 15 minutos sin recibir pago, pasa a **Expirada / Vencida** (estado final) liberando el cupo.
  - Si el pago es rechazado, puede reintentar mientras no hayan pasado los 15 minutos.
  - Si el pago es aprobado dentro de los 15 minutos, pasa a **Confirmada**.
  - Desde **Confirmada**, el cliente puede cancelar: si cancela con más de 48 hs de anticipación pasa a **Cancelada sin Penalidad (Reembolso Total)**; si cancela con menos de 48 hs pasa a **Cancelada con Penalidad**.
  - Llegada la fecha del viaje, el pasajero realiza el check-in pasando a **En Curso (Check-in Realizado)**, y al concluir el viaje pasa a **Finalizada**.

DIAGRAMA / ESTRUCTURA TÉCNICA UML



5.2. Final Julio 2026 Mesa B (`Parciales/Final_Diseno_Julio_2026.pdf`): Diagrama de Estados de un `Articulo` de Supermercado

- **Ciclo de vida del objeto `Articulo`:**
  - `●` ── `solicitarAProveedor()` ──► `Pedido a Proveedor`
  - `Pedido a Proveedor` ── `recibirMercaderia() [controlCalidadOK] / incrementarStock()` ──► `Disponible en Stock`
  - `Disponible en Stock` ── `vender() [stockActual > stockMinimo] / descontarStock()` ──► `Disponible en Stock` *(auto-transición)*
  - `Disponible en Stock` ── `vender() [stockActual <= stockMinimo AND stockActual > 0] / generarAlertaReposicion()` ──► `En Punto de Pedido (Stock Crítico)`
  - `En Punto de Pedido` ── `vender() [stockActual == 0]` ──► `Agotado (Sin Stock)`
  - `En Punto de Pedido` o `Agotado` ── `recibirReposicion() / actualizarStock()` ──► `Disponible en Stock`
  - `Disponible en Stock` ── `detectarVencimientoORotura() / retirarDeGondola()` ──► `Dado de Baja / Devolución a Proveedor` ──► `◉`.

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 7: PARCIALES 2026 RESUELTOS PASO A PASO (PETCARE, REENCUENTRO/ANIMALES EXÓTICOS Y AGENCIA DE VIAJES)

📑 **ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO**

PARTE 2: RESOLUCIÓN COMPLETA DEL PARCIAL 1C 2026 (COMISIÓN 4300 — ING. ANALÍA MARÍN) Pág. 43
PUNTO 1 (PARCIAL PETCARE): DIAGRAMA DE CLASES DE DISEÑO (MÁXIMO 4 CONTROLLERS) + GRASP Pág. 44
PUNTO 3 (PARCIAL PETCARE): APLICACIÓN Y JUSTIFICACIÓN DE 2 PATRONES GOF DE DIFERENTE CATEGORÍA Pág. 45
CASO: "SOLUCIÓN DE REENCUENTRO Y ADOPCIÓN DE MASCOTAS" Y ALERTA DE ANIMALES EXÓTICOS (Parciales/2. -Parcial 1--1C2026--1900--Valeria y Analia-----.pdf) Pág. 45
PUNTO 2 (REENCUENTRO Y ANIMALES EXÓTICOS): MODIFICACIÓN DEL DIAGRAMA DE COMPONENTES ACORDE A LOS 4 CONTROLADORES Pág. 47
PUNTO 4 (REENCUENTRO Y ANIMALES EXÓTICOS): 2 PATRONES GOF DE DIFERENTE CATEGORÍA (SIN SINGLETON NI FACADE) Pág. 48
CASO: "AGENCIA DE VIAJES" (Parciales/drive-download-20261111T183304Z-1-001/3. [Parcial 1][2C2026][1900][][][].pdf) Pág. 48
PUNTOS 3 Y 4 (AGENCIA DE VIAJES — TEMA 1): INTEGRACIÓN DEL DIAGRAMA DE ESTADOS DE Reserva Y DIAGRAMA DE CLASES COMPLETO (SOLO 4 CONTROLADORES + 2 PATRONES GOF A Y B) Pág. 49
CASO: RED DE VETERINARIAS "PETCARE" (Parciales/1. [Parcial 1][1C2026][4300][Analia][][].pdf) Pág. 43
PUNTO 2 (PARCIAL PETCARE): ESCENARIO Y DIAGRAMA DE SECUENCIA DE "Guardar Consulta" Pág. 45
PARTE 3: RESOLUCIÓN COMPLETA DEL PARCIAL 1C 2026 (COMISIÓN 1900 — VALERIA Y ANALÍA MARÍN) Pág. 45
PUNTO 1 (REENCUENTRO, ADOPCIÓN Y ANIMALES EXÓTICOS): DIAGRAMA DE CLASES (SOLO 4 CONTROLADORES) + GRASP Pág. 46
PUNTO 3 (REENCUENTRO Y ANIMALES EXÓTICOS): DIAGRAMA DE SECUENCIA — ESCENARIO DENUNCIA DE ANIMAL EXÓTICO (LEOPARDO EN MORÓN) Pág. 48
PARTE 4: RESOLUCIÓN COMPLETA DEL PARCIAL 2C 2026 (COMISIÓN 1900 — TEMA 1) Pág. 48
PUNTO 1 (AGENCIA DE VIAJES — TEMA 1): ANÁLISIS CRÍTICO Y MEJORA DEL DIAGRAMA DE COMPONENTES Pág. 49
PUNTO 2 (AGENCIA DE VIAJES — TEMA 1): ESCENARIO MEJORADO Y DIAGRAMA DE SECUENCIA PARA "realizar Pago" (PASO 4 EN ADELANTE) Pág. 50

PARTE 2: RESOLUCIÓN COMPLETA DEL PARCIAL 1C 2026 (COMISIÓN 4300 — ING. ANALÍA MARÍN)

CASO: RED DE VETERINARIAS "PETCARE" (`Parciales/1. [Parcial 1][1C2026][4300][Analia][][].pdf`)

🎓 **CRITERIO Y RECOMENDACIÓN DE LA PROFESORA**

Fuente verificada visualmente: Escaneo original de 2 páginas de `Parciales/1. [Parcial 1][1C2026][4300][Analia][][].pdf` + corrección en vivo de la Ing. Analía Marín y el JTP Alejandro en la **Clase 09** (`09_20261015_082342.txt`, **desde** `[01:55:00]` **hasta** `[03:45:00]`).

2.1. Enunciado Original Verificado y Elementos Impresos en la Hoja del Examen

1. **Dominio del Problema (`[EXIGENCIA EXPLÍCITA DEL ENUNCIADO]`):**
   - **"PetCare"** es una **red de veterinarias distribuida en distintas ciudades**.
   - Un **Usuario (dueño)** se registra en el sistema y registra a sus **Mascotas** (cada mascota posee una única **Historia Clínica**).
   - El usuario puede **buscar sucursales de** `Veterinaria` **por ubicación y horarios**, consultar los `Profesional`**es** que atienden en cada veterinaria (con su especialidad y días/horarios de atención) y **reservar un** `Turno` para una mascota con un profesional en una sucursal.
   - Durante la **Atención Veterinaria**, el profesional accede a la **Historia Clínica** de la mascota y registra la `Consulta` del día:
   - Actualiza el **peso** de la mascota.
   - Registra el `Diagnóstico` y el `Tratamiento` (opcional).
   - Puede emitir una `Receta` (medicamento, dosis/indicación), ordenar un `Estudio` y/o registrar la aplicación de una `VacunaAplicada` (`Vacuna`).
   - Puede programar un `Recordatorio` futuro (con fecha y motivo, ej. próxima visita o refuerzo) para que el sistema **notifique al dueño por WhatsApp o Mail**.
   - **Nota manuscrita en el borde superior del examen:** `max: 4 controladores`.
2. **Diagramas Base Impresos en la Hoja del Examen (`[EXIGENCIA EXPLÍCITA DEL ENUNCIADO]`):**
   - **Diagrama de Componentes Inicial:** Muestra los bloques `gestion/usuario`, `mascota`, `turno`, `veterinaria/profesional`, `AtencionVeterinario` (conectado a `historialClinico`) y `notificaciones`.
   - **Diagrama de Estados de** `Consulta Veterinaria`**:**

DIAGRAMA / ESTRUCTURA TÉCNICA UML



- - **Captura de Pantalla y Escenario Exacto del Punto 2 (`"Guardar Consulta"`):**
  - **Fecha:** `06/06/2026`.
  - **Caso:** El veterinario atiende **por primera vez** a `Homero`, un **gato siamés de 13 años**, cuyo peso actual es `5.2 kg`, y su dueño es `Juan Pérez`.
  - **Acciones cargadas en la pantalla antes de presionar el botón** `Guardar`**:**
  - Se le da **1 vacuna**: `"Rabia"`.
  - Se registra su **peso**: `5.2 kg`.
  - Se le da **1 receta** para tomar pastillas desparasitarias: `"Pastillas desparasitarias"`, `"1 comprimido"`.
  - Se registra `Diagnóstico`: `"SANO"`.
  - `Tratamiento`: vacío (`""`).
  - Se configura `Recordatorio`: `"volver a verlo en 1 año"` (fecha `01/06/2027`).
  - Al hacer clic en el botón `Guardar`, llega el mensaje `"Guardar Consulta"`, el cual debe crear y asociar los objetos correspondientes (`Consulta`, `Diagnostico`, `Receta`, `VacunaAplicada`, `Recordatorio`, etc.) y agregarlos a la `HistoriaClinica` de la mascota.
- **Lo que pide la Consigna (`Se Pide`):**
  - **Punto 1:** Construir el **Diagrama de Clases** completo que soporte todos los requerimientos e integre el Diagrama de Estados dado (aplicando **máximo 4 controladores**), e identificar y justificar **al menos 2 patrones GRASP**.
  - **Punto 2:** Construir el **Diagrama de Secuencia** para el escenario dado del gato `Homero` al presionar `"Guardar Consulta"`.
  - **Punto 3:** Aplicar **2 patrones GoF de diferente categoría** sobre el Diagrama de Clases, respondiendo: a) Categoría y nombre, b) Problema que resuelve, c) Justificación, d) Clases reutilizadas y nuevas.

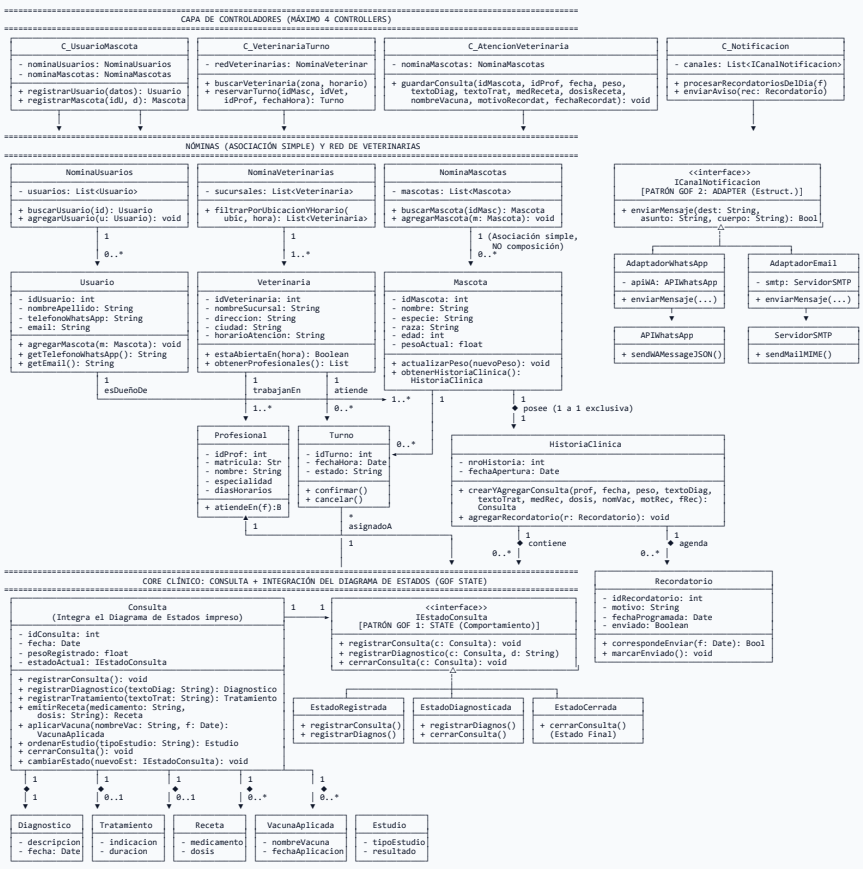
PUNTO 1 (PARCIAL PETCARE): DIAGRAMA DE CLASES DE DISEÑO (MÁXIMO 4 CONTROLLERS) + GRASP

1.A. Diferenciación entre Exigencias Explícitas y Decisiones de Diseño

- `[EXIGENCIA EXPLÍCITA DEL ENUNCIADO]`: Clases `Veterinaria`, `Profesional`, `Usuario`, `Mascota`, `Turno`, `HistoriaClinica`, `Consulta` (con los métodos del Diagrama de Estados: `+registrarConsulta()`, `+registrarDiagnostico()`, `+cerrarConsulta()`), `Diagnostico`, `Tratamiento`, `Receta`, `Estudio`, `VacunaAplicada` (`Vacuna`), `Recordatorio` y envío de notificaciones por WhatsApp/Mail. Tope máximo de **4 controladores**.
- `[DECISIÓN DE DISEÑO / INTERPRETACIÓN VALIDADA EN CLASE 09]`:
  - **Los 4 Controladores (`max: 4 controladores`):**
  - `C_UsuarioMascota`: Registro y búsqueda de usuarios (dueños) y sus mascotas.
  - `C_VeterinariaTurno`: Búsqueda de sucursales de `Veterinaria` por ubicación/horario, profesionales y reserva de `Turno`.
  - `C_AtencionVeterinaria`: Recibe el evento `"Guardar Consulta"` (`+guardarConsulta(...)`), coordina la actualización de la mascota, la creación de la `Consulta` en la `HistoriaClinica` y la programación del `Recordatorio`.
  - `C_Notificacion`: Gestiona el disparo de recordatorios y avisos a través de los canales externos (WhatsApp / Mail).
  - **Uso de `Nomina` vs. Contenedor propio (`Clase 09 [02:38:03]`):** Se incluye `NominaMascotas` (o `NominaUsuarios`) unida con **asociación simple (——)** para buscar globalmente a `Homero` por su ID al guardar la consulta, mientras que todas las consultas de `Homero` se guardan dentro de su propia `HistoriaClinica` (composición `1` a `1` con `Mascota`), sin inventar una "Nómina de Consultas".
  - **Los 2 Patrones GoF integrados:**
  - **Patrón GoF 1 (Comportamiento) — `State`:** Aplicado directamente para modelar el **Diagrama de Estados de `Consulta Veterinaria`** que viene impreso en el parcial (`<<interface>> IEstadoConsulta` implementado por `EstadoRegistrada`, `EstadoDiagnosticada`, `EstadoCerrada`). *(Nota: También es 100% válido usar `Observer` para los recordatorios, como discutió la profesora en Clase 09).*
  - **Patrón GoF 2 (Estructural) — `Adapter`:** Aplicado para desacoplar el envío de notificaciones de `Recordatorio` hacia las APIs externas de **WhatsApp** (`AdaptadorWhatsApp` → `API_WhatsAppCloud`) y **Mail** (`AdaptadorEmail` → `ServidorSMTPMail`) detrás de la interfaz `<<interface>> ICanalNotificacion`.

1.B. Diagrama de Clases Completo (`PetCare` — Com. 4300)

DIAGRAMA / ESTRUCTURA TÉCNICA UML



1.C. Justificación de los Patrones GRASP en el Diseño (`PetCare`)

1. **Controlador (*Controller*):** Se aplica en las 4 clases controladoras `C_UsuarioMascota`, `C_VeterinariaTurno`, `C_AtencionVeterinaria` y `C_Notificacion` (respetando el máximo de 4 controladores de la consigna). En particular, `C_AtencionVeterinaria` recibe el evento externo `+guardarConsulta(...)` cuando el veterinario presiona el botón `"Guardar"`, desacoplando la pantalla de las clases de dominio y delegando la actualización de la mascota y el registro clínico.
2. **Experto en Información (*Information Expert*):** Se aplica en `Mascota` con los métodos `+actualizarPeso(nuevoPeso)` y `+obtenerHistoriaClinica()` (porque `Mascota` conoce su propio atributo `-pesoActual` y posee la referencia directa `1 a 1` a su `HistoriaClinica`), y en `HistoriaClinica` con el método `+crearYAgregarConsulta(...)`, ya que es la clase experta que contiene y administra el historial completo de consultas y recordatorios de esa mascota.
3. **Creador (*Creator*):** Se aplica en `HistoriaClinica` al instanciar (`create`) a `Consulta` y a `Recordatorio` (porque `HistoriaClinica` agrega y contiene por composición todas las consultas y recordatorios de la mascota), y a su vez en `Consulta` al instanciar (`create`) a `Diagnostico`, `Receta` y `VacunaAplicada` (mediante `+registrarDiagnostico()`, `+emitirReceta()` y `+aplicarVacuna()`), ya que `Consulta` contiene de manera exclusiva los elementos clínicos generados en ese acto médico y recibe los datos de inicialización.
4. **Bajo Acoplamiento y Alta Cohesión (*Low Coupling & High Cohesion*):** `C_AtencionVeterinaria` no instancia directamente a `Diagnostico`, `Receta` ni `VacunaAplicada`, sino que delega en `HistoriaClinica` y `Consulta`, manteniendo bajo el acoplamiento del controlador y alta la cohesión de `Profesional` (que solo guarda sus datos profesionales sin convertirse en una clase gestora).

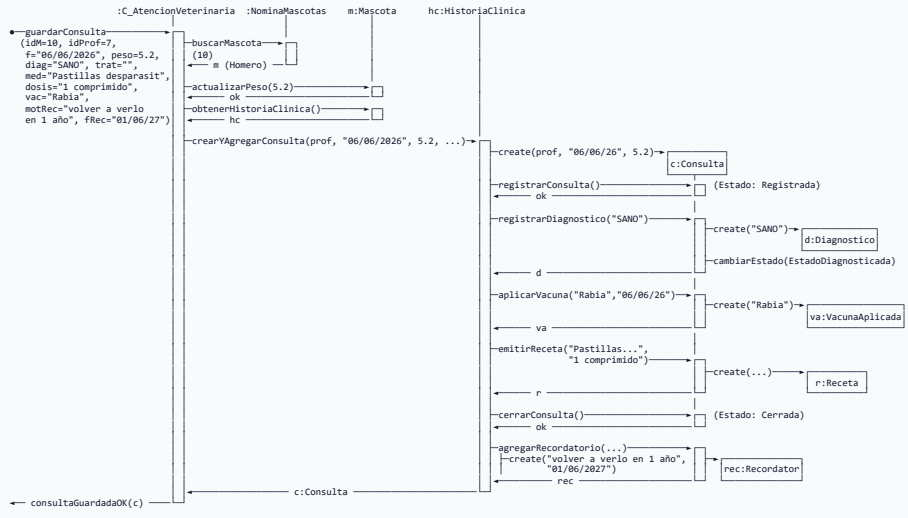
PUNTO 2 (PARCIAL PETCARE): ESCENARIO Y DIAGRAMA DE SECUENCIA DE `"Guardar Consulta"`

2.A. Escenario Concreto (Datos Exactos del Enunciado Impreso)

> 📌 **ESCENARIO CONCRETO / CONSIGNA DE PARCIAL**
>
> **Escenario (06/06/2026):** El profesional veterinario (`idProf = 7`) atiende por primera vez a `Homero` (`idMascota = 10` , gato siamés de 13 años, dueño `Juan Pérez`). Registra su peso actual de `5.2 kg` , aplica **1 vacuna** (`"Rabia"`), emite **1 receta** (`"Pastillas desparasitarias"` , `"1 comprimido"`), carga el **Diagnóstico** `"SANO"` , deja el **Tratamiento** vacío (`""`), y programa **1 Recordatorio** `"volver a verlo en 1 año"` para el `"01/06/2027"` . Presiona el botón `"Guardar"` disparando el mensaje `guardarConsulta(...)` .

2.B. Diagrama de Secuencia Completo (100% Trazable con el Diagrama de Clases y el Diagrama de Estados)

DIAGRAMA / ESTRUCTURA TÉCNICA UML



(Nota sobre Tratamiento vacío: Como en el escenario del enunciado el campo Tratamiento está vacío `""` , no se ejecuta `registrarTratamiento` en este escenario concreto, o bien puede encerrarse en un fragmento `opt [textoTrat != vacio]`).

PUNTO 3 (PARCIAL PETCARE): APLICACIÓN Y JUSTIFICACIÓN DE 2 PATRONES GOF DE DIFERENTE CATEGORÍA

Patrón GoF N° 1: `State` **(Categoría: Comportamiento)**

- **a) Categoría y Nombre:** Patrón de **Comportamiento** — **State (Estado)**.
- **b) Problema de diseño que resuelve en el caso:** El enunciado provee el Diagrama de Estados de `Consulta Veterinaria` (`Registrada` → `Diagnosticada` → `Cerrada`). Una consulta solo puede cerrarse si previamente fue diagnosticada, y una vez `Cerrada` no debe permitir modificar el diagnóstico ni agregar nuevas recetas o vacunas. Implementar esto con múltiples condicionales `if/switch` sobre un String dentro de `Consulta` violaría el principio Abierto/Cerrado y la Alta Cohesión.
- **c) Por qué es adecuado:** Encapsula cada estado del diagrama impreso en una clase separada que implementa la interfaz común `<<interface>> IEstadoConsulta` , delegando las transiciones (`+registrarConsulta()` , `+registrarDiagnostico()` , `+cerrarConsulta()`) al objeto de estado actual.
- **d) Clases reutilizadas del Punto 1 y Clases/Interfaces nuevas incorporadas:**
  - *Clase reutilizada (`Context`):* `Consulta` , que mantiene el atributo `-estadoActual: IEstadoConsulta` y el método `+cambiarEstado(e)` .
  - *Interfaz nueva incorporada (`State`):* `<<interface>> IEstadoConsulta` .
  - *Clases concretas nuevas incorporadas (`ConcreteStates`):* `EstadoRegistrada` , `EstadoDiagnosticada` y `EstadoCerrada` .

Patrón GoF N° 2: `Adapter` **(Categoría: Estructural)**

- **a) Categoría y Nombre:** Patrón **Estructural** — **Adapter (Adaptador)**.
- **b) Problema de diseño que resuelve en el caso:** El sistema PetCare necesita enviar las notificaciones de los `Recordatorio` s (como el recordatorio anual de `Homero`) y avisos de turnos a través de servicios externos de mensajería (**WhatsApp** y **Mail**), los cuales poseen APIs de terceros incompatibles entre sí y ajenas a nuestro modelo de dominio.
- **c) Por qué es adecuado:** Define una interfaz única del dominio (`<<interface>> ICanalNotificacion`) y utiliza clases adaptadoras (`AdaptadorWhatsApp` y `AdaptadorEmail`) que traducen el mensaje `+enviarMensaje(...)` a los métodos nativos de cada proveedor externo (`APIWhatsApp` y `ServidorSMTP`), logrando **Bajo Acoplamiento**, **Indirección** y **Variaciones Protegidas**.
- **d) Clases reutilizadas del Punto 1 y Clases/Interfaces nuevas incorporadas:**
  - *Clases reutilizadas (`Client`):* `C_Notificacion` y `Recordatorio` .
  - *Interfaz nueva incorporada (`Target`):* `<<interface>> ICanalNotificacion` .
  - *Clases nuevas incorporadas (`Adapters` y `Adaptees`):* `AdaptadorWhatsApp` (que envuelve a `APIWhatsApp`) y `AdaptadorEmail` (que envuelve a `ServidorSMTP`).

PARTE 3: RESOLUCIÓN COMPLETA DEL PARCIAL 1C 2026 (COMISIÓN 1900 — VALERIA Y ANALÍA MARÍN)

CASO: "SOLUCIÓN DE REENCUENTRO Y ADOPCIÓN DE MASCOTAS" Y ALERTA DE ANIMALES EXÓTICOS (`Parciales/2. -Parcial 1--1C2026--1900--Valeria y Analia-----.pdf`)

> 🎓 **CRITERIO Y RECOMENDACIÓN DE LA PROFESORA**
>
> **Fuente verificada visualmente:** Escaneo original de 2 páginas de `Parciales/2. -Parcial 1--1C2026--1900--Valeria y Analia-----.pdf` (y su copia en `Parciales/drive-download-20261111T183304Z-1-001/2. [Parcial 1][1C2026][1900][Valeria y Analia][][].pdf`).

3.1. Enunciado Original Verificado y Elementos Impresos en la Hoja del Examen

1. **Dominio del Problema (`[EXIGENCIA EXPLÍCITA DEL ENUNCIADO]`):**
   - **Ingreso de mascotas encontradas (2 canales):**

- *Canal Presencial:* Se ingresa al animal físicamente en un `Refugio` (validando capacidad/ubicación del refugio).
- *Canal Autogestionado:* Un ciudadano reporta el hallazgo y aloja temporalmente a la mascota en su domicilio (`HogarTemporal`), registrando la `UbicacionHallazgo` (calles de referencia, zona, coordenadas GPS), fotos y características físicas.
- **Módulo de Reclamo ("Soy el dueño"):**
- Cuando un usuario reclama a una mascota publicada presionando el botón *"Soy el dueño"*, el sistema dispara una **alarma inmediata**, cambia el estado del `Animal` de `"Perdido"` a `"En Proceso de Reclamo"`, registra el `Reclamo` (con su evaluación de pruebas de tenencia) y agenda un `Encuentro` presencial para la restitución.
- **Alerta de Reclamos Duplicados (`Conflicto`):**
- Si un **segundo usuario** reclama la tenencia del mismo animal mientras ya existe un reclamo activo, el sistema genera automáticamente un `Conflicto`, **suspende el** `Encuentro` pactado, cambia el estado del `Animal` a `"En Disputa Legal"` y **bloquea la agenda** para ese animal hasta que la administración resuelva la disputa.
- **Alerta sobre Encuentro de Animales Exóticos (`AnimalExotico`):**
- Si un ciudadano avista un animal exótico o peligroso en la vía pública, realiza una `Denuncia` de `AnimalExotico` indicando descripción, `UbicacionHallazgo` y evidencia (fotos/video).
- El sistema calcula el **nivel de peligrosidad (`ROJO`, `NARANJA`, `AMARILLO`, `VERDE`)** y **notifica automáticamente a dos organismos gubernamentales externos**: la **Dirección de Flora y Fauna** (`organismoFaunaFloraBsAs`) y la **Policía Ecológica**.
- **Clases esenciales mencionadas explícitamente en la consigna:** `Animal`, `Denuncia`, `Conflicto`, `Encuentro`, `AnimalExotico`, `UbicacionHallazgo`, `OrganismoExternos`, `organismoFaunaFloraBsAs`.
- **Diagrama de Componentes Base impreso en la hoja:** Muestra 5 bloques (`mascota`, `adminRefugio`, `DenunciaExotico`, `reclamo` [con sub-bloque `evaluacion`], `encuentro`).

2. **Lo que pide la Consigna (`Se Pide`):**
- **Punto 1:** Construir el **Diagrama de Clases completo** del sistema **usando solo 4 controladores**, justificando los patrones GRASP aplicados.
- **Punto 2: Modificar el Diagrama de Componentes** impreso en la hoja para adecuarlo a la limitación de los **4 controladores**.
- **Punto 3:** Construir el **Diagrama de Secuencia** para el **Escenario: Denuncia de Animal Exótico** (Un ciudadano ve un **Leopardo** en **Rivadavia, Morón, Bs.As.**), cumpliendo los **7 pasos impresos**:
- Recibir la denuncia con los datos y evidencia (foto/video).
- Registrar el animal exótico, ubicación, fecha y hora.
- Calcular el nivel de peligrosidad (`ROJO / NARANJA / AMARILLO / VERDE`).
- Notificar automáticamente a: **Dirección de Flora y Fauna** y **Policía Ecológica** *(nota impresa:* `// recomendable usar un Patron GOF`*)*.
- Marcar la denuncia como `organismoNotificado = SI` (`true`).
- Guardar la denuncia.
- Informar al ciudadano.
- **Punto 4:** Aplicar y justificar **2 patrones GoF de diferente categoría** (**prohibido usar Singleton y Facade**).

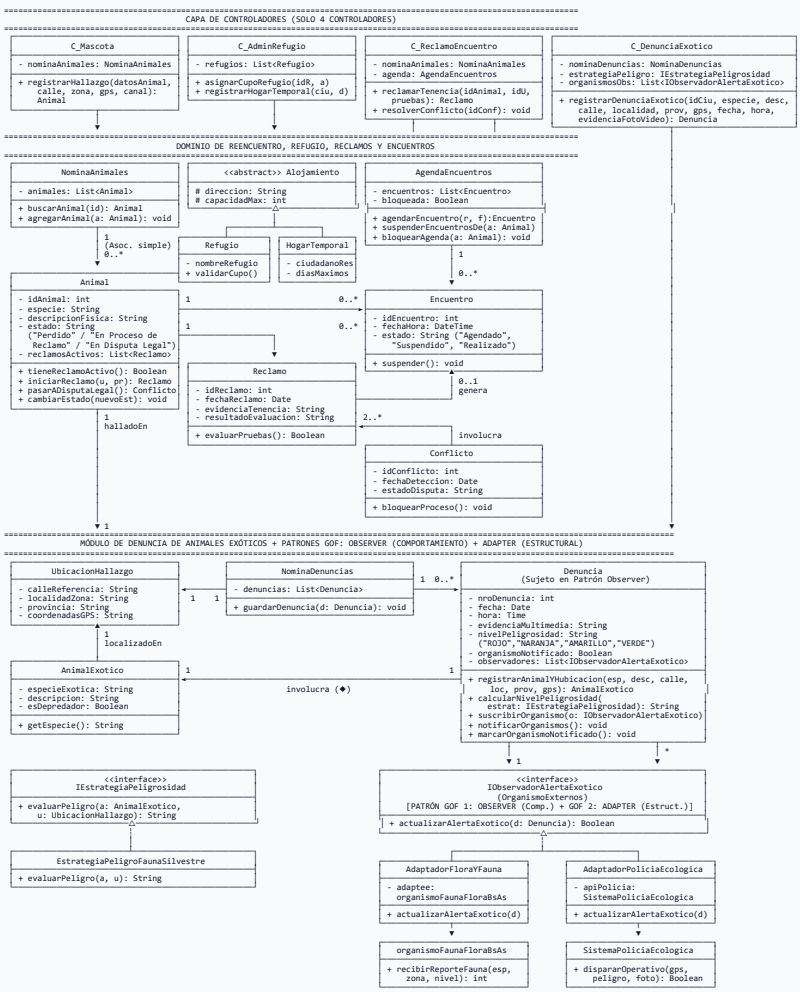
PUNTO 1 (REENCUENTRO, ADOPCIÓN Y ANIMALES EXÓTICOS): DIAGRAMA DE CLASES (SOLO 4 CONTROLADORES) + GRASP

1.A. Diferenciación entre Exigencias Explícitas y Decisiones de Diseño

- `[EXIGENCIA EXPLÍCITA DEL ENUNCIADO]`**:** Tope estricto de **4 controladores**, clases `Animal`, `Denuncia`, `Conflicto`, `Encuentro`, `AnimalExotico`, `UbicacionHallazgo`, `OrganismoExternos`, `organismoFaunaFloraBsAs`, estados de `Animal` (`Perdido`, `En Proceso de Reclamo`, `En Disputa Legal`), cálculo de peligrosidad (`ROJO/NARANJA/AMARILLO/VERDE`), atributo `organismoNotificado` en `Denuncia`, y prohibición de `Singleton` y `Facade`.
- `[DECISIÓN DE DISEÑO / INTERPRETACIÓN FUNDAMENTADA]`**:**
  - **Unificación en exactamente 4 Controladores** (que se corresponden 1 a 1 con el Diagrama de Componentes modificado del Punto 2):
  - `C_Mascota`: Gestiona el alta y búsqueda de mascotas halladas o en adopción.
  - `C_AdminRefugio`: Gestiona la administración de `Refugio`s y `HogarTemporal`es (capacidad, asignación de alojamiento).
  - `C_ReclamoEncuentro`: Unifica la recepción de `Reclamo` ("Soy el dueño"), la detección de reclamos duplicados (`Conflicto`), la evaluación y la gestión de la agenda de `Encuentro` (fusionando `reclamo` y `encuentro` del diagrama original de 5 bloques para cumplir los **4 controladores** con Alta Cohesión).
  - `C_DenunciaExotico`: Coordina el flujo completo de denuncias de `AnimalExotico`, cálculo de peligrosidad y notificación a organismos externos.
  - **Los 2 Patrones GoF integrados (sin usar Singleton ni Facade):**
  - **Patrón GoF 1 (Comportamiento) —** `Observer`**:** Sugerido en el paso 4 del enunciado. `Denuncia` actúa como **Sujeto Observado** que notifica automáticamente a todos los organismos suscritos a través de `<<interface>> IObservadorAlertaExotico`.
  - **Patrón GoF 2 (Estructural) —** `Adapter`**:** Como la **Dirección de Flora y Fauna** (`organismoFaunaFloraBsAs`) y la **Policía Ecológica** (`SistemaPoliciaEcologica`) son organismos externos con interfaces heterogéneas, se crean `AdaptadorFloraYFauna` y `AdaptadorPoliciaEcologica` que implementan `IObservadorAlertaExotico` (y `OrganismoExternos`) y traducen la alerta al formato nativo de cada entidad gubernamental. *(Además, se modela el patrón* `State` *sobre* `Animal` *o* `Strategy` *para el cálculo de peligrosidad).*

1.B. Diagrama de Clases Completo (`Reencuentro y Animales Exóticos` — Com. 1900)

DIAGRAMA / ESTRUCTURA TÉCNICA UML



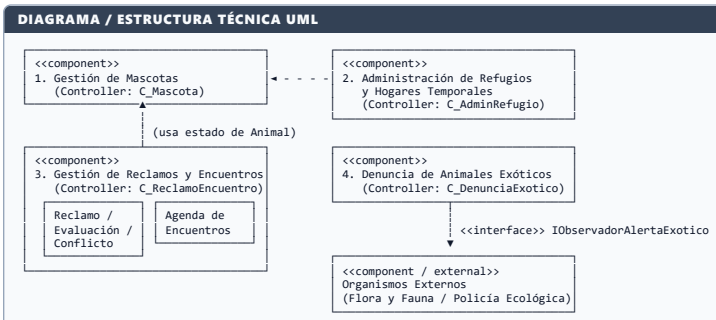
1.C. Justificación de los Patrones GRASP (`Reencuentro y Animales Exóticos`)

1. **Controlador (*Controller*):** Se aplica en los 4 controladores `C_Mascota`, `C_AdminRefugio`, `C_ReclamoEncuentro` y `C_DenunciaExotico`, cumpliendo la restricción de usar solo 4 controladores. Por ejemplo, `C_DenunciaExotico` captura el evento de entrada `+registrarDenunciaExotico(...)` y coordina los 7 pasos del caso de uso sin realizar él mismo la lógica interna del dominio.
2. **Creador (*Creator*):** Se aplica en `Denuncia` mediante el método `+registrarAnimalYHubicacion(...)`, el cual instancia (`create`) a `AnimalExotico` y a `UbicacionHallazgo`, ya que la `Denuncia` contiene de manera exclusiva los datos del avistamiento denunciado y recibe todos sus parámetros de inicialización. Asimismo, `Animal` crea a `Conflicto` en `+pasarADisputaLegal()` cuando detecta que ya poseía un reclamo activo.
3. **Experto en Información (*Information Expert*):** Se aplica en `Denuncia` con los métodos `+calcularNivelPeligrosidad(...)` y `+marcarOrganismoNotificado()`, dado que `Denuncia` conoce al `AnimalExotico`, su `UbicacionHallazgo` y su propio atributo `-organismoNotificado`.

PUNTO 2 (REENCUENTRO Y ANIMALES EXÓTICOS): MODIFICACIÓN DEL DIAGRAMA DE COMPONENTES ACORDE A LOS 4 CONTROLADORES

- **Problema del Diagrama de Componentes Base impreso en el parcial:** El diagrama impreso tenía **5 componentes separados** (`mascota`, `adminRefugio`, `DenunciaExotico`, `reclamo` [con `evaluacion`] y `encuentro`) y además no mostraba explícitamente el componente de integración con los `Organismos Externos`. Si cada componente principal tiene su controlador, 5 componentes implicarían 5 controladores, violando el tope de **4 controladores**.
- **Modificación aplicada (Alineación 1 a 1 con los 4 Controladores + Alta Cohesión):**
  - Se fusionan `reclamo` **(con** `evaluacion` **)** y `encuentro` dentro de un único componente cohesivo llamado `Gestión de Reclamos y Encuentros` (controlado por `C_ReclamoEncuentro`), ya que el `Encuentro` y el bloqueo por `Conflicto` son consecuencia directa del flujo de un `Reclamo`.
  - Quedan exactamente **4 Componentes de Dominio** (1 por cada uno de los 4 Controladores) más la interfaz hacia los **Organismos Externos**:

DIAGRAMA / ESTRUCTURA TÉCNICA UML



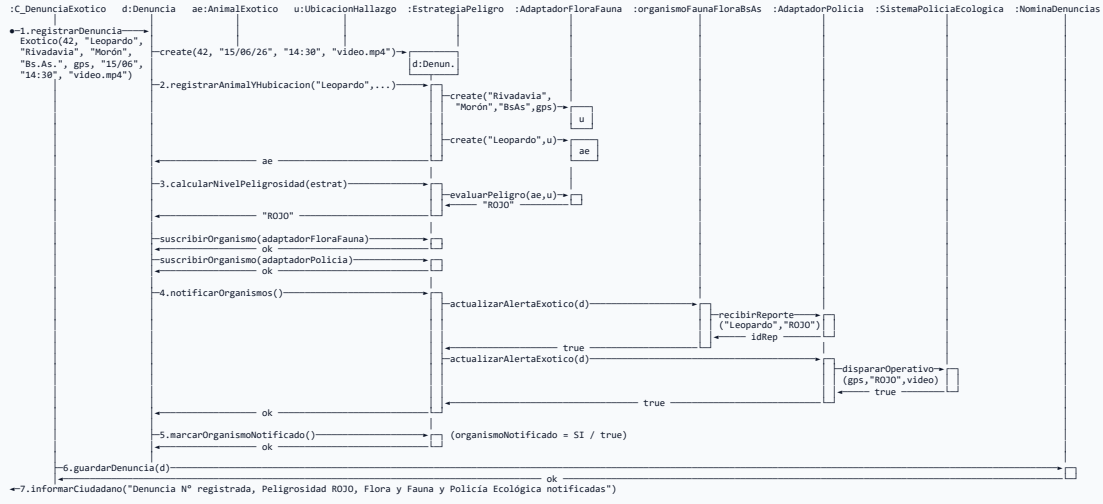
PUNTO 3 (REENCUENTRO Y ANIMALES EXÓTICOS): DIAGRAMA DE SECUENCIA — ESCENARIO DENUNCIA DE ANIMAL EXÓTICO (LEOPARDO EN MORÓN)

3.A. Escenario Concreto (Datos Exactos del Enunciado Impreso)

> 📌 **ESCENARIO CONCRETO / CONSIGNA DE PARCIAL**
>
> **Escenario:** Un ciudadano (`idCiudadano = 42`) avista un `"Leopardo"` suelto en la calle `"Rivadavia"`, localidad de `"Morón"`, provincia de `"Buenos Aires"` (`gps = "-34.65, -58.62"`), en la fecha `"15/06/2026"` a las `"14:30"` hs, adjuntando evidencia `"video_leopardo.mp4"`. El sistema registra la denuncia, el animal exótico y su ubicación, calcula el nivel de peligrosidad resultando `"ROJO"`, notifica automáticamente a la **Dirección de Flora y Fauna** y a la **Policía Ecológica**, marca `organismoNotificado = true`, guarda la denuncia en la nómina e informa al ciudadano.

3.B. Diagrama de Secuencia Completo (Los 7 Pasos Exactos del Enunciado)

DIAGRAMA / ESTRUCTURA TÉCNICA UML



PUNTO 4 (REENCUENTRO Y ANIMALES EXÓTICOS): 2 PATRONES GOF DE DIFERENTE CATEGORÍA (SIN SINGLETON NI FACADE)

Patrón GoF N° 1: `Observer` (Categoría: Comportamiento)

- **a) Categoría y Nombre:** Patrón de **Comportamiento** — **Observer (Observador)**.
- **b) Problema de diseño que resuelve en el caso:** En el paso 4 de la denuncia de un `AnimalExotico`, el sistema debe disparar notificaciones automáticas simultáneas a múltiples organismos estatales (**Dirección de Flora y Fauna** y **Policía Ecológica**, y potencialmente Defensa Civil o Zoonosis en el futuro) sin acoplar la clase `Denuncia` a una lista fija de organismos concretos.
- **c) Por qué es adecuado:** Define una relación uno-a-muchos entre el Sujeto (`Denuncia`) y la interfaz `<<interface>> IObservadorAlertaExotico`, de modo que cuando se invoca `+notificarOrganismos()`, la denuncia recorre su colección polimórfica de observadores suscritos enviándoles `+actualizarAlertaExotico(this)`.
- **d) Clases reutilizadas y nuevas:**
  - *Clase reutilizada (`Subject`):* `Denuncia`, con `-observadores: List<IObservadorAlertaExotico>`, `+suscribirOrganismo(o)` y `+notificarOrganismos()`.
  - *Interfaz nueva (`Observer`):* `<<interface>> IObservadorAlertaExotico` (que representa el contrato `OrganismoExternos`).
  - *Observadores concretos:* `AdaptadorFloraYFauna` y `AdaptadorPoliciaEcologica`.

Patrón GoF N° 2: `Adapter` (Categoría: Estructural)

- **a) Categoría y Nombre:** Patrón **Estructural** — **Adapter (Adaptador)**.
- **b) Problema de diseño que resuelve en el caso:** Los sistemas informáticos de la **Dirección de Flora y Fauna de Bs.As.** (`organismoFaunaFloraBsAs`) y de la **Policía Ecológica** (`SistemaPoliciaEcologica`) son sistemas externos gubernamentales preexistentes con firmas de métodos y protocolos distintos e incompatibles con el método uniforme `+actualizarAlertaExotico(d: Denuncia)` que invoca nuestro dominio.
- **c) Por qué es adecuado:** Cada clase adaptadora implementa la interfaz esperada por nuestro dominio (`IObservadorAlertaExotico`) y mantiene una asociación hacia el sistema externo real (`organismoFaunaFloraBsAs` y `SistemaPoliciaEcologica`), traduciendo los datos de la `Denuncia` a los parámetros específicos de cada organismo sin violar la restricción de no usar `Facade`.
- **d) Clases reutilizadas y nuevas:**
  - *Interfaz Objetivo (`Target`):* `<<interface>> IObservadorAlertaExotico`.
  - *Clases nuevas (`Adapters`):* `AdaptadorFloraYFauna` y `AdaptadorPoliciaEcologica`.
  - *Clases adaptadas (`Adaptees`):* `organismoFaunaFloraBsAs` (provista en el enunciado) y `SistemaPoliciaEcologica`.

PARTE 4: RESOLUCIÓN COMPLETA DEL PARCIAL 2C 2026 (COMISIÓN 1900 — TEMA 1)

CASO: "AGENCIA DE VIAJES" (`Parciales/drive-download-20261111T183304Z-1-001/3. [Parcial 1][2C2026][1900][][][].pdf`)

> 🎓 **CRITERIO Y RECOMENDACIÓN DE LA PROFESORA**
>
> **Fuente verificada visualmente:** Escaneo original de 2 páginas de `Parciales/drive-download-20261111T183304Z-1-001/3. [Parcial 1][2C2026][1900][][][].pdf` (y sus copias en `Parciales/1.jpeg` y `2.jpeg`).

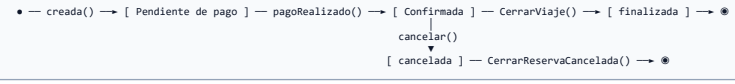
4.1. Enunciado Original Verificado y Elementos Impresos en la Hoja del Examen (`TEMA 1`)

1. **Dominio del Problema (`[EXIGENCIA EXPLÍCITA DEL ENUNCIADO]`):**
   - Una **Agencia de Viajes** ofrece la contratación de `PaqueteTuristico`**s** (ej. Bariloche, Mendoza, Cataratas). Cada paquete tiene un **único destino**, **duración**, **fecha de vigencia** y un **hotel base**, e incluye servicios de:

- `Transporte` (avión, micro, etc.),
- `Alojamiento` (hotel, apart, cabaña),
- `Excursion` **es**.
- **Flujo del Caso de Uso impreso en el examen:**
- El `Cliente` busca paquetes disponibles filtrando por **destino, fechas y cantidad de pasajeros**.
- El cliente selecciona un `PaqueteTuristico` y opcionalmente puede **cambiar el hotel** y **agregar o quitar excursiones**.
- El sistema **verifica la disponibilidad** de los servicios (consultando sistemas externos de aerolínea y hotel) y **calcula el precio total de contratación** (el cual varía según **cantidad de pasajeros, fecha del viaje, servicios incluidos y excursiones extras**).
- El cliente selecciona el **medio de pago** (**tarjeta de crédito, tarjeta de débito o transferencia**) y realiza el `Pago`.
- Si el pago es aprobado por el sistema externo de pago, se genera/confirma la `Reserva` y se **envía notificación por mail**.
- Una reserva confirmada puede **cancelarse** posteriormente gestionando el **reintegro/reembolso** según corresponda.

2. **Diagramas Impresos en la Hoja del Examen (`[EXIGENCIA EXPLÍCITA DEL ENUNCIADO]`):**
   - **Diagrama de Componentes Defectuoso (para analizar en el Punto 1):** Muestra un gran bloque monolítico llamado `Solucion` que encierra acopladamente a `paquete/servicio`, `reserva`, `pago` y `cliente`, mientras afuera aparecen sueltos `notificaciones`, `Sist Ext. Aerolinea`, `Sist Externo Pago` y `Sist Ext. Hotel`.
   - **Diagrama de Estados de `Reserva` (para integrar en el Punto 3):**

DIAGRAMA / ESTRUCTURA TÉCNICA UML



1. **Lo que pide la Consigna (`TEMA 1 - Se Pide`):**
   - **Punto 1: Analizar el Diagrama de Componentes** dado y **proponer mejoras** considerando **cohesión, acoplamiento, separación de responsabilidades y mantenibilidad**.
   - **Punto 2:** Realizar el **Diagrama de Secuencia para `"realizar Pago"`** basándose desde el **paso 4 en adelante, MEJORANDO EL ESCENARIO**.
   - **Punto 3: Integrar el Diagrama de Estados de `Reserva`** provisto dentro del modelo.
   - **Punto 4:** Construir el **Diagrama de Clases completo usando solo 4 controladores**, aplicando dos patrones GoF:
   - **A-** La complejidad del **cálculo del precio de contratación según diferentes políticas**.
   - **B-** Otro problema de diseño que identifique en el sistema.

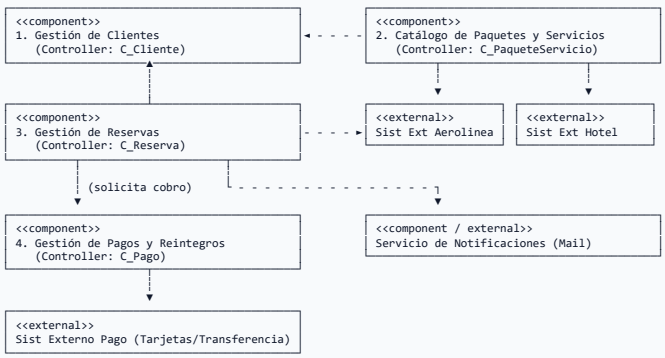
PUNTO 1 (AGENCIA DE VIAJES — TEMA 1): ANÁLISIS CRÍTICO Y MEJORA DEL DIAGRAMA DE COMPONENTES

1.A. Problemas Detectados en el Diagrama de Componentes Original Impreso

1. **Baja Cohesión (Bloque Monolítico `"Solucion"`):** El diagrama original mete dentro de un único macro-componente `Solucion` módulos con responsabilidades completamente distintas (`cliente`, `paquete/servicio`, `reserva`, `pago`), violando el principio de **Cohesión Funcional** y los criterios de **Descomponibilidad y Comprensibilidad Modular**.
2. **Alto Acoplamiento e Inconsistencia de Dependencias:** En el dibujo original, `cliente` depende de `pago` y `reserva` de forma cruzada y cíclica, y las conexiones hacia los sistemas externos (`Sist Ext. Aerolinea`, `Sist Ext. Hotel`, `Sist Externo Pago`) y `notificaciones` no están mediadas por interfaces claras ni asignadas al componente responsable.
3. **Falta de Alineación con los 4 Controladores:** Para garantizar la trazabilidad entre la Vista de Implementación (Componentes) y la Vista Lógica (Clases con **4 controladores**), el sistema debe dividirse en **4 componentes internos cohesivos** más los componentes de sistemas externos conectados mediante interfaces provistas/requeridas.

1.B. Diagrama de Componentes Mejorado (4 Componentes Internos Cohesivos + Sistemas Externos Desacoplados)

DIAGRAMA / ESTRUCTURA TÉCNICA UML



PUNTOS 3 Y 4 (AGENCIA DE VIAJES — TEMA 1): INTEGRACIÓN DEL DIAGRAMA DE ESTADOS DE `Reserva` Y DIAGRAMA DE CLASES COMPLETO (SOLO 4 CONTROLADORES + 2 PATRONES GOF `A` Y `B`)

4.A. Resolución de los Patrones GoF Pedidos (`A` y `B`) e Integración del Diagrama de Estados

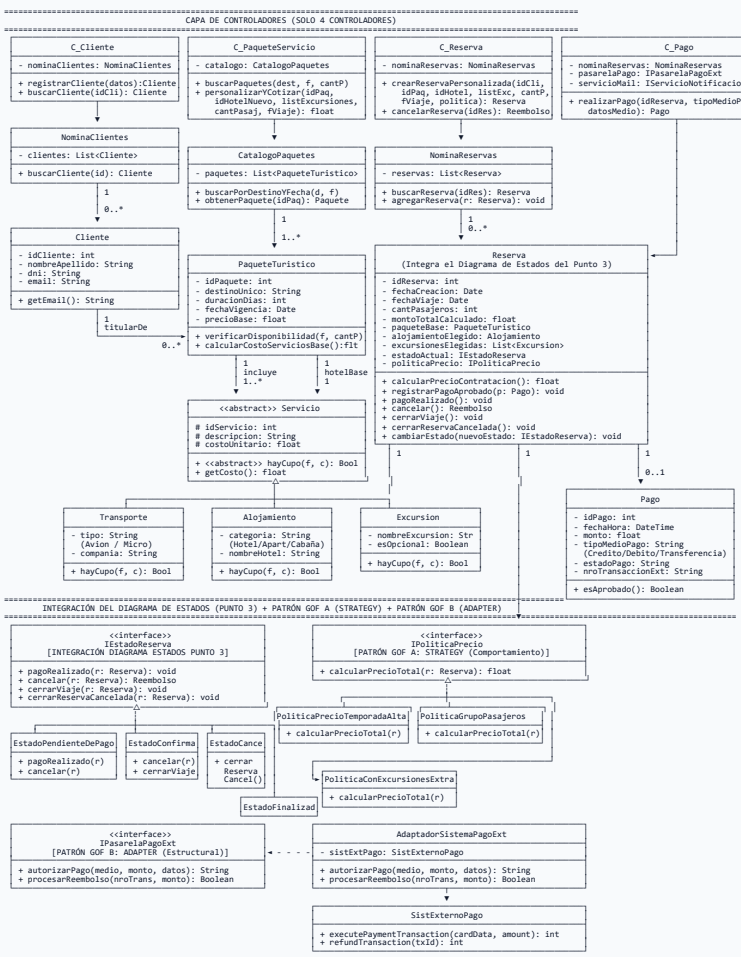
- **Integración del Diagrama de Estados de `Reserva` (Punto 3):**
  - Se incorporan en la clase `Reserva` todos los métodos correspondientes a las transiciones del Diagrama de Estados impreso: constructor `+crearReserva()` (estado inicial `Pendiente de pago`), `+pagoRealizado()` (transiciona a `Confirmada`), `+cancelar()` (transiciona a `Cancelada` y dispara el cálculo de reembolso), `+cerrarViaje()` (transiciona a `Finalizada`) y `+cerrarReservaCancelada()`.
  - Además, para modelar el cambio de comportamiento por estado sin condicionales `if/switch`, se aplica el patrón `State` (`<<interface>> IEstadoReserva` con las 4 clases de estado: `EstadoPendienteDePago`, `EstadoConfirmada`, `EstadoCancelada`, `EstadoFinalizada`).
- **Patrón GoF `A` (Complejidad del cálculo del precio de contratación según diferentes políticas):**
  - **Categoría y Nombre:** Patrón de **Comportamiento —** `Strategy` **(Estrategia)**.
  - **Problema que resuelve:** El precio total de contratación varía según múltiples políticas (temporada/fecha del viaje, cantidad de pasajeros/grupo familiar, paquete con excursiones extras).

- **Diseño:** La clase `Reserva` (o `ConfiguracionPaquete`) se asocia a la interfaz `<<interface>> IPoliticaPrecio` (`+calcularPrecioTotal(paqBase, cantPasajeros, fechaViaje, hotelElegido, excursionesExtras): float`), implementada por `PoliticaPrecioTemporadaAlta`, `PoliticaPrecioGrupoFamiliar` y `PoliticaPrecioPromocional`.

- **Patrón GoF `B` (Segundo problema de diseño en el sistema — De diferente categoría: Estructural):**
  - **Categoría y Nombre:** Patrón **Estructural — `Adapter` (Adaptador)** *(Nota: También puede combinarse con `Decorator` o `Composite` para los servicios del paquete).*
  - **Problema que resuelve:** El sistema necesita comunicarse con `Sist Externo Pago`, `Sist Ext. Aerolinea` y `Sist Ext. Hotel`, cuyas APIs externas son incompatibles con el modelo de dominio.
  - **Diseño:** Para el proceso de pago y reintegro se define la interfaz `<<interface>> IPasarelaPagoExt` (`+procesarCobro(medio, monto, datos): Boolean`, `+ejecutarReintegro(idTransaccion, monto): Boolean`), implementada por `AdaptadorSistemaPagoExt` que envuelve y adapta a la clase externa `SistExternoPago` (y análogamente `IServicioProveedorExt` con `AdaptadorAerolineaExt` y `AdaptadorHotelExt`).

4.B. Diagrama de Clases Completo (`Agencia de Viajes` — 4 Controladores + Estados de `Reserva` + GoF `A` Strategy + GoF `B` Adapter)

DIAGRAMA / ESTRUCTURA TÉCNICA UML



PUNTO 2 (AGENCIA DE VIAJES — TEMA 1): ESCENARIO MEJORADO Y DIAGRAMA DE SECUENCIA PARA `"realizar Pago"` (PASO 4 EN ADELANTE)

2.A. Escenario Mejorado (A partir del Paso 4 del Enunciado)

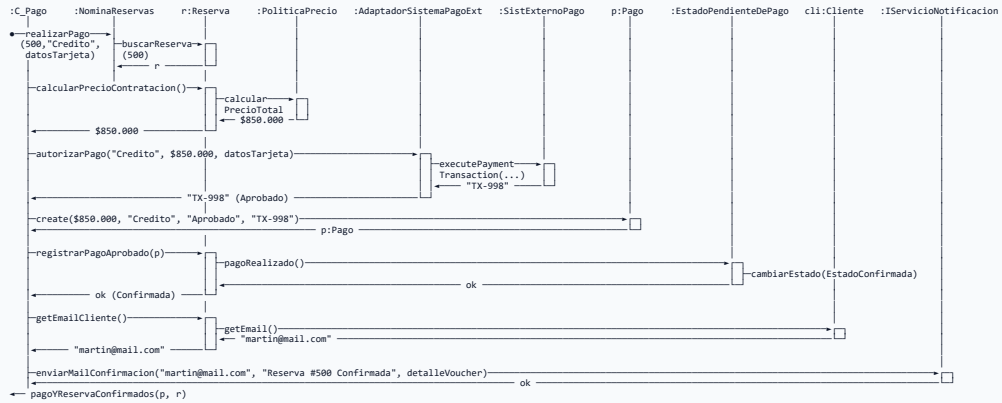
📌 ESCENARIO CONCRETO / CONSIGNA DE PARCIAL

Escenario Mejorado: El cliente `"Martín Gómez"` (`email = "martin@mail.com"`) ya seleccionó y personalizó el paquete a `"Bariloche"` para `2` pasajeros con `1` excursión extra, habiéndose creado la `Reserva` (`idReserva = 500`) en estado `"Pendiente de pago"`. En el **Paso 4**, el cliente selecciona como medio de pago `"Tarjeta de Crédito"` e ingresa los datos de su tarjeta. El controlador `C_Pago` recibe el mensaje `realizarPago(500, "TarjetaCredito", datosTarjeta)`, le pide a la `Reserva` su precio final calculado mediante su `IPoliticaPrecio`, solicita la autorización del cobro a través de `AdaptadorSistemaPagoExt` hacia `SistExternoPago`, y al recibir la aprobación (`txId = "TX-998"`):

1. Instancia el objeto `Pago` aprobado y lo asocia a la `Reserva`.
2. Dispara la transición `pagoRealizado()` en `Reserva`, pasando su estado de `EstadoPendienteDePago` a `EstadoConfirmada`.
3. Obtiene el email del `Cliente` titular y envía la notificación de confirmación por mail a través de `IServicioNotificacion`.

2.B. Diagrama de Secuencia Completo de `"realizar Pago"`

DIAGRAMA / ESTRUCTURA TÉCNICA UML



GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 8: TABLAS COMPARATIVAS Y SÍNTESIS DE UNIDADES TEÓRICAS FINALES

📑 **ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO**

8.0. MATRICES Y TABLAS COMPARATIVAS RÁPIDAS PARA EL EXAMEN Pág. 52
Tabla Comparativa 1: Diferencias entre los 8 Diagramas UML de la Materia Pág. 52
Tabla Comparativa 2: Desempate Rápido entre Patrones GoF Parecidos (Pregunta Fija de Teoría y Práctica) Pág. 52
PARTE 6: SÍNTESIS DE LAS UNIDADES TEÓRICAS FINALES Y BANCO DE PREGUNTAS DE EXAMEN Pág. 53
6.1. Calidad de Software y Atributos de Calidad (02-Diseño Calidad- UNLaM-2026Agosto.pdf) Pág. 53
6.2. Diseño de Interfaz de Usuario y UX (02-DiseñoUX-Alumno-2Q-2026-JUEVES.pdf) Pág. 53
6.3. Arquitectura Orientada a Servicios (SOA), Microservicios y APIs (11/soa 2026-v10.pdf — Clase 11) Pág. 53
6.4. Pruebas de Software / Testing (11/Testing UNLaM.ppt — Clase 11 y Parcial 2023) Pág. 53
6.5. Sistemas de Tiempo Real (STR) (11/str-alumno.pdf — Clase 11) Pág. 53
6.6. Solucionario Oficial Completo de Clases/Clases/11/preguntasGuiaAutoEstudio-2.docx (Arquitectura Web, HTTP, Servidores, HTML y APIs) Pág. 54

8.0. MATRICES Y TABLAS COMPARATIVAS RÁPIDAS PARA EL EXAMEN

Tabla Comparativa 1: Diferencias entre los 8 Diagramas UML de la Materia

Diagrama UML	Vista de Kruchten	Tipo	¿Qué modela exactamente?	Elemento Central y Regla de Oro de Analía Marín
1. Diagrama de Clases	Vista Lógica / Diseño	Estático / Estructural	Clases, atributos, métodos, visibilidades y relaciones (`——` , `◇` , `◆` , `——▷` , `- - -▷` , `- - ->`).	Arriba los `Controllers` (máx. 4), debajo `Nomina` (con asoc. simple) y Dominio + Patrones GoF. Prohibido poner ABMs triviales o getters/setters masivos.
2. Diagrama de Secuencia	Vista de Procesos / Lógica	Dinámico / Interacción	Orden temporal (vertical) del intercambio de mensajes entre objetos para un escenario puntual.	Escribir **Escenario concreto** arriba. **SIN Actor humano** (entra directo al `:C_Controller`). Todo mensaje `m()` hacia `:B` debe ser un método `+m()` en la clase `B` .
3. Diagrama de Comunicación	Vista de Procesos / Lógica	Dinámico / Interacción	Vínculos estructurales (grafo de enlaces) entre objetos que colaboran, con numeración de mensajes (`1` , `1.1` , `1.2`).	Es **semánticamente equivalente (isomorfo)** al Diagrama de Secuencia, pero enfatiza las conexiones entre objetos en vez del eje vertical de tiempo.
4. Diagrama de Estados	Vista Lógica / Diseño	Dinámico / Comportamiento	Ciclo de vida de **UN ÚNICO OBJETO** de una clase reactiva (estados, eventos, guardas y acciones).	Estados = **Adjetivos/Participios** (`Pendiente` , `Confirmada`). Flechas = `evento() [guarda] / acción` . Todos los eventos deben ser métodos de esa clase en el DC.
5. Diagrama de Actividad	Vista de Procesos	Dinámico / Flujo	Flujo de trabajo (*workflow*) paso a paso, decisiones (`◇`) y concurrencia en paralelo (barras **Fork/Join**).	Dividido en carriles (*Swimlanes*) por responsable. Las acciones llevan **Verbo + Objeto**.
6. Diagrama de Casos de Uso	Vista +1 (Escenarios)	Funcional / Requerimientos	Interacción entre Actores externos y los Casos de Uso dentro del límite del sistema (`<<include>>` , `<<extend>>`).	Especificación en **Tabla de 2 Columnas (`Acción del Actor` \ `Respuesta del Sistema`)** + Flujos Alternativos numerados (`2a` , `4a`).
7. Diagrama de Componentes	Vista de Implementación / Desarrollo	Estructural / Modular	Descomposición del software en módulos/subsistemas e interfaces provistas (`—○`) y requeridas (`—(`) o dependencias (`- - ->`).	**Regla de Trazabilidad:** Cada Componente interno principal del negocio se corresponde 1 a 1 con un `Controller` (`C_...`) en el Diagrama de Clases.
8. Diagrama de Despliegue	Vista Física / Despliegue	Estructural / Físico	Topología física de Hardware (**Nodos 3D**), entornos de ejecución, artefactos desplegados y red.	Separar por **Dominios/Ambientes de Red** y rotular **SIEMPRE el protocolo** (`HTTPS` , `REST/JSON` , `SOAP/XML` , `TCP/IP` , `JDBC` , `USB` , `MQTT`) en cada línea.

Tabla Comparativa 2: Desempate Rápido entre Patrones GoF Parecidos (Pregunta Fija de Teoría y Práctica)

Par de Patrones GoF	¿En qué se parecen?	Diferencia Exacta para No Confundirlos en el Parcial
`Strategy` vs. `State`	Ambos son de **Comportamiento** y tienen **exactamente la misma estructura UML** (`Contexto` → `<<interface>>` ← Clases concretas).	- `Strategy` encapsula **algoritmos intercambiables** para hacer una misma tarea (ej. calcular precio por Temporada Alta vs. Grupo Familiar). El cliente suele elegir la estrategia. - `State` encapsula el **comportamiento dependiente del estado del ciclo de vida** del objeto (`Pendiente` → `Confirmada` → `Cancelada`), y el propio estado dispara la transición al estado siguiente.
`Adapter` vs. `Facade`	Ambos son **Estructurales** y sirven como envoltorios/intermediarios (**Indirección**), muy usados frente a sistemas externos.	- `Adapter` **convierte una interfaz incompatible** en otra interfaz específica que el cliente espera (traductor 1 a 1; **permitido y recomendado en todos los parciales 2026**). - `Facade` **simplifica y unifica** el acceso a un subsistema complejo con muchas clases mediante una interfaz de alto nivel (*Caso Banco Olivares*, pero **restringido en Parciales 2026**).
`Adapter` vs. `Proxy`	Ambos envuelven a un objeto y delegan la llamada.	- `Adapter` cambia la interfaz porque el objeto adaptado tiene una firma incompatible. - `Proxy` implementa exactamente la **misma interfaz** que el objeto real para **controlar el acceso** (permisos, carga diferida de videos/imágenes pesadas, caché).
`Composite` vs. `Decorator`	Ambos son **Estructurales** y usan composición recursiva sobre una misma interfaz común.	- `Composite` representa una **jerarquía de árbol Todo-Parte** (`Paquete` compuesto por `Servicios` hijos) para tratar igual a la hoja y al compuesto. - `Decorator` tiene un solo componente envuelto (`1 a 1`) para **apilar responsabilidades o adicionales dinámicamente** en tiempo de ejecución.
`Factory Method` vs. `Abstract Factory` vs. `Builder`	Los tres son **Creacionales**.	- `Factory Method` crea **un producto** delegando la instanciación a subclases mediante un método sobrescrito. - `Abstract Factory` crea **familias enteras de productos relacionados** sin especificar sus clases concretas. - `Builder` construye **paso a paso un único objeto complejo** con múltiples partes opcionales (`agregarTransporte()` , `agregarHotel()` , `obtenerResultado()`).

PARTE 6: SÍNTESIS DE LAS UNIDADES TEÓRICAS FINALES Y BANCO DE PREGUNTAS DE EXAMEN

Para asegurar el 100% de los puntos en las preguntas teóricas del Parcial y del Final, aquí tenés el desarrollo exacto según las diapositivas de la Prof. Analía Marín:

6.1. Calidad de Software y Atributos de Calidad (`02-Diseño Calidad- UNLaM-2026Agosto.pdf`)

1. **David Garvin (Dimensiones de la Calidad, Pág. 7):** Desempeño, Características, Confiabilidad, Conformidad, Durabilidad, Servicio, Estética y Percepción.
2. **Robert Glass (Calidad del Diseño, Pág. 8):** Es el grado en que el diseño cumple las funciones y características especificadas en el modelo de requerimientos.
3. **Factores de Calidad de McCall (Págs. 9–10 — Pregunta del Final Julio 2026 Mesa B):** Se agrupan en **3 ejes**:
   - **Operación del Producto:** *Corrección* (cumple especificaciones), *Confiabilidad* (precisión sin fallas), *Eficiencia* (uso de recursos), *Integridad/Seguridad* (control de acceso), *Usabilidad* (facilidad de uso).
   - **Revisión del Producto:** *Facilidad de Mantenimiento* (localizar y arreglar errores), *Flexibilidad* (modificar el sistema operativo), *Facilidad de Prueba* (validar el software).
   - **Transición del Producto:** *Portabilidad* (llevar a otro hardware/SO), *Reusabilidad* (reusar partes en otros sistemas), *Interoperabilidad* (conectarse con otros sistemas).
4. **Atributos de Calidad Clave para Evaluar Arquitecturas (Págs. 11 y 16 — Pregunta de Parcial 2023, Final Dic 2025 y Final Jul 2026):**
   - **Performance (Rendimiento):** Tiempo de respuesta y capacidad de procesamiento (*throughput*) bajo una carga determinada.
   - **Escalabilidad:** Capacidad de crecer para manejar un incremento brusco de usuarios o transacciones (ej. e-commerce en CyberMonday) agregando recursos (escalado horizontal o vertical) sin degradar el rendimiento.
   - **Disponibilidad (*Availability*):** Proporción de tiempo en que el sistema está operativo y accesible cuando se lo requiere (99.9%), estrechamente ligada a la **Tolerancia a Fallos** y **Confiabilidad**.
   - **Seguridad:** Capacidad de resistir accesos no autorizados garantizando Confidencialidad, Integridad y Autenticación.
   - **Mantenibilidad y Extensibilidad:** Facilidad para corregir defectos o agregar nueva funcionalidad con mínimo impacto (lograda con **Alta Cohesión, Bajo Acoplamiento y Patrones GoF**).

6.2. Diseño de Interfaz de Usuario y UX (`02-DiseñoUX-Alumno-2Q-2026-JUEVES.pdf`)

1. **Las 3 Reglas Doradas de Theo Mandel (Pág. 4 — Pregunta de Parcial 2C 2025):**
   - **Regla 1: Dejar el control al usuario:** No obligar al usuario a realizar acciones innecesarias, permitir que la interacción sea interrumpible y **reversible (Deshacer/Undo)**, y ocultar los detalles técnicos internos.
   - **Regla 2: Reducir la carga de memoria del usuario:** No obligar al usuario a recordar datos de una pantalla a otra; ofrecer valores por defecto coherentes, atajos intuitivos y metáforas del mundo real.
   - **Regla 3: Hacer que la interfaz sea consistente:** Mantener coherencia visual, de colores, tipografía, ubicación de botones y comportamiento en todas las pantallas para que la experiencia sea predecible.
2. **Herramientas de Investigación UX vistas en Clase 02 (Págs. 12–18):**
   - **User Personas:** Arquetipos o personajes ficticios basados en datos reales de usuarios para comprender sus objetivos, frustraciones, contexto y expectativas.
   - **Pruebas de Usabilidad (7 pasos):** Observar a usuarios reales ejecutando un guion de tareas sobre el prototipo sin ayudarlos, midiendo dónde dudan o fallan.
   - **Heat Maps (Mapas de Calor):** Representación visual de dónde hacen clic o fijan la vista los usuarios (zonas rojas = calientes/más vistas; zonas azules = frías/ignoradas).

6.3. Arquitectura Orientada a Servicios (SOA), Microservicios y APIs (`11/soa 2026-v10.pdf` — Clase 11)

1. **¿Qué es SOA? (Págs. 1–8):** Es un estilo arquitectónico que estructura los sistemas como un conjunto de **servicios de negocio débilmente acoplados, reutilizables e interoperables** que se comunican mediante protocolos estándar (históricamente SOAP/XML).
   - **ESB (*Enterprise Service Bus*, Págs. 4–5):** Middleware o bus de interconexión que implementa la **lógica de integración, enrutamiento, transformación de mensajes y orquestación** entre servicios. **Regla de Oro de la Profesora (Pág. 5):** "*El ESB implementa la lógica de integración de los servicios, NUNCA la lógica de negocios (la cual reside en los componentes)*".
   - **BPM (*Business Process Management*) y BAM (*Business Activity Monitoring*, Pág. 6):** BPM modela y orquesta los procesos de negocio de punta a punta; BAM monitorea en tiempo real los indicadores (KPIs) y alertas de esos procesos.
2. **SOA vs. Microservicios (Pág. 10):**
   - **Microservicios:** Evolución moderna donde la aplicación se divide en servicios **pequeños, autónomos y especializados**, cada uno con **su propia base de datos independiente** y desplegable por separado en contenedores, comunicándose mediante APIs ligeras (REST/JSON o colas de eventos) sin un ESB pesado centralizado.
3. **API y Swagger / OpenAPI (Págs. 11–12):**
   - **API (*Application Programming Interface*):** Contrato o puerta de entrada que permite que otros programas se conecten e intercambien datos (mediante verbos HTTP: `GET`, `POST`, `PUT`, `DELETE`).
   - **Swagger / OpenAPI:** Herramienta y especificación estándar que genera una página web interactiva documentando todos los endpoints de una API REST, sus parámetros de entrada, códigos de respuesta y botones para probar cada endpoint en vivo.

6.4. Pruebas de Software / Testing (`11/Testing UNLaM.ppt` — Clase 11 y Parcial 2023)

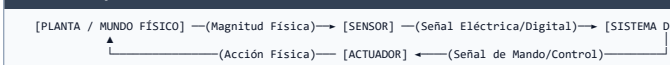
1. **Objetivo del Testing (Glenford Myers):** El testing es el proceso de ejecutar un programa con la intención de **encontrar errores**. Una buena prueba es aquella que tiene alta probabilidad de descubrir un error aún no descubierto (el testing demuestra la presencia de defectos, nunca su ausencia total).
2. **Verificación vs. Validación (Boehm):**
   - **Verificación:** "*¿Estamos construyendo el producto correctamente?*" (Compara contra la especificación técnica de diseño).
   - **Validación:** "*¿Estamos construyendo el producto correcto?*" (Compara contra las necesidades y expectativas reales del usuario/cliente).
3. **Estrategias y Niveles de Prueba (De menor a mayor alcance — Pregunta de Parcial 2023 y Recuperatorio 2023):**
   - **Pruebas Unitarias (*Unit Testing*):** Prueban el componente más pequeño del diseño (una clase o método individual) de forma aislada, generalmente aplicando **Caja Blanca** (conociendo la estructura interna del código, caminos lógicos, bucles y condiciones).
   - **Pruebas de Integración:** Prueban la interacción e interfaces entre módulos o componentes combinados (enfoque *Top-Down* usando *Stubs*, *Bottom-Up* usando *Drivers*, o *Regression Testing* / Prueba de Regresión para asegurar que un cambio nuevo no rompió lo que ya funcionaba, y *Smoke Testing* / Prueba de Humo).
   - **Pruebas de Validación / Aceptación:** Verifican los requerimientos funcionales del usuario mediante **Caja Negra** (evaluando entradas y salidas sobre la interfaz sin mirar el código interno; incluye pruebas **Alfa** en el entorno del desarrollador y pruebas **Beta** en el entorno real del cliente).
   - **Pruebas del Sistema:** Prueban el sistema completo integrado con el hardware y otros elementos (incluye **Pruebas de Recuperación** ante fallos, **Pruebas de Seguridad**, **Pruebas de Estrés / Esfuerzo** llevando el sistema al límite de carga, y **Pruebas de Rendimiento**).

6.5. Sistemas de Tiempo Real (STR) (`11/str-alumno.pdf` — Clase 11)

1. **¿Qué es un STR? (Págs. 2–3):** Es un sistema informático que interactúa fuertemente con un **ambiente físico real (la Planta)** donde la **validez del resultado depende NO SOLO de que la respuesta sea lógicamente correcta, sino también del INSTANTE DE TIEMPO en que se produce**.

2. **Características de un STR (Pág. 3):** Interacción con un proceso real irreversible ("el sistema no vuelve atrás"), cumplimiento de **restricciones temporales (*deadlines*)**, alta confiabilidad, concurrencia y manejo de estímulos periódicos y aperiódicos (asincrónicos).
3. **Clasificación de STR (Pág. 4):**
   - **Tiempo Real Estricto (*Hard Real Time*):** Es absolutamente crítico que la respuesta ocurra dentro del límite de tiempo; incumplir el *deadline* provoca una **catástrofe** con riesgo de vidas humanas o destrucción física (ej. control de vuelo de un avión, frenos ABS, airbag, reactor nuclear, marcapasos).
   - **Tiempo Real Blando / Flexible (*Soft Real Time*):** Se requiere respuesta rápida, pero superar ocasionalmente el límite de tiempo solo **degrada la calidad del servicio** sin causar una catástrofe (ej. streaming de video/audio, reservas online, domótica hogareña).
4. **Arquitectura de un STR: Sensores, Controlador y Actuadores (Págs. 5–12):**

DIAGRAMA / ESTRUCTURA TÉCNICA UML



- - **Sensores (Págs. 5–8):** Dispositivos de entrada que detectan una magnitud física del mundo real (temperatura, presión, luz, movimiento, huella) y la convierten en una señal eléctrica/digital para el controlador. Según su salida se clasifican en:
  - **Analógicos:** Entregan una señal continua variable en un rango (ej. termistor que varía voltaje de 0 V a 5 V).
  - **Digitales:** Entregan un valor numérico codificado en bits discretos.
  - **Lógicos (Todo o Nada / Binarios):** Solo tienen **dos estados posibles** (`1` o `0` , encendido/apagado, abierto/cerrado; ej. sensor magnético de puerta o detector de presencia simple).
  - **Actuadores (Pág. 10):** Dispositivos de salida que reciben la orden del sistema de control y **modifican físicamente el mundo real (la planta)** (ej. motor eléctrico, electroválvula, relé, sirena, destrabe de molinete).

6.6. Solucionario Oficial Completo de `Clases/Clases/11/preguntasGuiaAutoEstudio-2.docx` (Arquitectura Web, HTTP, Servidores, HTML y APIs)

En la carpeta `Clases/Clases/11/preguntasGuiaAutoEstudio-2.docx` (vinculada también al TP inicial `Clases/Clases/01/TP_HTML_JavaScript_Formularios_v2.docx`), la cátedra lista el temario integral (RUP, Metodologías Ágiles, Scrum, Kanban, Historias de Usuario, UX, UML, GoF, GRASP, Arquitectura, API/Endpoint, Testing, SOA, Calidad, STR) y formula **18 preguntas específicas de Arquitectura Web, Backend y Frontend**. Aquí están todas respondidas de forma precisa y directa para el examen:

1. **¿Qué responsabilidades tiene un navegador web (*Web Browser*)?**
   - Es el cliente HTTP encargado de: (1) enviar peticiones HTTP/HTTPS al servidor a partir de una URL o acción del usuario; (2) recibir la respuesta del servidor (HTML, CSS, JavaScript, JSON, imágenes); (3) **interpretar y renderizar** el código HTML/CSS construyendo el árbol DOM para mostrar la interfaz visual al usuario; y (4) ejecutar el código JavaScript del lado del cliente y validar formularios en el frontend.
2. **¿Qué es una URL y qué partes tiene?**
   - **URL (*Uniform Resource Locator*):** Es el identificador o dirección única que permite localizar un recurso en Internet.
   - **Partes de una URL** (ej. `https://turnos.petcare.com.ar:443/api/mascotas?id=10#historial`):
   - **Protocolo o Esquema:** `https://` (o `http://`).
   - **Subdominio:** `turnos.`
   - **Dominio (y TLD):** `petcare.com.ar`
   - **Puerto *(opcional)*:** `:443` (por defecto `80` en HTTP y `443` en HTTPS).
   - **Ruta (*Path*):** `/api/mascotas` (ubicación lógica del recurso o endpoint en el servidor).
   - **Parámetros de consulta (*Query String*):** `?id=10` (pares `clave=valor` separados por `&`).
   - **Fragmento o Ancla (*Anchor*):** `#historial` .
3. **¿Qué es una petición HTTP (*HTTP Request*)?**
   - Es un mensaje estructurado de texto enviado por el cliente (navegador o aplicación) hacia un servidor siguiendo el protocolo **HTTP (*HyperText Transfer Protocol*)** para solicitar una acción sobre un recurso. Consta de: (1) **Línea de petición** (Método HTTP + URI + versión), (2) **Cabeceras (*Headers*)** con metadatos (`Content-Type` , `Authorization` , `Accept`), y (3) **Cuerpo (*Body* opcional)** con los datos enviados (ej. un JSON en un `POST`).
4. **¿Cuáles son los métodos (verbos) HTTP más comunes?**
   - `GET` : Solicita recuperar/consultar la representación de un recurso (no modifica datos en el servidor; es idempotente).
   - `POST` : Envía datos en el cuerpo de la petición para **crear** un nuevo recurso o disparar un procesamiento en el servidor.
   - `PUT` : Actualiza o reemplaza **íntegramente** un recurso existente.
   - `PATCH` : Actualiza **parcialmente** algunos campos de un recurso existente.
   - `DELETE` : Elimina un recurso específico en el servidor.
5. **¿Qué es un recurso en la Web?**
   - Es cualquier entidad de información o servicio identificable mediante una URI/URL a la que se puede acceder o manipular a través de la web (una página HTML, una imagen, un documento PDF o una entidad de negocio expuesta por una API REST como `/mascotas/10`).
6. **¿Qué es un dominio y qué es un subdominio?**
   - **Dominio:** Es el nombre único y legible por humanos (ej. `unlam.edu.ar` o `petcare.com`) que traduce una dirección IP numérica de un servidor a través del sistema **DNS (*Domain Name System*)**.
   - **Subdominio:** Es una subdivisión o prefijo jerárquico antepuesto al dominio principal para organizar distintas secciones o servicios independientes del sitio (ej. `miel.unlam.edu.ar` , `api.petcare.com` , `turnos.petcare.com`).
7. **¿Qué función cumple un Servidor HTTP (*Web Server*)?**
   - Escucha permanentemente peticiones entrantes en los puertos HTTP/HTTPS (`80` / `443`), procesa la solicitud y devuelve como respuesta los recursos estáticos solicitados (archivos HTML, CSS, JS, imágenes) o actúa como proxy inverso redirigiendo las peticiones dinámicas al Servidor de Aplicaciones, junto con el **código de estado HTTP** (`200 OK` , `201 Created` , `400 Bad Request` , `401 Unauthorized` , `404 Not Found` , `500 Internal Server Error`).
8. **¿Qué es el *Hosting* (Alojamiento Web)?**
   - Es el servicio de infraestructura (física o en la nube) que provee servidores conectados a Internet las 24 horas para almacenar los archivos, bases de datos y artefactos de una aplicación web para que sean accesibles públicamente.
9. **¿Qué es un *Application Server* (Servidor de Aplicaciones), dónde se aloja, para qué sirve y qué softwares son los más comunes?**
   - Es el entorno de ejecución de software (*nodo de ejecución en el Diagrama de Despliegue*) alojado en el backend/datacenter que se encarga de **ejecutar la lógica de negocio (nuestros Controladores y Clases del Dominio)**, gestionar transacciones, seguridad, pools de conexiones a la Base de Datos y exponer APIs dinámicas.
   - **Softwares más comunes:** Apache Tomcat, WildFly / JBoss, GlassFish (en Java EE / Spring), IIS / Kestrel (.NET), Node.js / Express, Gunicorn / Uvicorn (Python).
10. **¿En qué se diferencia un *Web Server* de un *Application Server*?**

- El **Web Server (Servidor Web, ej. Apache HTTP Server, Nginx)** está optimizado para servir **contenido estático** (HTML, CSS, imágenes) por protocolo HTTP y balancear carga; mientras que el **Application Server (Servidor de Aplicaciones)** ejecuta **código de negocio dinámico** en el backend, procesa reglas transaccionales y se comunica con la base de datos y sistemas externos.

11. **¿Qué es HTML y qué son las etiquetas (*Tags*)?**
    - **HTML (*HyperText Markup Language*):** Es el lenguaje de marcado estándar utilizado para definir la **estructura semántica y el contenido** de una página web (no es un lenguaje de programación).
    - **Etiquetas (*Tags*):** Son los delimitadores entre corchetes angulares (`<etiqueta>...</etiqueta>`) que le indican al navegador qué representa cada bloque de contenido (`<h1>` , `<p>` , `<table>` , `<form>` , `<input>` , `<button>`).
12. **¿Qué significa que una web sea *Responsive* (Diseño Web Adaptable)?**
    - Significa que el diseño y la disposición visual de las páginas se **adaptan automáticamente al tamaño y resolución de cualquier pantalla o dispositivo** (computadora de escritorio, notebook, tablet, teléfono móvil) mediante grillas flexibles y *Media Queries* de CSS, garantizando una buena **Usabilidad (UX)** sin necesidad de hacer zoom horizontal.
13. **¿Para qué sirve la etiqueta `<form>` en HTML?**
    - Define un **formulario interactivo** que agrupa controles de entrada de datos para que el usuario ingrese información y la envíe hacia un endpoint del servidor (definido en el atributo `action` mediante el método HTTP indicado en `method="POST"` o `"GET"`), disparando el evento que recibe nuestro `Controller` (`C_...`).
14. **¿Para qué sirve la etiqueta `<input>` y cuáles son sus tipos más comunes?**
    - Es el elemento de formulario que permite capturar un dato puntual ingresado por el usuario.
    - **Tipos de `<input>` más comunes (`type="..."`):** `text` (texto simple), `password` (contraseña oculta), `number` (numérico), `email` (valida formato de correo), `date` (selector de fecha), `checkbox` (casillas de selección múltiple), `radio` (opción excluyente), `file` (adjuntar archivo/foto), `submit` (botón de envío del formulario).
15. **¿Qué significa *Renderizar* páginas web?**
    - Es el proceso mediante el cual el motor del navegador transforma el código fuente (HTML, CSS y JS) en la **representación gráfica e interactiva en píxeles** que el usuario ve en su pantalla.
16. **¿Qué es una API y para qué sirve en una aplicación web (Respuestas, Datos y Formatos)?**
    - Una **API (*Application Programming Interface*)** es un contrato de comunicación que expone un conjunto de **endpoints (URLs)** para que el Frontend (o un sistema externo) pueda solicitar servicios o intercambiar datos con el Backend sin conocer su implementación interna.
    - **Respuestas, Datos y Formatos:** Cuando el cliente invoca un endpoint de la API (por ejemplo en arquitectura **REST**), la API devuelve un **código de estado HTTP** y los **datos estructurados** serializados en formatos livianos e independientes del lenguaje, principalmente **JSON (*JavaScript Object Notation*)** o **XML** (como vimos en las integraciones con VISA, Veraz, ARCA y sistemas externos en el Diagrama de Despliegue y en el patrón GoF **Adapter**).

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 9: CATÁLOGO DE ERRORES FRECUENTES Y ELIMINATORIOS EN EL PARCIAL

📑 **ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO**

9.1. LOS 15 ERRORES MÁS FRECUENTES Y ELIMINATORIOS EN EL PARCIAL (Y CÓMO EVITARLOS) Pág. 56

9.1. LOS 15 ERRORES MÁS FRECUENTES Y ELIMINATORIOS EN EL PARCIAL (Y CÓMO EVITARLOS)

🎓 **CRITERIO Y RECOMENDACIÓN DE LA PROFESORA**

[CRITERIO DE CORRECCIÓN — CLASE 05, CLASE 07 Y CLASE 09]: Esta tabla resume exactamente los errores por los cuales la Ing. Analía Marín y el JTP Alejandro bajan puntos o desaprueban un ejercicio en el parcial práctico.

#	Error Frecuente Detectado en Hojas de Alumnos	¿Por qué está MAL para la Profesora?	Cómo Hacerlo BIEN en tu Hoja de Examen
1	**Falta de trazabilidad Secuencia ↔ Clases:** Enviar el mensaje `validarVacuna()` hacia `:HistoriaClinica` en el Diagrama de Secuencia, pero olvidarse de escribir `+validarVacuna()` en la clase `HistoriaClinica` del Diagrama de Clases.	**Es el Error Eliminatorio N° 1.** Las vistas de Kruchten deben ser 100% consistentes. Si el mensaje no existe como método en la clase receptora, el código no compila.	Dejá abierto el borde inferior de las cajas del Diagrama de Clases mientras dibujás el Diagrama de Secuencia y copiá cada mensaje inmediatamente en la clase receptora.
2	**Poner el método en la clase que ENVÍA el mensaje en vez de en la clase que lo RECIBE.**	Si `A` llama a `B.calcular()`, el método `+calcular()` pertenece a `B`, no a `A`.	Mirá siempre la **punta de la flecha** en el Diagrama de Secuencia: la línea de vida a la que llega la punta de la flecha es la dueña del método.
3	**Convertir al Actor humano (`Veterinario`, `Recepcionista`, `Secretaria`) en una clase Gestora llena de métodos del sistema.**	El actor humano está afuera del software apretando botones en la UI. Si le ponés `+guardarConsulta()`, `+emitirReceta()`, violás **Experto en Información** y **Alta Cohesión**.	El evento entra directo al `Controller` (`C_...`), y el Controller delega en `HistoriaClinica` y `Consulta`. La clase `Profesional` / `Veterinario` solo guarda sus datos (`matricula`, `nombre`, `horarios`) asociada al `Turno` o `Consulta`.
4	**Confundir "la Ficha individual" con "el Fichero (Colección)":** Poner `+buscarMascota(id)` adentro de la clase `Mascota` o `+buscarConsultasAnteriores()` adentro de `Consulta`.	Una instancia de `Mascota` es *un solo animal* (`Homero`); no conoce a todas las demás mascotas de la veterinaria.	Para buscar entre todas las instancias globales usá una clase contenedora `NominaMascotas` o `Padron` (unida con **asociación simple** ——). Para las consultas de una mascota, usá su contenedor de dominio `HistoriaClinica`.
5	**Unir la `Nomina` con la clase entidad mediante Composición fuerte (◆).**	La `Nomina` es una Fabricación Pura para búsqueda/listado; no es dueña física exclusiva de la vida de la entidad en el dominio.	Uní siempre `Nomina` con la entidad mediante **Asociación simple (——, de `1` a `0..*`)** (`Clase 09 [02:38:03]`).
6	**Llenar el Diagrama de Clases de `alta()`, `baja()`, `modificacion()`, `get...()`, `set...()` o poner 10 Controllers de ABM.**	En los parciales de Diseño **NO se evalúan ABMs triviales**, sino el **CORE del negocio**. Los getters/setters masivos ensucian el diagrama.	Usá como máximo **4 Controladores** (uno por módulo principal) y escribí métodos con nombres de reglas de negocio (`+guardarConsulta()`, `+calcularNivelPeligrosidad()`, `+realizarPago()`).
7	**Dibujar al Actor (stickman) en el Diagrama de Secuencia de Diseño o olvidarse de escribir el Escenario arriba.**	En Diseño el backend es agnóstico de la interfaz (MVC). Y sin escenario escrito arriba no se sabe qué camino concreto estás secuenciando.	Escribí un recuadro de **Escenario concreto** arriba de la Hoja 2 e iniciá la secuencia con un mensaje entrante directo hacia `:C_Controller`.
8	**Tirar líneas de asociación sólidas desde el `Controller` hacia las 20 clases del Diagrama de Clases.**	Genera una "telaraña" ilegible y conceptualmente falsa: el Controller solo coordina las clases raíz/nóminas y usa referencias locales temporales.	Recordá el cuadro de la Pág. 19 de `EJEMPLO-Practica...`: **la Visibilidad Local NO aparece graficada en el Diagrama de Clases**. Conectá el Controller solo con las 2 o 3 clases raíz/nóminas que coordina.
9	**Justificar los patrones GRASP con definiciones teóricas de libro sin nombrar las clases ni métodos de tu diagrama.**	Escribir "*Experto asigna la responsabilidad al que tiene la información*" sin decir en qué clase y método lo aplicaste equivale a 0 puntos.	Usá siempre la fórmula: **Patrón GRASP + Clase exacta de tu DC + Método exacto + Atributos/colección por los cuales esa clase es experta o creadora**.
10	**Elegir 2 patrones GoF de la MISMA categoría (ej. `Strategy` + `Observer`, ambos de Comportamiento) o usar `Singleton` / `Facade`.**	La consigna exige explícitamente **2 patrones GoF de DIFERENTE categoría** y prohíbe `Singleton` y `Facade`.	Combiná siempre **1 de Comportamiento (`Strategy`, `State` u `Observer`)** con **1 Estructural (`Adapter`, `Composite` o `Decorator`)** o **1 Creacional (`Builder`, `Factory Method` o `Prototype`)**.
11	**Nombrar un patrón GoF en la hoja escrita pero no dibujar sus interfaces y clases concretas en el Diagrama de Clases.**	El patrón GoF debe existir estructuralmente en el Diagrama de Clases.	Dibujá siempre la interfaz `<<interface>>` y al menos 2 clases concretas que la implementen (`- - -▷`), conectadas a la clase del dominio.
12	**En el Diagrama de Estados, mezclar estados de distintas clases o poner verbos de acción adentro de las cajas de estado.**	El Diagrama de Estados modela el ciclo de vida de **1 solo objeto**, y los estados son situaciones duraderas (adjetivos/participios), no acciones.	Adentro de la caja poné participios (`Registrada`, `Diagnosticada`, `Cerrada`, `Pendiente de pago`, `Confirmada`); sobre la flecha poné el método/evento (`registrarDiagnostico() [guarda] / acción`).
13	**Ignorar el Diagrama de Estados o de Componentes que viene impreso en la hoja del parcial.**	En los 3 parciales de 2026 vienen diagramas impresos para integrarlos o mejorarlos. Si tu clase `Consulta` o `Reserva` no tiene los métodos del Diagrama de Estados impreso, pierdes puntos por inconsistencia.	Copiá literalmente todos los nombres de eventos de las flechas del Diagrama de Estados impreso como métodos públicos (`+`) de esa clase en tu Diagrama de Clases.
14	**Poner el rombo de Agregación (◇) o Composición (◆) del lado de la Parte en lugar del lado del Todo.**	Invierte por completo el significado semántico de la relación (diría que un `ItemFactura` está compuesto por muchas `Facturas`).	Dibujá **SIEMPRE el rombo pegado a la clase contenedora (el Todo)** con multiplicidad `1` en el extremo del rombo negro (`Factura 1 ◆—— 1..* ItemFactura`).
15	**Entregar varios diagramas amontonados en la misma carilla o sin numerar las hojas.**	Incumple la regla formal impresa en mayúsculas en todos los parciales: "*ENTREGAR UN DIAGRAMA POR HOJA, ENUMERAR LAS HOJAS A ENTREGAR*".	Usá hojas separadas y numeradas (`Hoja 1 de 4: Diagrama de Clases y GRASP`, `Hoja 2 de 4: Escenario y Diagrama de Secuencia`, etc.).

GUÍA MAESTRA DE DISEÑO DE SISTEMAS — CÁTEDRA ING. ANALÍA MARÍN

CAPÍTULO 10: CONSEJOS FINALES Y CHECKLISTS PARA EL EXAMEN

📑 ÍNDICE DE CONTENIDOS DE ESTE CAPÍTULO

10.1. LOS 12 MANDAMIENTOS DE ANALÍA MARÍN (CHECKLIST PRE-ENTREGA EN EL AULA) Pág. 57
10.2. ESTRATEGIA DE DISTRIBUCIÓN DEL TIEMPO EN EL EXAMEN (120 MINUTOS) Pág. 57

10.1. LOS 12 MANDAMIENTOS DE ANALÍA MARÍN (CHECKLIST PRE-ENTREGA EN EL AULA)

Reservá los **últimos 10 minutos del examen** para tildar uno por uno estos 12 puntos sobre tus hojas antes de entregar:

- [ ] **1. Hojas separadas y numeradas:** ¿Entregué **un diagrama por hoja** y numeré todas las hojas (`Hoja 1 de X`, `Hoja 2 de X`) con mi nombre y DNI?
- [ ] **2. Tope de Controladores y Orden Visual:** ¿Ubiqué en la franja superior del Diagrama de Clases como máximo **4 Controladores** (`C_...`) y están alineados 1 a 1 con los componentes principales del Diagrama de Componentes?
- [ ] **3. Cero Actores como Gestores:** ¿Verifiqué que ningún actor humano (`Veterinario`, `Recepcionista`, `Ciudadano`) tenga métodos de coordinación del sistema?
- [ ] **4. Cero ABMs triviales y Cero Getters/Setters masivos:** ¿Todos los métodos de mis clases tienen nombres de **reglas de negocio del CORE**?
- [ ] **5. Ficha vs. Fichero (`Nomina`):** ¿Separé la clase individual (`Mascota`, `Aficionado`, `Denuncia`) de quien administra la colección (`HistoriaClinica` para la colección propia de una mascota, o `Nomina...` / `Padron` con **asociación simple** —— para búsquedas globales)?
- [ ] **6. Visibilidades y Multiplicidades:** ¿Puse `-` en todos los atributos, `+` en los métodos públicos, `#` en los atributos de superclases abstractas, y multiplicidades (`1`, `0..1`, `0..*`, `1..*`) en todos los extremos de asociación (sin dibujar visibilidades locales en el DC)?
- [ ] **7. Estereotipos Explícitos:** ¿Escribí `<<interface>>` y `<<abstract>>` arriba del nombre de cada interfaz y clase abstracta, diferenciando herencia (—▷ línea continua) de realización (`- - -▷` línea punteada)?
- [ ] **8. Integración del Diagrama de Estados:** Si el parcial trajo impreso un Diagrama de Estados (ej. `Consulta` o `Reserva`), ¿incluí todos los eventos de sus flechas como métodos de esa clase en el Diagrama de Clases?
- [ ] **9. Escenario Concreto Arriba de Secuencia:** ¿Escribí el párrafo del **Escenario Concreto** con datos puntuales arriba del Diagrama de Secuencia?
- [ ] **10. Secuencia SIN Actor y con `create()`:** ¿Inicié el Diagrama de Secuencia directamente con el mensaje entrante hacia `:C_Controller` (sin dibujar al actor humano) y usé `create(...)` para los objetos que nacen en esa corrida?
- [ ] **11. Trazabilidad 100% Secuencia ↔ Clases:** ¿Controlé flecha por flecha que **todos los mensajes del Diagrama de Secuencia estén escritos como métodos en la clase receptora del Diagrama de Clases**?
- [ ] **12. Patrones GRASP y GoF Completos:** ¿Justifiqué los GRASP nombrando las clases y métodos exactos de mi diagrama, y elegí **2 patrones GoF de DIFERENTE categoría (sin usar Singleton ni Facade)** dibujándolos en el Diagrama de Clases y respondiendo los incisos `a)`, `b)`, `c)` y `d)`?

10.2. ESTRATEGIA DE DISTRIBUCIÓN DEL TIEMPO EN EL EXAMEN (120 MINUTOS)

Franja Horaria	Actividad Exacta	Entregable que queda listo
Min 00 – 15	Lectura completa del enunciado con código de 3 colores (`Amarillo` = Clases, `Verde` = Verbos/Métodos, `Celeste` = Atributos). Identificación de los **4 Controladores** y elección de los **2 Patrones GoF de distinta categoría**.	Lista de clases por módulo, los 4 `C_...` y los 2 patrones GoF elegidos.
Min 15 – 45	Dibujo del **Diagrama de Clases** en **Hoja 1** (dejando abierto el borde inferior del compartimento de métodos de cada clase) + integración de los métodos del Diagrama de Estados impreso + dibujo de las clases/interfaces de los **2 Patrones GoF**.	Esqueleto completo del Diagrama de Clases en Hoja 1.
Min 45 – 75	Redacción del **Escenario Concreto** y construcción del **Diagrama de Secuencia** en **Hoja 2**, copiando en simultáneo cada mensaje nuevo dentro del compartimento de métodos de la clase receptora en la Hoja 1.	**Hoja 2 (Secuencia)** terminada y **Hoja 1 (Clases)** 100% consistente y cerrada.
Min 75 – 95	Si el parcial pide modificar o mejorar el **Diagrama de Componentes**, dibujarlo en su hoja dedicada alineado con los 4 Controladores. Redactar los incisos `a)`, `b)`, `c)` y `d)` de los **2 Patrones GoF**.	**Hoja de Componentes** y **Hoja de Justificación GoF** terminadas.
Min 95 – 110	Redacción de las justificaciones de los **Patrones GRASP** (Experto, Creador, Bajo Acoplamiento, Alta Cohesión, Controlador) citando clases y métodos exactos de la Hoja 1.	**Justificaciones GRASP** completas.
Min 110 – 120	Repaso final con el **Checklist de los 12 Mandamientos**, control cruzado de métodos y numeración de todas las hojas.	Examen listo para entregar con estándar de 10 puntos.